The Continuous Tensor Abstraction and Its High Performance Compilation

by

Jaeyeon Won

B.S, Seoul National University, 2020.
S.M, Massachusetts Institute of Technology, 2023.

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

September 2026

Authored by: Jaeyeon Won
Department of Electrical Engineering and Computer Science
August 14, 2026

Certified by: Saman Amarasinghe
Professor of Electrical Engineering and Computer Science
Thesis Supervisor

Certified by: Joel S. Emer
Professor of the Practice of Electrical Engineering and Computer Science
Thesis Supervisor

Accepted by: Leslie A. Kolodziejski
Professor of Electrical Engineering and Computer Science
Chair, Department Committee on Graduate Students

THESIS COMMITTEE

Thesis Supervisors

Saman Amarasinghe
Professor of Electrical Engineering and Computer Science
Department of Electrical Engineering and Computer Science

Joel S. Emer
Professor of the Practice of Electrical Engineering and Computer Science
Department of Electrical Engineering and Computer Science

Thesis Reader

Fredrik Berg Kjølstad
Assistant Professor of Computer Science at Stanford University
Department of Computer Science at Stanford University

The Continuous Tensor Abstraction and Its High Performance Compilation

by

Jaeyeon Won

Submitted to the Department of Electrical Engineering and Computer Science on August 14, 2026 in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

ABSTRACT

Tensor programming has become a standard abstraction for expressing and optimizing dense and sparse computations. However, existing tensor systems are built around integer coordinate domains, which makes it difficult to express applications whose data and computation are defined over real-valued coordinates, such as point clouds, genomic intervals, and spatial queries. As a result, these workloads are scattered across fragmented libraries, each with its own hand-written kernels and data structures.

This thesis introduces the continuous tensor abstraction, which extends tensor programming to real-valued domains. Importantly, continuous tensors use the same language as tensors over integer coordinates, and many algorithms can be expressed using the same tensor expressions across both domains; for example, convolution can be written using the same expression whether operating over pixels on an integer grid or point clouds in a real-valued coordinate space. Under a piecewise-constant assumption, continuous tensors can be represented finitely, reduced over continuous regions, and compiled into executable code. On top of this abstraction, the thesis develops format-aware reformulation: a program rewrite that turns continuous tensor programs into array operations with the format information embedded in the expression, so that highly optimized dense tensor compiler infrastructure, including tensor cores on GPUs, can execute them efficiently. Together, these contributions broaden tensor programming beyond integer coordinate domains, allowing the same tensor programming model to express and efficiently execute applications over both discrete and continuous domains.

Thesis supervisor: Saman Amarasinghe
Title: Professor of Electrical Engineering and Computer Science

Thesis supervisor: Joel S. Emer
Title: Professor of the Practice of Electrical Engineering and Computer Science

Acknowledgments

I am deeply grateful to my advisors, Saman Amarasinghe and Joel Emer, for their guidance and support throughout my PhD.

Saman taught me how to find important research problems and to think beyond simply solving problems. Throughout my PhD, Saman often challenged me to ask whether a problem was truly worth solving, who would benefit from the solution, and why the problem mattered. These questions fundamentally changed how I approach research and helped me grow into a more thoughtful researcher. I may not always know the right problem and answer, but I feel that I've learned how to think in the right direction.

Joel taught me the importance of abstraction and taxonomy. In our meetings, we often organized complex research ideas into simple abstractions. More importantly, the experience of designing these abstractions together taught me how much careful thought goes into finding the right way to frame a problem. Many of the ideas in this thesis were shaped by this way of thinking. I am grateful to Joel for teaching me how the right abstraction can reveal both what is already known and what remains unexplored.

I also thank Fredrik Kjolstad for serving on my thesis committee. Much of my research builds on foundations established by Fred's work on sparse tensor compilation. Coming from a computer architecture background, TACO was one of the first compiler systems I studied in depth, and it strongly influenced how I came to think about tensor compilers.

I was fortunate to work with many excellent collaborators. Charith introduced me to machine-learning-based autotuning through our work on WACO. Willow helped establish the foundation of continuous tensors and built its first implementation in Finch. Thea played an important role in developing the mathematical foundations of continuous tensors. Changwan helped me implementing sparse convolutions during Unified Convolution Framework project. I also thank Mit for suggesting equivariant tensor products as an application for the Insum project and for working with me on high-performance GPU kernels. I am grateful to all current and former members of the MIT COMMIT group, including Ajay, Tom, Changwan, Caleb, Runsheng, Manya, Daniel, and Logan, for making the group such an enjoyable and intellectually stimulating environment.

I would also like to thank Prof. Jae W. Lee, my undergraduate advisor at Seoul National University. He showed me how exciting the research on high-performance system could be, and inspired me to pursue graduate school and continue doing research in this area.

Outside of research, I was fortunate to have many close friends who made my years at MIT much more enjoyable. I especially thank Eunseok, Jaekang, Kyounghun, and Joonhyuk for their friendship and for the countless conversations and memories we shared. I am also grateful to the friends I met through tennis, especially Seowoo, for the many matches and tournaments we played together.

Most importantly, I thank my family. My parents have always trusted and supported my decisions, even when the path I chose may not have been obvious to them. Their encouragement gave me the confidence to take risks, pursue difficult challenges, and not be afraid of failure. I am deeply grateful for everything they have done for me. Finally, I thank my girlfriend, Jimin, for being my best friend throughout my PhD.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Tensors[1] are the central data representation in deep learning [1,2], scientific computing [3], linear algebra, and many other computational domains. Tensor programming provides a high-level abstraction for expressing computations over such data.

Throughout this thesis, a *tensor* is a multidimensional logical mapping from coordinate tuples to values, independent of how those values are represented in memory. We call each dimension of the tensor a *tensor rank*; other literature may call it a dimension, axis, or mode. A tensor's data may be *dense* or *sparse* depending on the distribution of its nonempty values; these are properties of the data rather than its representation. To distinguish the logical tensor from physical storage, we use *array* to mean a concrete, contiguous region of linear memory. In the simplest representation, one array stores the tensor, and a linear layout maps every multidimensional tensor point to a location in that array. Other formats may use different representations while encoding the same logical tensor.

An *Einsum* is a concise notation for expressing a tensor computation. For example, $C_{x,y} = A_{x,z} \times B_{z,y}$ is the matrix-matrix multiplication of tensors A and B to form the tensor C. We call the variables in its tensor subscripts *rank variables*. The ranges of these variables define the *iteration space*, which contains every possible combination of their values.

[1] Currently, "tensor programming" is often used interchangeably with "array programming," where "tensor" denotes a multidimensional array. For this thesis, we adopt the term "tensor programming."

Tensor programming has a long history, with its origins dating back to FORTRAN's introduction [4] in 1957 and to early array languages such as APL [5] and MATLAB [6]. Tensors and related data structures subsequently became ubiquitous in general-purpose languages, including C [7], C++ [8], Python [9], and Java [10]. Modern tensor programming builds on this abstraction by making operations over multidimensional arrays a primary unit of expression and optimization. Today, languages, libraries, and compilers such as NumPy [11], Halide [12], TVM [13], and JAX [14] allow programmers to describe computations using high-level tensor expressions while the system generates efficient code.

Tensor computations constitute a substantial and rapidly growing fraction of modern high-performance computing. GPU-accelerated AI servers alone accounted for more than 40% of U.S. data-center server electricity consumption in 2023, as inferred from their consumption of more than 40 TWh out of nearly 100 TWh consumed by all servers [15]. Meanwhile, the principal benchmarks used to rank and evaluate supercomputers are composed of kernels over dense and sparse tensors: The HPL benchmark solves dense systems of linear equations [16], whereas the HPCG benchmark exercises sparse matrix–vector multiplication, vector operations, dot products, and sparse triangular solves [17].

More recent tensor compilers extend this framework beyond arrays, exploiting data properties such as sparsity and symmetry. The Tensor Contraction Engine [3] exploits symmetry in quantum chemistry tensors, while TACO [18,19] and Finch [20,21] support more complex data representations for sparse tensors by compressing out zeros. Together, these systems show that tensor programming can separate the mathematical computation from the physical representation of the data: the user writes a tensor expression, specifies how tensors are stored, and relies on the compiler to generate code that traverses the representation efficiently.

Despite this progress, this thesis identifies two gaps in the tensor programming landscape: an *expressiveness gap*, where still a lot of important applications cannot be written as tensor programs at all, and a *performance gap*, where programs that can be written as tensor programs cannot be compiled to efficient code on modern hardware.

The Expressiveness Gap of Existing Abstractions

Existing tensor programming systems are built around tensors whose coordinates are discrete integers. This abstraction works well for dense linear algebra, stencils, convolutions, and many sparse matrix computations, but it does not naturally capture workloads whose data and computation are defined over continuous coordinates.

Such workloads arise throughout spatial, graphics, geometric, and scientific computing. Point clouds, geometric regions, genomic intervals, and spatial databases operate over positions, intervals, boxes, circles, and other geometries defined over real-valued domains. Many of their computations are simple to describe at a high level: intersect geometric objects, count overlapping regions, query points inside a spatial domain, or aggregate (integrate) values over a continuous region. However, traditional tensor programming does not allow tensor coordinates to range over real-valued spaces. As a result, these applications are typically implemented in domain-specific or hand-written libraries, such as Bedtools [22] for genomic interval processing or TorchSparse [23] for point-cloud operations, rather than in a unified abstraction.

This restriction to integer coordinates, however, is a convention rather than a fundamental requirement of tensor programming. It rests on three conventional assumptions built into tensor programming systems. While prior works have implicitly questioned the first two, this thesis challenges all three.

1. **Computations are performed at all points in the *iteration space***—all tuples of coordinate assignments to the Einsum's rank variables that lie within the tensors' shapes. This includes both *effectual* computations (those contributing to the final output) and *ineffectual* computations (those that do not affect the output, e.g., `out += in * 0`). Like other sparse compilers [18,21], this thesis breaks this assumption at the implementation level by computing only at effectual points and skipping ineffectual points

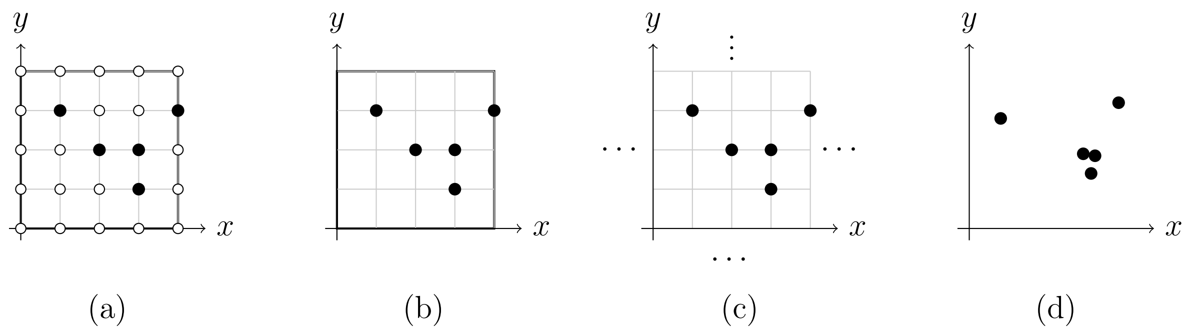


Figure 1.1: Breaking the conventional assumptions of tensor programming. (a) The traditional iteration space is an integer grid bound by tensor shapes, with computations defined at every point. (b) Breaking the first assumption, implementations can skip ineffectual computations and visit only effectual points. (c) Once only effectual points are visited, the iteration space need not have a finite bound or shape and can extend over an unbounded space. (d) Because effectual points are specified by explicit coordinates, those coordinates need not be integers and can instead lie in a continuous real domain.

without sacrificing correctness.

2. **The iteration space is finite**, because every iteration-space point must project to a tuple of valid coordinates in each accessed tensor. Traditionally, computations at out-of-bounds coordinates were considered invalid or caused errors. By conceptualizing tensors as implicitly holding empty values (such as zero for multiplication) at out-of-bounds coordinates, the iteration space can be extended—even to infinity—because computations at these coordinates are ineffectual and can be skipped. Some prior tensor compilers, such as Halide [12] and Finch [21], implicitly break this assumption, and this thesis relies on breaking it as well. Breaking this assumption allows users to write clean expressions over unbounded spaces without explicit boundary checks, enabling this thesis to extend iteration spaces seamlessly to continuous domains.

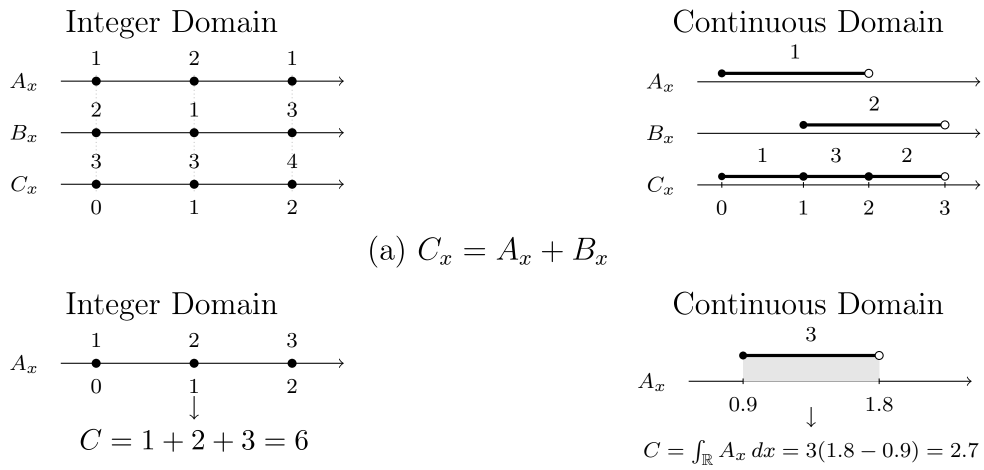
3. **The iteration space is an integer lattice.** Traditionally, the points in an N-dimensional iteration space are N-tuples of discrete integer coordinates. But once the iteration space can be infinite, its domain does not have to be integers either: coordinates can be real numbers, and the iteration space can be continuous rather than discrete, with effectual computations occurring at real-valued coordinates. Prior

systems leave this assumption intact; relaxing this assumption is the central point of this thesis.

This Thesis: The Continuous Tensor Abstraction This thesis generalizes the tensor abstraction itself from discrete to continuous domains. A tensor becomes a mapping from real-valued coordinates to values, rather than from integer coordinates: a point cloud is a three-dimensional tensor $A_{x,y,z}$ that holds a feature value at each occupied position $(x, y, z) \in \mathbb{R}^3$ and zero everywhere else, and a set of genomic intervals is a one-dimensional tensor that is one inside each interval and zero outside. The iteration space generalizes accordingly: instead of a finite integer lattice, it is a continuous, infinite space of real-valued points, and computation is now defined over that space.

Beyond providing a tensor defined on the real domain, this new abstraction demonstrates that familiar tensor notation extends naturally to the continuous setting. Users can write Einsums that concisely express tensor operations based on their underlying mathematical definitions such as $C_x = A_x \times B_x$ or $C_{x,y} = A_{x,z} \times B_{z,y}$, exactly as they would in the discrete setting, except that rank variables now range over the reals and reduction over a rank variable becomes integration over a continuous domain. No new programming language is needed because the same notation that describes tensor algebra also describes computation over continuous tensors, so users carry their existing intuition directly into the continuous domain. On the storage side, instead of restricting tensor storage to arrays and conventional sparse formats, the format language describes the representations needed for continuous data. Under a *piecewise-constant* assumption, continuous tensors can be represented finitely in memory, reduced over continuous domains, and compiled into executable code.

Figure 1.2 illustrates how the same Einsums apply to both discrete and continuous domains. The addition $C_x = A_x + B_x$ remains pointwise in either domain. The reduction $C = A_x$, however, has different semantics: it sums values over coordinates for a tensor on an integer grid but integrates values over real-valued regions for a continuous tensor, weighting



(a) $C_x = A_x + B_x$

(b) $C = A_x$

Figure 1.2: Discrete and continuous vector operations in the same Einsum.

each value by the length of its interval.

The Performance Gap on Modern Hardware

Expressing a computation as a high-level tensor program is only useful if the compiler can produce low-level code that competes with hand-written implementations, and here sparse tensor programming faces a second gap. GPUs have become the hardware backbone of high-performance computing, yet implementing efficient sparse or irregular kernels on GPUs is notoriously difficult. Unlike dense computations, these kernels require indirect memory accesses and irregular data dependencies, and developers must manage a wide range of GPU-specific concerns: shared memory utilization, load balancing, vectorization, memory coalescing, and exploiting Tensor Cores. The resulting implementations are specialized and hard to maintain. TorchSparse [23], a state-of-the-art GPU sparse convolution library, comprises thousands of lines of CUDA, and its underlying strategies have generated a sequence of research papers, each addressing a specific performance challenge.

Sparse tensor compilers were designed to remove exactly this burden, but they have largely focused on sparse-sparse kernels and CPU code generation, where the complex control flow of coordinate intersection and union is the main focus. Many real-world GPU workloads are instead sparse-dense: only one operand is sparse, and the bulk of the work is dense computation. Existing sparse compilers fail to fully optimize these dense components, while dense tensor compilers cannot express the irregular access patterns from sparse components. Continuous tensor programs inherit this problem in an even sharper form: they are a new class of structured computation with no hand-optimized GPU libraries to fall back on.

This Thesis: Format-Aware Compilation of Continuous Tensor Programs. This problem suggests that current compilation strategies are too tied to particular formats and targets. This thesis argues that many applications with continuous data can be brought into tensor programming by compiling continuous tensor programs to efficient code on both CPUs and GPUs. On the compilation side, this thesis develops *format-aware computation*, a new code generation strategy for continuous tensor programs on GPUs. We first demonstrate format-aware computation in a simpler setting: a single sparse operand over a discrete domain. The key idea is to rewrite format-agnostic Einsums into indirect Einsums, format-conscious expressions such as $C_{AI_p} = AV_p \times B_{AI_p}$ that encode a format's data and metadata directly through indirect indexing, and to lower them onto a dense tensor compiler such as PyTorch. Because the dense backend supplies kernel fusion, tensor-core code generation, and GPU targeting, this strategy brings structured tensor programs to modern GPU hardware without hand-written kernels. We then extend format-aware computation to arbitrarily many sparse operands and to continuous domains by introducing the *mask-product-merge* strategy.

1.1 Contributions

This thesis makes the following contributions.

- **The continuous tensor abstraction.** To the best of our knowledge, this work is the first to extend the tensor programming model from integer coordinate tensors to tensors defined over real-valued domains. This abstraction brings applications involving continuous data, such as point-cloud processing, geospatial analytics, and interval-based computation, into the domain of tensor programming.

- **Pinpoints and Intervals.** We introduce pinpoints for values at isolated real coordinates and intervals for values associated with continuous regions. Together, they provide finite representations of point data and region-valued data in continuous tensors.

- **Continuous Einsums and Reductions.** We generalize Einsum-style notation to continuous iteration spaces, allowing familiar expressions such as $C_{x,y} = A_{x,z} \times B_{z,y}$ to be interpreted with real-valued rank variables. We introduce reductions such as integration over rank variables with continuous coordinate domains by using counting and Lebesgue measures. Consequently, programmers can carry their existing intuition about tensor algebra directly into the continuous domain.

- **A piecewise-constant representation for continuous tensors.** We introduce piecewise-constant tensors as a practical finite representation of continuous tensors. Tensor values are represented over a finite collection of subdomains, such as intervals holding constant values , making continuous data amenable to finite in-memory storage and computation. Building on this representation, we generalize the fibertree abstraction, a hierarchical tree representation for describing traditional tensor storage, to describe storage formats over continuous domains. Although this thesis focuses on piecewise-constant tensors, the continuous tensor abstraction itself can admit much broader set of representations and implementation strategies.

- **A compiler for continuous tensor programs.** We develop, to the best of our knowledge, the first compiler for continuous tensor programs. The compiler generalizes sparse tensor compilation from iteration over discrete pinpoints to iteration over piecewise-constant intervals and lowers continuous Einsums into executable code using the piecewise-constant assumption developed in this thesis.

- **A format-aware reformulation of format-agnostic Einsums.** We introduce format-aware computation, an implementation technique that embeds the data and metadata associated with sparse storage formats into array operations. This formulation makes the computation format-specific while enabling the reuse of dense tensor compiler infrastructure.

- **Fixed-length sparse formats.** We identify fixed-length sparse formats as a new class of sparse storage formats and introduce **GroupCOO** and **BlockGroupCOO**. These formats regularize sparse metadata and computation so that format-aware sparse operations can be expressed efficiently as indirect Einsums, an extension of Einsums that can have tensors as rank variable expressions to represent data-dependent access patterns [24].

- **Insum.** We develop Insum, a compiler that lowers indirect Einsums into high-performance GPU kernels by reusing dense tensor compiler infrastructure. Insum targets PyTorch FX graphs and extends the PyTorch compiler with native Tensor Core matrix multiplication support and a *lazy broadcasting* code-generation technique.

- **Format-aware compilation for multiple sparse operands and continuous domains.** We generalize format-aware computation from a single sparse operand over a discrete iteration domain to programs involving multiple sparse operands and continuous iteration domains. This extension lowers continuous tensor programs to GPU hardware using a mask–product–merge compilation strategy.

- **Empirical evaluations.** Our evaluations demonstrate the following:
  - The continuous tensor abstraction expresses applications from bioinformatics, geospatial analytics, 3D point-cloud processing, and Neural Radiance Fields while reducing implementation size by factors of approximately 18×, 62×, 101×, and 6×, respectively, relative to domain-specific implementations.

- The continuous tensor compiler, implemented by extending the Finch sparse tensor compiler, is competitive with hand-written, domain-specific libraries on CPU. Relative to these hand-written libraries, it achieves average speedups of 9.20× on radius-search queries, 1.22× on genomic interval-overlap queries, and 1.69× on trilinear interpolation for Neural Radiance Fields.
- Insum matches or outperforms specialized, hand-optimized GPU libraries, achieving speedups of 1.14×–3.81× across structured and unstructured sparse matrix multiplication, equivariant tensor products, and sparse convolution. At the same time, it replaces implementations containing 202–4,491 lines of code with a single indirect Einsum expression.
- The GPU implementation of format-aware continuous Einsums outperforms the Finch-based CPU implementation by up to 229× across the evaluated Einsum workloads.

1.2 Thesis Outline

The remainder of this thesis is organized as follows. Chapter 2 reviews background and related work on tensor programming, Einsums, the fibertree abstraction, sparse tensor formats, and the tensor compilers on which this thesis builds.

Chapter 3 and 4 present the continuous tensor abstraction, including continuous tensors, continuous reductions, and the generalized format language. It also introduces piecewise-constant tensors as the concrete finite representation used throughout this thesis.

Chapter 5 and 6 introduce format-aware computation for sparse tensors over discrete iteration domains. It presents indirect Einsums, fixed-length sparse formats, and the Insum compiler.

Chapter 7 and 8 generalize format-aware computation to multiple sparse operands and continuous iteration domains. It presents the mask–product–merge strategy for compiling continuous tensor programs to GPU hardware.

Finally, Chapter 9 presents related work, and Chapter 10 concludes the thesis and discusses directions for future work.

Chapter 2

Background

This chapter presents the background necessary to understand the rest of this thesis. We begin by defining a tensor and its associated terminology. We view a tensor as a logical mapping from a set of coordinates to values, independent of how that mapping is stored. We then turn to storage, introducing the fibertree abstraction and level formats, which describe how a logical tensor is realized as a concrete representation in memory. Next, we introduce the Einsum notation, the declarative language for tensor computation: we first define its terminology, give an operational definition of how an Einsum is evaluated over an iteration space, and survey syntactic extensions such as affine and indirect rank-variable expressions that broaden the class of expressible applications. Finally, we examine how existing systems implement Einsums over various formats, contrasting dense tensor (array) compilers with sparse tensor compilers and highlighting the complementary strengths and weaknesses of each. A common thread runs through all three discussions: every layer of this stack, such as the fibertree abstraction, the Einsum notation, and the implementation, assumes that tensor coordinates and iteration spaces live in the integer domain.

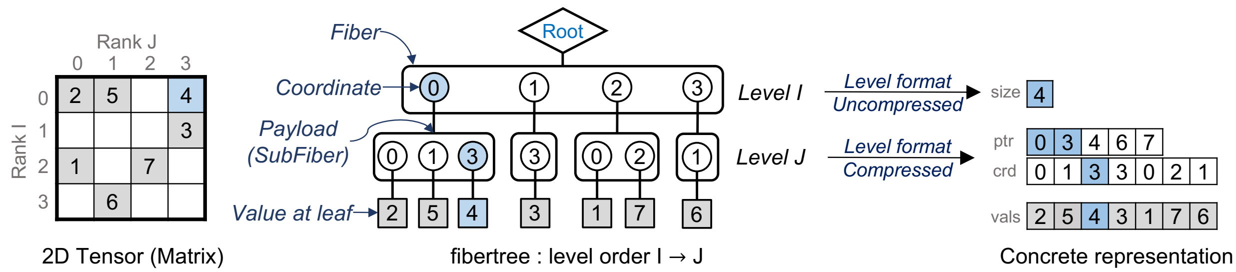


Figure 2.1: A sparse matrix, its fibertree abstraction, and its concrete representation in memory.

2.1 Format Abstraction

We define a tensor as a multidimensional logical mapping from tensor points to values, independent of its physical representation. A tensor point is a tuple containing one coordinate per tensor rank. What we call a *tensor rank* is also commonly called a dimension, axis, or mode. We use *rank* for logical tensors and their fibertrees. Indexing a tensor at a tensor point retrieves its value, or payload. A format language separately specifies how the tensor is represented in memory. Many format languages are based on the coordinate-tree model, which was introduced by TACO's format abstraction [19] and later generalized and formalized as the *fibertree* abstraction [25]. A tensor itself is a purely logical object and has no inherent storage format. For example, a matrix may be dense or sparse, while CSR and COO are storage formats. Thus, the role of the format abstraction is to connect this logical view of a tensor to a concrete representation in memory.

Definition of the fibertree The fibertree abstraction represents a tensor as a tree in which each *level* corresponds to one labeled rank of the tensor. The order of the levels gives the tensor's *rank order*. For example, a row-major matrix $A_{i,j}$ has rank order $[I, J]$, where the ranks appear from top to bottom in the tree.

Each level contains one or more *fibers*. A fiber is a set of coordinate/payload pairs whose entries share the same coordinates at all higher levels of the tree. Fibers are therefore more general than rows or columns, since the same definition applies naturally to tensors of any

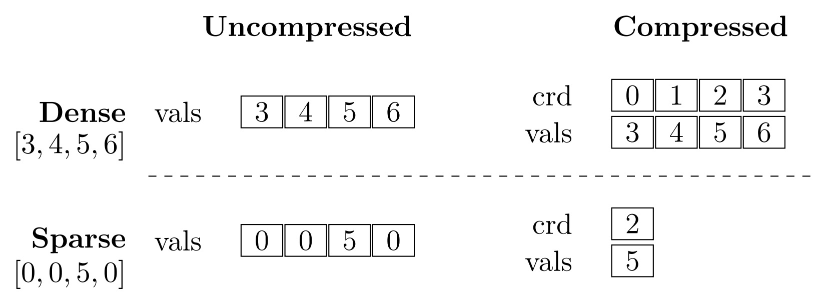


Figure 2.2: Dense and sparse vectors represented using Uncompressed and Compressed level formats. Dense and sparse describe properties of the logical tensor, whereas Uncompressed and Compressed describe its physical representation.

order. At an intermediate level, the payload of a coordinate is a reference to a fiber at the next level. At a leaf level, the payload is a scalar value.

The *shape* of a fiber is the set of legal values that its coordinates may take. When the shape is given by an integer N, it denotes the range $[0, N)$. The shape of a rank is the union of the shapes of all fibers at that rank, and the shape of a tensor is the list of its rank shapes in rank order.

Figure 2.1 illustrates the fibertree of a two-dimensional sparse matrix. One advantage of this abstraction is that it handles dense and sparse tensors in a uniform way. A fully dense tensor has a nonempty value at every coordinate in its shape, so its fibertree is complete. A sparse tensor may omit entries with empty payloads, where an empty payload is either a zero value or an empty fiber. The meaning of operations on fibers remains the same in both cases.

Level Formats A fibertree is an abstract description rather than a concrete memory layout. To store it in memory, each level is assigned a *level format*, which defines how the fibers at that level are represented.

Two common level formats for integer coordinates are *Uncompressed* and *Compressed*. An Uncompressed level represents the full range $[0, N)$ implicitly, so it does not need to store each coordinate. A Compressed level stores only the coordinates that are present, together with offset pointers that mark the boundaries of each fiber.

Figure 2.2 illustrates that sparsity and level format are distinct concepts. *Dense* and *sparse* describe properties of the data distribution, namely whether a tensor contains many or few nonempty values, whereas *Uncompressed* and *Compressed* describe how those values and their coordinates are represented in memory. Thus, a sparse tensor may use an Uncompressed level, potentially storing many empty values, while a dense tensor may use a Compressed level, redundantly storing every coordinate explicitly.

We call a representation an *array* when every tensor rank uses an Uncompressed level format. This is the conventional representation assumed by non-sparse tensor compilers, traditionally called dense tensor compilers.

Different combinations of rank orders and level formats recover familiar tensor formats. For example, consider a matrix stored in the $I \rightarrow J$ rank order. Using an Uncompressed format for the I level and a Compressed format for the J level gives the Compressed Sparse Row (CSR) format, as shown on the right side of Figure 2.1. We refer the reader to [25,26] for further details.

Coordinates and Positions A *coordinate* identifies a logical location in a tensor, whereas a *position* identifies an entry in its concrete memory representation. Thus, coordinates belong to the logical fibertree, while positions belong to the encoded data structure.

A level format defines how the coordinate space of each fiber is mapped to its position space. In a Compressed level, a position is an integer offset into the stored coordinate and payload arrays, and the offset array gives the range of positions belonging to each fiber. In an Uncompressed level, the coordinates are implicit, so a coordinate can be mapped directly to its position. This distinction allows tensor operations to be described in terms of logical coordinates while their implementations traverse concrete storage positions.

Tree Manipulation The fibertree abstraction also supports transformations that change the structure of the tree without changing the tensor's contents. *Rank flattening* combines two adjacent ranks into a single rank, whose coordinates are tuples formed from the coordinates of the original ranks. *Rank partitioning* splits one rank into two ranks, with each coordinate in the new upper rank denoting the first legal coordinate of the fiber below it. Finally, *rank swizzling* changes the rank order by reordering the levels of the tree [26].

Limitation Despite its generality, the fibertree abstraction is inherently discrete. Its coordinates identify individual logical points, although a coordinate may become a tuple after a transformation such as rank flattening. The common level formats make the same assumption: an Uncompressed level represents the full range of discrete coordinates, while a Compressed level stores an explicit list of discrete coordinates.

As a result, neither the existing abstraction nor the level formats directly represent tensors defined over continuous domains. For example, they cannot naturally represent a single value associated with the real interval $[0.5, 2.3)$ rather than with a set of individual discrete points.

2.2 Einstein Summation Notation and its Extension.

While the fibertree abstraction describes *how* each tensor is stored, Einsum notation describes tensor computations in a compact and expressive way. Tensor compilers [1,2,14,27,28] widely adopt a syntax based on Einstein summation notation (Einsums) [18,29,30]. In this notation, an operation is expressed over tensor accesses subscripted by *rank variables*; for example, matrix multiplication is written as $C_{m,n} = A_{m,k} \times B_{k,n}$. The coordinate domains of the rank variables define the iteration space of the computation. Reductions are implicit: if a rank variable appears in the input tensors but not in the output, the computation reduces over that rank variable. In the example above, the computation reduces over k, removing the need for an explicit summation symbol. While reduction typically implies summation, it is not limited to it; modern generalizations of the notation support arbitrary pointwise operators and reduction operators such as `min` and `max` [21,30,31]. Certain combinations of pointwise and reduction operators can also be viewed as computations over a *semiring*, as in GraphBLAS [32], although the Einsum abstraction itself does not require the operators to satisfy semiring properties.

Operational Definition. An Einsum can be understood operationally as a traversal over the *iteration space*: the set of points formed by assigning one coordinate to every rank variable [18,25]. This space is typically derived from the shapes of the tensor ranks appearing in the expression, but it does not always correspond directly to the shape of a tensor. Rank-variable expressions can further constrain the valid coordinates. For example, in $A_{i,i}$, the range of i is limited by the smaller of the two corresponding rank extents of A, while convolution expressions with affine accesses similarly restrict the coordinates over which all operand accesses are well-defined. Thus, the iteration space is generally the set of points for which every tensor access in the expression is valid.

For matrix multiplication, $C_{m,n} = A_{m,k} \times B_{k,n}$, the iteration space is $[0, M) \times [0, K) \times [0, N)$, where A has shape $M \times K$ and B has shape $K \times N$. Each iteration-space point (m, k, n) is *projected* to a tensor point for each operand: (m, k) for A, (k, n) for B, and (m, n) for C. Indexing A and B at their tensor points retrieves their values; the pointwise expression is evaluated and accumulated into C at its tensor point. The reduction operator is applied whenever multiple iteration-space points project to the same output point—precisely the points that differ only in rank variables absent from the output. Crucially, an Einsum specifies only *what* computation to perform, not *how* to perform it: the order in which the iteration space is traversed, the data layout of each tensor, and the binding to a particular architecture are left to separate concerns [12,26]. This separation of concerns is what makes the Einsum a powerful declarative abstraction: prior sparse compilers [18,21] exploit it by decoupling the computation specification from the storage format [19,25], allowing users to select an appropriate (e.g., compressed) representation for each tensor independently of the expressed computation.

Extensions. Modern systems adopt Einsums in many different forms, and each extension of the syntax—in particular, of the rank-variable expressions permitted inside a tensor subscript—expands the class of applications the notation can express. In its basic form, where each rank of a tensor access is subscripted by a single rank variable, the notation captures tensor algebra proper: contractions, elementwise operations, and their compositions, as in matrix multiplication; Numpy [11] and TACO [18] support Einsums of this restricted form. Allowing simple affine functions of one variable, such as the strided access $C_i = A_{2\times i}$, admits strided and dilated computations. Permitting *full* affine rank-variable expressions, in which multiple rank variables combine within a single access, further unlocks sliding-window applications such as convolution, stencil computations, and scan. For instance, a 1D convolution with filter size S is written as

$$C_q = A_{q+s} \times B_s,$$

where the iteration space is $[0, Q) \times [0, S)$ and the reduction over s realizes the sliding-window summation. Similarly, a prefix sum (scan) can be written as $C_i = A_{i-j}$ with j reduced over, exploiting the convention that a projection to a nonexistent tensor coordinate (e.g., A_{i-j} for $j > i$) returns an empty value (e.g., 0) [30]. Going beyond affine expressions, *indirect* indexing allows a tensor access to appear within the rank-variable expression of another tensor (e.g., $C_i = A_{B_i}$), enabling gather/scatter-style applications such as embedding lookup, sparse data manipulation, and graph computations [12]. Taken together, these extensions broaden Einsums; throughout the remainder of this thesis, we use the term *Einsums* to refer to these extended Einsums as a whole.

Halide's algorithm language [12] and TVM's algorithm language [13] support many of the same computations as Einsums, including affine rank-variable expressions in tensor accesses, indirect accesses, and recursive updates such as scans. Their syntax can also look similar. Nevertheless, they differ in how reductions and iteration spaces are specified. First, reduction is implicit in an Einsum. In $C_{i,j} = A_{i,k} \times B_{k,j}$, Einsums automatically treat k as a reduction rank variable because it appears in the right hand side but not in the left hand side; initialization and accumulation are also implicit. Halide instead represents the same reduction explicitly as an initialization followed by an update:

```
C(i,j) = 0;     C(i,j) += A(i,k) * B(k,j).
```

Second, the iteration space is derived differently. Einsums derive the iteration space from the shapes of the input operands, whereas Halide derives it from the requested output size. For example, evaluating `C.realize({128, 64})` sets the (i, j) to $[0, 128) \times [0, 64)$. Halide then propagates this information backward, inferring that A must provide 128 rows and B must provide 64 columns. Because the requested output size does not determine the reduction iteration space for `k`, the user must specify it explicitly using an `RDom`. Thus, the primary distinction is not expressiveness but how the computation is specified: Einsums implicitly derive the iteration space and reduction structure from operand shapes and rank-variable occurrences, whereas Halide derives the iteration space from the output and represents reduction through explicit updates.

Limitation Throughout all of these extensions, one assumption remains fixed: coordinates are integers. Every rank variable ranges over an integer interval, the iteration space is a finite set of integer tuples that is traversed point by point, and every rank-variable expression, however complex, evaluates to an integer coordinate. Consequently, applications whose natural domains are continuous—for example, data defined over real-valued coordinates such as point clouds—cannot be expressed directly and must first be discretized onto an integer grid.

2.3 Compiling Einsums across Tensor Formats

Efficiently implementing an Einsum requires optimizing both computation and data movement through techniques such as loop reordering, tiling, vectorization, and parallelization. Because an Einsum specifies only *what* to compute, an implementation must decide *how*: the loop order in which to traverse the iteration space, how to tile and parallelize that traversal, and how to access each tensor's underlying representation. Viewed through the fibertree abstraction, the space of implementations is organized by the level formats of the tensors involved. Dense tensor frameworks and compilers handle arrays, while sparse tensor compilers additionally consume a format specification for each operand and generate code that operates directly on the specified representations. Restricting tensors to arrays makes dense implementations simple and fast, while supporting general level formats forces sparse compilers to confront data-dependent iteration, as we describe below.

2.3.1 Dense Tensor (Array) Frameworks and Compilers

Deep learning and array frameworks such as NumPy [11], PyTorch [1], and JAX [14] provide Einsum primitives natively (e.g. basic Einsums without extensions: `numpy.einsum,` `torch.einsum`), and they implement basic Einsums by reducing it to calls into pre-optimized kernels.

A multi-operand expression is first decomposed into a sequence of pairwise contractions—the order of which can change the asymptotic cost, motivating dedicated contraction-path optimizers [33]—and each pairwise contraction is then lowered via the Transpose-Transpose-GEMM-Transpose (TTGT) technique [34], which permutes and reshapes the operands so that the core computation becomes a (batched) matrix multiplication. Since GEMM dominates the runtime and maps directly onto matmul accelerators such as Nvidia's Tensor Cores [35], this strategy achieves high performance for contractions, at the cost of materializing transposed intermediates and of covering only computations expressible as GEMM.

Dense tensor *compilers*, by contrast, generate loop nests for Einsum-like expressions directly. Systems such as Halide [12] and TVM [13] introduce *scheduling languages* that decouple the algorithm (the Einsum-like expression) from its execution strategy (the *schedule*): the user composes loop-level transformations such as `split`, `fuse`, and `reorder` to express a wide variety of implementations of the same computation, and autoschedulers [36,37] can search this space automatically, albeit with considerable search time. Polyhedral compilers such as Pluto [38] and Tiramisu [39] take a related approach, modeling loop nests whose bounds and accesses are affine as integer polyhedra and deriving schedules through integer linear programming; notably, this machinery accommodates full affine rank-variable expressions by construction. Finally, the PyTorch compiler stack [27] compiles Einsum-like expressions by lowering them to Triton [40], a tile-based GPU programming model, for GPU execution. Unlike Halide or TVM, the PyTorch compiler does not expose a full scheduling language; instead, it applies a fixed set of heuristics together with autotuning over a small search space (e.g., tile sizes). This design enables fast compilation while still achieving performance competitive with, or better than, other existing tensor compilers [27]. While these systems differ in how they compile Einsum-like language, they all assume that tensors are stored as arrays. Sparse tensor compilers relax this assumption by supporting additional formats.

2.3.2 Sparse Tensor Compilers

Sparse tensor compilers such as TACO [18], Finch [21], and `mlir-sparse` [41] take two inputs: an Einsum and a format specification for each operand. To skip computations involving empty values, the generated code must co-iterate over the compressed representations of multiple operands. For multiplication, the effectual iteration space becomes the intersection of the operand; for addition, it becomes their union. Co-iteration discovers those coordinates without traversing the full tensor shape. To compute intersections or unions, sparse tensor compilers use algorithms such as two-finger merge and leader-follower methods [18,21,24,42,43]. This requires complex, data-dependent control flow that dense tensor compilers cannot express. As a result, these systems typically build their own code generation frameworks from scratch. TACO enumerates the co-iteration cases each loop must handle using *merge lattices* [18], and was later extended with a sparse scheduling language [44]. Finch instead generates code through *Looplets* [20], which hierarchically decompose each rank of a tensor into structured regions (e.g., runs of zeros or repeated values) and thereby generalize beyond sparsity to other regular structures. Because of the inherent control-flow complexity, most sparse tensor compilers target CPU code generation and tend to under-optimize the dense portions of the computation; on GPUs, hand-optimized kernel libraries [45,46] remain the dominant way to run sparse workloads.

Some recent efforts instead build sparse compilers on top of existing dense compiler infrastructures. COMET [47], built on MLIR [28], primarily targets CPU code generation. SparseTIR [48], built on TVM [13], composes per-rank format annotations with TVM's scheduling and code generation backend to produce high-performance sparse GPU kernels; however, it still requires manual scheduling, which demands significant user effort and expertise.

Limitation Finally, both families share the discreteness assumption of the abstractions they implement. The code they generate enumerates integer points: dense compilers emit counted loops over integer coordinate ranges with affine address arithmetic, and sparse compilers merge sorted sequences of integer coordinates during co-iteration. In both cases, the iteration space is a finite set of integer tuples that can be visited one point at a time—an assumption that breaks down when the iteration space is continuous and thus uncountable.

Chapter 3

The Continuous Tensor Abstraction

While traditional tensor systems back a wide range of applications in deep learning and scientific computing, they all share one fundamental assumption: coordinates are integers. Many domains, however, naturally produce data at real-numbered coordinates, such as point clouds with spatial coordinates. Rather than quantize continuous coordinates onto a discrete integer grid, this chapter extends the tensor model directly into continuous space.

The most natural starting point is the sparse tensor of Chapter 2. In a sparse tensor, a few nonzeros sit on an integer grid. What if these nonzeros were located at real-numbered coordinates, rather than integer coordinates? Figure 3.1 answers with a *sparse continuous tensor*: a function mapping real-valued coordinates to values, where most of the coordinates map to zero.

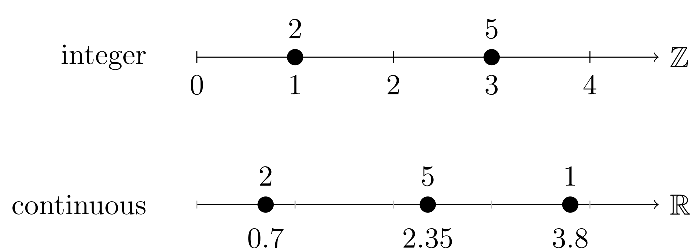


Figure 3.1: A sparse vector on the integer grid (top) and its continuous counterpart (bottom), where nonzeros may sit at arbitrary real coordinates.

We can also extend this concept to a dense counterpart by defining values over continuous

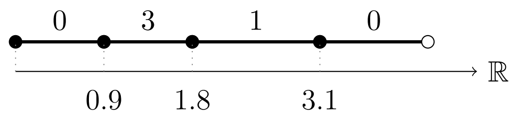


Figure 3.2: A dense continuous vector: values are held over intervals that partition the domain (closed/open circles mark half-open intervals $[a, b)$).

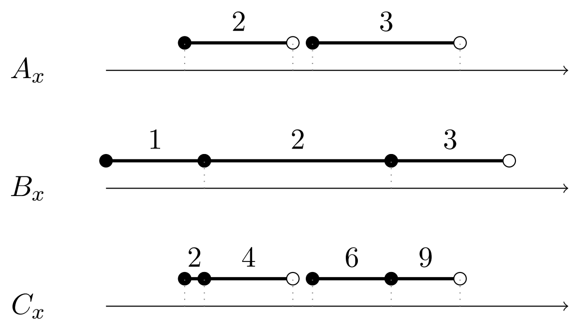


Figure 3.3: Elementwise multiplication $C_x = A_x \times B_x$ on continuous vectors. The Einsum is unchanged from the discrete case; the output is nonzero wherever the nonzero regions of A and B overlap (e.g., $C_x = 2 \cdot 2 = 4$ over $[1.0, 1.9)$).

intervals rather than at isolated points (Figure 3.2). For example, a continuous vector may hold the value 3 uniformly across the interval $[0.9, 1.8)$. We refer to isolated nonzeros as *pinpoints* and region-valued nonzeros as *intervals*; a continuous tensor whose intervals partition and cover the entire domain forms a *dense* continuous tensor. Unlike on an integer grid where finitely many points can cover a grid, a continuous domain contains infinitely many points, so dense coverage requires intervals rather than pinpoints.

Computing with continuous tensors. Many languages could describe computation over such tensors; we found that the Einsum notation of Chapter 2 extends to continuous space with remarkably little change. Consider the elementwise product $C_x = A_x \times B_x$ (Figure 3.3). The expression is *identical* to its discrete counterpart—the only difference is that the rank variable x now ranges over real numbers. The output holds a nonzero value wherever the nonzero regions of A and B overlap, with the value being the product of the two overlapping values.

What does it mean to reduce a tensor in a continuous domain? When reducing over

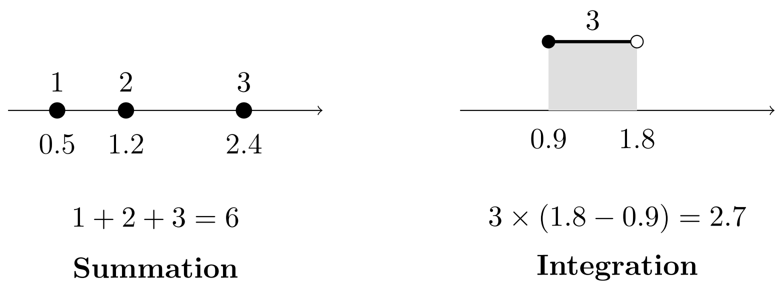


Figure 3.4: Reduction over pinpoints (left) sums the values; reduction over an interval (right) integrates—the value times the extent over which it holds (shaded area).

pinpoints, reduction behaves like standard summation across discrete coordinates. But "summing" over an interval is meaningless because uncountably many coordinates hold that value. Instead, the natural interpretation is to weight the value by the extent over which it holds, turning reduction into integration (Figure 3.4). Because real-world applications often require both summation and integration (sometimes even within the same program), the language must support both semantics.

Chapter roadmap. The rest of this chapter makes these intuitions precise. We first motivate why traditional integer-grid tensor models make continuous applications difficult to express. We then define continuous tensors and continuous Einsums through a denotational semantics (Section 3.2), in which expressions specify output tensors as functions over real coordinates under arbitrary semirings. Because a continuous iteration space cannot be enumerated point by point, we next introduce a finite *piecewise* representation: each tensor consists of finitely many pieces, and each piece associates an axis-aligned hyperrectangle (AAHR) with a function defined over that region. Although this abstraction permits arbitrary within-piece functions, supporting them would require symbolic algebra and integration. We therefore specialize the implementation to *piecewise-constant* tensors (Section 3.3), where every piece carries a single scalar value; for this specialization, we develop an evaluation semantics and establish validity conditions, checked by an SMT-based procedure, that guarantee program results remain representable in the same form.

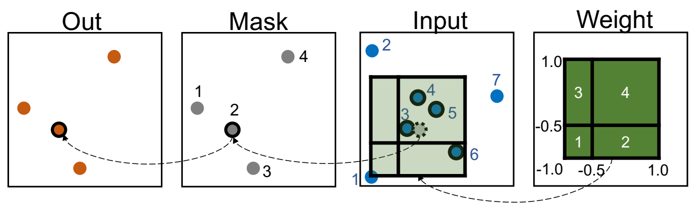
3.1 Motivation

Traditional tensor abstractions rely on integer coordinates, which map naturally to memory addresses in standard arrays. Data with real-numbered coordinates, however, introduces a fundamental challenge: real-numbered coordinates cannot serve directly as array offsets, so their values cannot be stored in arrays in the same manner as tensors on an integer grid are represented as arrays. Therefore, storing such data requires specialized data structures, such as COO formats [18,21] or interval trees [49,50]. Consequently, users must manually write low-level code to traverse these data structures and compute their intersections efficiently. While domain-specific tools like scientific visualization languages and spatial SQL attempt to simplify continuous data management, they lack the expressiveness required for a broad range of spatial computations. To illustrate these difficulties, we first demonstrate the complexity of implementing a masked point-cloud convolution using traditional array abstractions, and then contrast existing programming models with our continuous tensor abstraction.

3.1.1 Challenge with Real-valued Coordinates: Point Cloud Convolution

Point-cloud convolution provides a simple example of a tensor computation whose coordinates are not restricted to an integer grid. Unlike an image, a point cloud stores values only at a discrete set of pinpoints with real-valued coordinates. We consider the masked point-cloud convolution illustrated in Figure 3.5, based on LargeKernel [51]. At a high level, the operation is simple: *for each point in the mask, collect nearby input points and compute a weighted sum of their values.*

The computation has three operands. `Mask` specifies the points (x, y) at which outputs should be computed. `Input` stores points with real-valued coordinates and associated feature values. Finally, `Weight` defines the convolution kernel as a function of the relative displacement (r, s) between a mask point and an input point.



Mx	0.1	1.2	1.8	2.3			
My	1.5	1.3	0.2	2.8			
Mval	1	2	3	4			
Inx	0.1	0.1	1.2	1.4	1.5	1.9	2.6
Iny	0.1	2.8	1.4	1.8	1.5	0.5	1.9
Inval	1	2	3	4	5	6	7
Wstx	-1.0	-0.5	-1.0	-0.5			
Wenx	-0.5	1.0	-0.5	1.0			
Wsty	-1.0	-1.0	-0.5	-0.5			
Weny	-0.5	-0.5	1.0	1.0			
Wval	1	2	3	4			

(a) A masked point-cloud convolution and the storage of its coordinates and values.

```
# Tensor Rank Format Specification
Out.dim    = [Pinpoint, Pinpoint]
Mask.dim   = [Pinpoint, Pinpoint]
Input.dim  = [Pinpoint, Pinpoint]
Weight.dim = [Interval, Interval]

# Continuous Einsum
Out[x,y] = Mask[x,y] * Input[x+r,y+s] * Weight[r,s]
```

(b) The computation expressed as a continuous Einsum.

```
std::vector<float> Outx;
std::vector<float> Outy;
std::vector<float> Outval;

for (int mp = 0; mp < 4; ++mp){
  float acc = 0.0f;
  for (int ip = 0; ip < 7; ++ip){
    for (int wp = 0; wp < 4; ++wp){
      float Maskx = Mx[mp];
      float Masky = My[mp];
      float Inx = Ix[ip];
      float Iny = Iy[ip];
      float WStx = Wstx[wp];
      float WEnx = Wenx[wp];
      float WSty = Wsty[wp];
      float WEny = Weny[wp];

      bool x1 = Maskx+WStx <= Inx;
      bool x2 = Inx < Maskx+WEnx;
      bool y1 = Masky+WSty <= Iny;
      bool y2 = Iny < Masky+WEny;

      if (x1 && x2 && y1 && y2) {
        acc += Mval[mp] *
          Inval[ip] * Wval[wp];
      }
    }
  }
  if (acc != 0) {
    Outx.push_back(Mx[mp]);
    Outy.push_back(My[mp]);
    Outval.push_back(acc);
  }
}
```

(c) An equivalent implementation in C++ using coordinate arrays and explicit range checks.

Figure 3.5: For each point in the mask, the operator collects nearby input points and computes a weighted sum based on their relative coordinates. A continuous Einsum expresses the operation directly over real-valued coordinates.

Consider a mask point at (x, y). An input point at $(x+r, y+s)$ contributes to this output when its relative displacement (r, s) lies within the support of the kernel. Its input value is multiplied by the kernel value at (r, s), and the contributions from all such input points are summed. Thus, each mask point defines the center of a local neighborhood over the input point cloud.

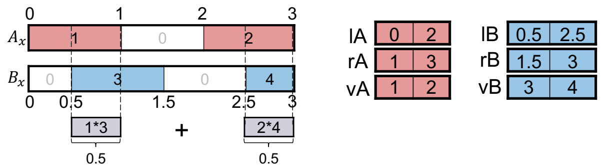
A continuous Einsum expresses this computation as

$$\text{Out}_{x,y} = \text{Mask}_{x,y} \times \text{Input}_{x+r,y+s} \times \text{Weight}_{r,s},$$

where r and s are reduction rank variables. As in a conventional convolution, reduction rank variables describe the weighted accumulation. The key difference is that x, y, r, and s range over real-valued coordinate domains rather than integer array positions.

This distinction also appears in the storage formats. The *pinpoint* format represents values that exist only at exact coordinates, making it natural for the `Mask` and `Input` point sets. The *interval* format represents a value over a continuous coordinate range, allowing the piecewise-constant regions of `Weight` to be represented directly.

Implementing the same operation with conventional arrays requires explicitly iterating over mask points, input points, and kernel regions, computing relative coordinates, performing range checks, and accumulating the matching products. Continuous Einsums instead separate storage from computation: the formats describe how values occupy continuous coordinates, while a single Einsum expression describes the convolution itself. This yields a concise representation for various point-cloud operators that closely mirrors their mathematical formulations in prior point-cloud deep learning literature [23,51–54].



(a) Visualization of interval dot product along with the memory representation of the coordinates (`lA,rA,lB,rB`) and values (`vA,vB`).

```
# Format Spec of Continuous Tensors
A = Interval(Element(vA),lA,rA)
B = Interval(Element(vB),rA,rB)
# Continuous Einsum
val = A[x] * B[x] * d(x,μλ)
```

(b) Interval dot product in continuous tensor abstraction, where $d(x, \mu_\lambda)$ denotes integration over x with Lebesgue measure μ_λ.

```
field#1(1)[] A, B;
strand dotprod(){
real t = 0;
real tmax = 3;
real step = 0.01;
real val = 0;
update {
  val += A(t)*B(t)*step;
  t += step;
  if (t > tmax){stabilize;}
}}
```

(c) Interval dot product *approximately* written in SciVis language Diderot [55]

```
SELECT
 SUM(
  ST_Length(
   ST_Intersection(A.geo,B.geo)
  )
  * A.value * B.value
 )
FROM A
JOIN B
  ON ST_Intersects(A.geo,B.geo)
GROUP BY A.id
```

(d) Interval dot product written in spatial SQL.

```
pA, pB = 0, 0
while (pA<2 and pB<2):
 lA,rA,vA = A[pA]
 lB,rB,vB = B[pB]
 lAB = max(lA,lB)
 rAB = min(rA,rB)
 if lAB <= rAB:
  sum += (rAB-lAB)*vA*vB
 pA += (rA==rAB)
 pB += (rB==rAB)
```

(e) Interval dot product written in array programming with integer coordinates.

Figure 3.6: Illustration of an interval dot product written in four different languages, highlighting how our continuous tensor abstraction provides a concise and straightforward implementation compared to others.

3.1.2 State of Existing Programming Tools: Interval Dot Product

Programming over continuous domains is still challenging in current tools. In contrast, our abstraction is simple, intuitive, and concise, while still producing code that matches or outperforms existing approaches. This advantage is especially clear in applications, such as scientific visualization [55,56] and spatial deep learning [53,57,58], which require both (1) geometric operations and (2) numerical computation on geometry-tied values, a combination rarely supported by current systems.

Figure 3.6 shows the interval dot product, which sums the product of values over the lengths of overlapping intervals, requiring both intersection computation and numerical integration. This operation is useful in many areas, such as bioinformatics. For example, when comparing two genomic intervals A and B, it calculates how much of gene A's position overlaps with gene B's position on the chromosome. More operations on genomic intervals are discussed in Section 4.3. Existing models—scientific visualization languages [55,56], spatial databases [59,60], and tensor frameworks [8,11]—typically focus only one of two aspects, making such tasks difficult to express.

SciVis. Languages like Diderot [55] support pointwise evaluations on continuous fields, but lack mechanisms for expressing geometric operations like intersections. As shown in Figure 3.6(c), Diderot allows querying field values at specific points on real domain, but cannot compute field intersections directly, requiring users to approximate the interval dot product via Riemann integration.

Spatial Database. Spatial databases [59,61] support geometric queries and efficient spatial structures like R-trees [62], but are limited in expressing numerical computations. As shown in Figure 3.6(d), spatial SQL easily handles intersections and lengths, but becomes cumbersome for complex computations on multidimensional tensors.

Tensor Programming. Tensor frameworks excel at high-performance numerical computation but assume arrays and integer coordinate domains, making them poorly suited for continuous domains. Supporting real-valued coordinates requires custom data structures (e.g., COO formats, interval trees [19,50,63]) and significant engineering to implement geometric operations like intersection. As shown in Figure 3.6(e), low-level implementations can perform efficient intersection and weighted sums but must be re-written for each new computation.

To bridge this gap, we propose a continuous tensor abstraction that unifies the strengths of existing models. Tensors are defined over real coordinates and have nonzero values on geometrically meaningful regions (e.g., boxes, lines). We generalize tensor indexing and extend Einsums [11,26,29] to continuous domains, enabling concise, expressive geometric and numerical computation.

Continuous Tensor Abstraction. As shown in Figure 3.6(b), our abstraction allows users to define data structures using a *level format abstraction* [19,25] and write computations using an *extended Einsum notation* with real-valued rank variables. The resulting code is compiled into efficient low-level implementations, equivalent to those shown in Figure 3.6(e). Users simply write a familiar Einsum expression and format specification, yielding a unified programming model for continuous domain that combines both geometric and numerical computation.

3.2 Syntax and Denotational Semantics

Figures 3.7 and 3.8 present the syntax and denotational semantics of our language. The language closely resembles conventional Einsum notation, but tensors are defined over continuous domains and rank variables range over those domains. A continuous tensor can therefore be viewed as a function that maps points in a real-valued domain to values.

One notable difference from conventional Einsums is that a reduction over a continuous domain may correspond either to a discrete operation, such as summation, or to a continuous operation, such as integration. To distinguish between these cases, the user specifies a reduction *measure* for each reduction rank variable using `d(<redRankVar>...,` `<measure>...)`.

Specifically, Einsum statements in our language can take one of the following forms:

$$C_{x_1,\dots,x_n} = P_{x_1,\dots,x_n} \quad \text{if there is no reduction}$$

$$C_{x_1,\dots,x_n} = P_{x_1,\dots,x_n,rx_1,\dots,rx_k} * d(rx_1,\dots,rx_k;\mu_1,\dots,\mu_k) \quad \text{otherwise}$$

$$
\begin{aligned}
\langle einsum \rangle &::= \langle stmt \rangle \\
&\quad \mid \langle stmt \rangle * d(\langle reductionRankVar \rangle \ldots ; \langle measure \rangle \ldots) \\
\langle stmt \rangle &::= \langle tensorName \rangle [\langle rankVar \rangle \ldots] = \langle expr \rangle \\
\langle expr \rangle &::= \langle val \rangle \\
&\quad \mid \langle affineRankExpr \rangle \\
&\quad \mid \langle tensorName \rangle [\langle affineRankExpr \rangle \ldots] \\
&\quad \mid (\langle expr \rangle) \\
&\quad \mid \langle expr \rangle \ \langle op \rangle \ \langle expr \rangle
\end{aligned}
$$

$$
\begin{aligned}
\langle affineRankExpr \rangle &::= \langle affineTerm \rangle \\
&\quad \mid \langle affineRankExpr \rangle + \langle affineTerm \rangle \\
\langle affineTerm \rangle &::= \langle val \rangle \mid \langle rankVar \rangle \mid \langle val \rangle * \langle rankVar \rangle \\
\langle tensorName \rangle &::= \langle identifier \rangle \\
\langle rankVar \rangle &::= \langle identifier \rangle \\
\langle reductionRankVar \rangle &::= \langle identifier \rangle \\
\langle op \rangle &::= + \mid - \mid * \mid \text{AND} \mid \text{OR} \ldots \\
\langle measure \rangle &::= \mu_{\wedge\vee} \mid \mu_{\#} \mid \mu_{\lambda} \mid \ldots \\
\langle val \rangle &::= \langle \text{Real Number} \rangle \\
\langle identifier \rangle &::= [a{-}zA{-}Z]
\end{aligned}
$$

Figure 3.7: Grammar of continuous Einsums.

Here, C is the output tensor, whose values are computed over the rank variables $x_1, \ldots, x_n, rx_1, \ldots, rx_k$ defined on a real domain. The tuple $(rx_1, \ldots, rx_k) \in D$ assigns coordinates to the reduction rank variables (where D is a reduction domain). $P_{x_1,\ldots,x_n}$ denotes a general expression that may involve one or more tensors and depends on the rank variables $x_1, \ldots, x_n$. We adopt subscript notation for consistency with standard tensor accesses; however, this notation does not imply that P is necessarily a single tensor. In fact, P may incorporate multiple tensor accesses and semiring operations [64–66]. In the denotational semantics presented in Figure 3.8, a statement involving reductions is interpreted as follows:

$$
C_{x_1,\ldots,x_n} = \int_{rx_1,\ldots,rx_k}^{\oplus} P_{x_1,\ldots,x_n,rx_1,\ldots,rx_k} d\mu_1(rx_1) \ldots d\mu_k(rx_k)
$$

Einsum

$$[\![\texttt{tensorName}[\texttt{rankVar}_1, ..., \texttt{rankVar}_n]]\!]^{\mathrm{Env}} = (x \rightarrow [\![\texttt{expr}]\!]^{\mathrm{Env} \cup \{\texttt{rankVar}_1, ..., \texttt{rankVar}_n \mapsto x\}} : x \in \mathbb{R}^n)$$

$$[\![\texttt{tensorName}[\texttt{rankVar}_1, ..., \texttt{rankVar}_n] = \texttt{expr} * d(\texttt{redRankVar}_1, ..., \texttt{redRankVar}_k;\ \texttt{measure}_1, ..., \texttt{measure}_k)]\!]^{\mathrm{Env}} =$$
$$\text{Let } f(x) : \mathbb{R}^{n+k} \rightarrow [\![\texttt{expr}]\!]^{\mathrm{Env} \cup \{\texttt{rankVar}_1, ..., \texttt{rankVar}_n, \texttt{redRankVar}_1, ..., \texttt{redRankVar}_k \mapsto x\}}$$
$$\text{in Let } g(x) : \mathbb{R}^n \rightarrow \int^{\oplus}_{\texttt{redRankVar}_{1...k}} f(\texttt{rankVar}_{1,...,n}, \texttt{redRankVar}_{1,...,k})\, d\mu_1(\texttt{redRankVar}_1)...d\mu_k(\texttt{redRankVar}_k) \text{ in } g$$

Expression

$$[\![\texttt{val}]\!]^{\mathrm{Env}} = \texttt{val}$$

$$[\![\texttt{tensorName}[\texttt{affineRankExpr} \dots]]\!]^{\mathrm{Env}} = \mathrm{Env}[\texttt{tensorName}]([\![\texttt{affineRankExpr}]\!]^{\mathrm{Env}} \dots)$$

$$[\![(\texttt{expr})]\!]^{\mathrm{Env}} = [\![\texttt{expr}]\!]^{\mathrm{Env}}$$

$$[\![\texttt{expr}_1 \texttt{ op } \texttt{expr}_2]\!]^{\mathrm{Env}} = [\![\texttt{op}]\!] \left([\![\texttt{expr}_1]\!]^{\mathrm{Env}}, [\![\texttt{expr}_2]\!]^{\mathrm{Env}}\right)$$

Affine Rank-Variable Expression

$$[\![\texttt{rankVar}]\!]^{\mathrm{Env}} = \mathrm{Env}[\texttt{rankVar}]$$

$$[\![\texttt{val} * \texttt{rankVar}]\!]^{\mathrm{Env}} = [\![\texttt{val}]\!]^{\mathrm{Env}} * [\![\texttt{rankVar}]\!]^{\mathrm{Env}}$$

$$[\![\texttt{affineRankExpr}_1 + \texttt{affineRankExpr}_2]\!]^{\mathrm{Env}} = [\![\texttt{affineRankExpr}_1]\!]^{\mathrm{Env}} + [\![\texttt{affineRankExpr}_2]\!]^{\mathrm{Env}}$$

Figure 3.8: Denotational semantics of our language. We assume an environment Env providing: **(1)** a mapping from each tensor name to a function $\mathbb{R}^n \rightarrow \mathbb{S}$ (where $\mathbb{S}$ is a semiring domain), **(2)** a mapping from each rank-variable name to a real number, and **(3)** a mapping from each measure name to a corresponding semiring-valued measure μ_i. Each μ_i is referenced in the syntax as $\texttt{measure}_i$ and, when combined with $\texttt{redRankVar}_i$, enables integrating over the specified reduction rank variables.

The operator $\int^{\oplus}$ represents an integral using semiring addition $\oplus$, combining the values of P across the reduction rank variables. This reduction is weighted by the *semiring-valued measures* $\mu_1, \dots, \mu_k$ for each reduction rank variable $rx_1, \dots, rx_k$, respectively.

Calculations are performed within a semiring $\mathbb{S} = (R, \oplus, \otimes, 0, 1)$, which is defined by an addition operation $\oplus$ (associative, commutative, with identity 0) and a multiplication operation $\otimes$ (associative, with identity 1, distributing over $\oplus$). Common examples include the real number semiring $(\mathbb{R}, +, \times, 0, 1)$ and the boolean semiring $(\{T, F\}, \vee, \wedge, F, T)$. To formally define the reduction process, a measure-theoretic approach [67–69] is used. Given a sigma-algebra over the reduction rank variables $V \subset \mathbb{P}(D)$ (a power set of D), a *semiring-valued measure* $\mu : V \rightarrow \mathbb{S}$ is defined on subsets of D, satisfying $\mu(\emptyset) = 0$ and disjoint additivity: $\mu(A \cup B) = \mu(A) \oplus \mu(B)$ for disjoint subsets A and B.

Using measures $\mu_1, \ldots, \mu_k$ for each reduction rank variable $rx_1, \ldots, rx_k$, the reduction is rewritten as:

$$C_{x_1,\ldots,x_n} \;=\; \bigoplus_{m \in \mathbb{S}} \Big[(\mu_1 \otimes \cdots \otimes \mu_k)(\{(rx_1, \ldots, rx_k) \in D : P_{x_1,\ldots,x_n,rx_1,\ldots,rx_k} = m\})\Big] \;\otimes\; m.$$

This expression defines each output $C_{x_1,\ldots,x_n}$ by summing over all possible values $m \in \mathbb{S}$ that P can take. Each value m is weighted by the product of semiring-valued measures associated with the reduction rank variables. Specifically, since $m \in \mathbb{S}$, it may represent any element in the semiring: for example, any real number in the real semiring, or true and false in the boolean semiring. The measure μ provides these weights, determining how much each m contributes to $C_{x_1,\ldots,x_n}$. Finally, the semiring addition $\oplus$ is used to combine these weighted contributions into a single value.

$\oplus$ and $\otimes$ could represent real addition and multiplication or the logical OR and AND. While this subsection focuses on the real number semiring and the Boolean semiring, the framework can accommodate additional semirings, allowing the language to support various reduction semantics.

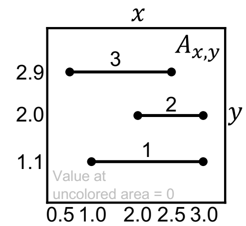
Real Number Semiring $(\boldsymbol{R}, +, *, 0, 1)$

For Real number addition ($\oplus = +$), users specify a measure μ over D, choosing either the **Lebesgue measure** μ_λ or the **counting measure** $\mu_\#$ [70]. The Lebesgue measure, suitable for continuous intervals, is defined as $\mu_\lambda([a, b]) = b - a$. The counting measure, suitable for discrete points, is defined as follows: for isolated pinpoints $[a, a]$, $\mu_\#([a, a]) = 1$; for intervals like $[a, b]$ with $a < b$, $\mu_\#([a, b]) = \infty$. Users can flexibly model both continuous and discrete accumulations by selecting the proper measure.

Given a function f_P representing P evaluated at $(x_1, \ldots, x_n, rx_1, \ldots, rx_k)$, we define:

$$C_{x_1,\ldots,x_n} = \int_{rx_1 \ldots rx_k} f_P(x_1, \ldots, x_n, rx_1, \ldots, rx_k)\, d\mu_1(rx_1) \ldots d\mu_k(rx_k)$$

In the right example, the Einsum $s = A_{x,y} * d(x, y; \mu_\lambda, \mu_\#)$ evaluates $s = \int_y \int_x f_A(x, y)\, d\mu_\lambda(x)\, d\mu_\#(y) = 1 * (3.0 - 1.0) + 2 * (3.0 - 2.0) + 3 * (2.5 - 0.5)$, where $f_A(x, y)$ is the function evaluating the tensor $A_{x,y}$. Another Einsum, $s = A_{x,y} * d(x, y; \mu_\lambda, \mu_\lambda)$, yields zero because the 2D xy area over a 1D x interval is zero. An easy analogy is that the measure μ_λ acts as an integral over intervals, while the measure $\mu_\#$ resembles a discrete sum over points.



Boolean Semiring $(\{T, F\}, \vee, \wedge, F, T)$

For logical disjunction, we use a function $f_P : (x_1, \ldots, x_n, rx_1, \ldots, rx_k) \to \{\mathrm{F}, \mathrm{T}\}$. The reduction is defined by the logical measure $\mu_{\wedge\vee}$:

$$C_{x_1,\ldots,x_n} = \bigvee_{rx_1 \ldots rx_k} f_P(x_1, \ldots, x_n, rx_1, \ldots, rx_k) \wedge \mu_{\wedge\vee}(rx_1) \wedge \ldots \mu_{\wedge\vee}(rx_k)$$

where $C_{x_1,\ldots,x_n} = T$ if $f_P(x_1, \ldots, x_n, rx_1, \ldots, rx_k) = T$ for any $rx_1 \ldots rx_k$; otherwise, $C_{x_1,\ldots,x_n} = F$. In this case, the boolean measure $\mu_{\wedge\vee}$ assigns T to any non-empty set, and F to the empty set.

3.3 Piecewise-Constant Specialization

The operational definition of traditional Einsums [26] cannot be directly applied to continuous tensor programs because it is impossible to traverse infinitely many points in a continuous iteration space. We instead represent each tensor as a finite set of *pieces*. Each piece specifies both *where* it applies (a region of the domain) and *what values* the tensor takes there (a function over that region). Traversing a finite set of pieces is possible, but evaluating the function within each piece remains a separate challenge. In principle, each function could be arbitrary. Implementation supporting arbitrary functions would require symbolic manipulation for algebraic operations and integration, and the chosen function class would need to remain closed under Einsums supported operations.

The continuous tensor abstraction itself is defined for arbitrary functions within each piece. The implementation developed in this thesis, however, supports only the simplest function class: constant functions. Each piece therefore carries a single scalar value, allowing the evaluator to traverse finitely many pieces and perform simple scalar computations on their values. We call such tensors **piecewise-constant** and focus on them throughout the remainder of this thesis. This implementation restriction enables an efficient implementation while still supporting a wide range of useful applications. Later chapter 10 discusses the requirements for richer piecewise function classes and leaves their implementation to future work.

In this section, we first present evaluation semantics for continuous tensor programs under the piecewise-constant assumption. We then describe the validity conditions and explain how to verify whether a given program satisfies them.

3.3.1 Piecewise-Constant Tensor

We introduce the piecewise-constant restriction to create a practical and implementable solution. For the rest of the thesis, we assume: ***All continuous tensors must be piecewise-constant.***

We define a **tensor** `A` as a logical mapping whose value at a tensor point $(x_1, \ldots, x_n)$ can be obtained by evaluating $\texttt{A[}x_1, \ldots, x_n\texttt{]}$. We can express its values as a mathematical function $f(x_1, \ldots, x_n) = \texttt{A}[x_1, \ldots, x_n]$. A tensor is said to be **piecewise constant** when its values are constant over specific regions of its domain and can be expressed as $f(x_1, \ldots, x_n) = \sum_m V_m \times [\![(x_1, \ldots, x_n) \in I_m]\!]$, where: V_m is the constant value in the m-th piece; I_m is an N-dimensional interval represented as an axis-aligned hyperrectangle (AAHR) and denotes the domain of the m-th piece; $[\![]\!]$ denotes the Iverson bracket, defined as $[\![cond]\!] = 1$ if the $cond$ is true, and 0 otherwise.

In this definition, the N-dimensional interval I_m must satisfy the following rules:

1. **Disjoint Intervals**: The intervals are pairwise disjoint, $\forall\, m_1 \neq m_2,\ I_{m_1} \cap I_{m_2} = \emptyset$.

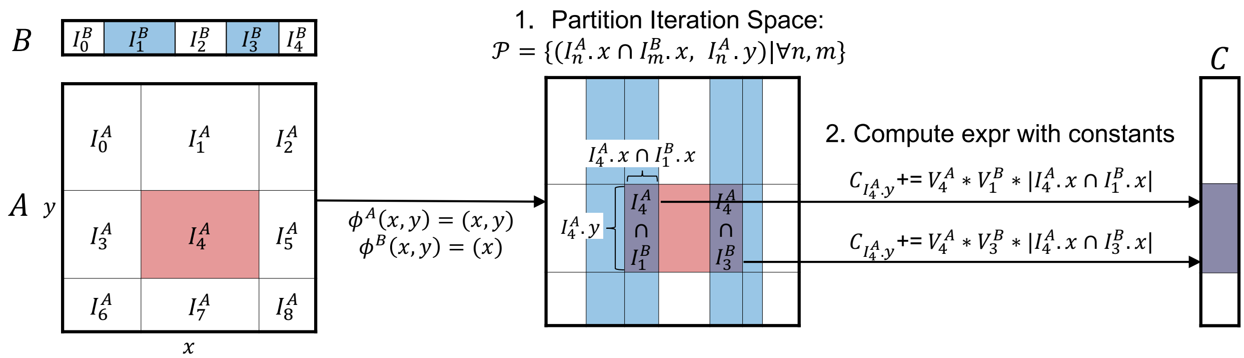


Figure 3.9: Evaluation of $\boldsymbol{C_y = A_{x,y} * B_x * d(x, \mu_\lambda)}$, where $d(x, \mu_\lambda)$ is a Lebesgue integral on x. The computation iterates over partitions formed by intersecting intervals of $A_{x,y}$ and B_x, with non-colored regions as zeros, and evaluates the expression within each. The two right arrows indicate computations on non-zero tensor elements, highlighting only interactions that contribute non-zero values.

2. **Complete Coverage**: The union of all intervals covers the entire domain, $\bigcup_m I_m = \mathbb{R}^N$.

3. **AAHRs**: Each interval I_m is an AAHR (i.e., it has no diagonal or curved boundary).

3.3.2 Piecewise-Constant Evaluation Semantics

In tensor computations involving piecewise constant tensors, we first partition the iteration space into regions where each tensor maintains a constant value. Rather than traversing all real-numbered coordinates in the entire space, we traverse each partition and evaluate the tensor expression using the constant values for that partition. We formalize an evaluation semantics as follows :

1. Partitioning the Iteration Space. The goal is to partition the iteration space into regions where all tensors used in the expression remain constant. To achieve this, we first map piecewise-constant intervals of each tensor into the iteration space using projection functions. A projection function maps an iteration-space point to a tensor point according to rank variable expressions. By applying the inverse of this projection, each tensor's constant interval is converted into a corresponding region in the iteration space (Figure 3.10(a)).

Next, the iteration space is partitioned by intersecting these regions from all tensors. Each resulting intersection indicates a unique region in which every tensor in the expression remains simultaneously constant (Figure 3.10(b)).

```
# Step 1a: Map tensor's constant intervals into
# iteration space with inverse of projection
inverse_pieces = {}
for access in einsum:
  tensor = access.tensor
  projection = access.projection
  preimage = inverse(projection)
  inverse_pieces[tensor] = [
      {'interval': preimage(piece.interval),
       'constant': piece.constant}
      for piece in tensor
  ]
```

(a) Step 1a.

```
# Step 1b: Partition iteration space into regions
#          where tensors remain constant
for piece1 in inverse_pieces[tensor1]:
  ...
  for pieceN in inverse_pieces[tensorN]:
    intervals = [piece1.interval, ...,
                 pieceN.interval]
    partition = intersect(intervals)
    if partition.is_empty(): continue
    # Step 2: Evaluate expression using constants
    weight = 1
    if reduction exists: # if d(x,μ) exists
      for (axis, measure) in einsum.d_exprs:
        weight ⊗= measure(partition.axis)
    out_constant = weight ⊗ compute_expr(
      piece1.constant, ..., pieceN.constant)
    out_interval = out.projection(partition)
    out[out_interval] ⊕= out_constant
```

(b) Steps 1b & 2.

Figure 3.10: Pseudocode of the piecewise-constant evaluation semantics of our language.

For instance, in Figure 3.9, the tensor expression $C_y = A_{x,y} * B_x * d(x, \mu_\lambda)$ is partitioned as follows. The projection functions are $\phi^A(x, y) = (x, y)$ and $\phi^B(x, y) = (x)$, with their respective pre-images:

$$\phi^{A^{-1}}(I^A) = \{(x, y) \in \mathcal{S} \mid (x, y) \in I^A\}, \quad \phi^{B^{-1}}(I^B) = \{(x, y) \in \mathcal{S} \mid x \in I^B\},$$

where I^A and I^B are intervals of tensor A and B, and $\mathcal{S}$ denotes the iteration space of rank variables (x, y). Note that the pre-image of an interval in tensor B spans the entire region along the y-axis. The partitioned iteration space $\mathcal{P}$ is then defined as an intersection of the pre-image of:

$$\mathcal{P} = \left\{ \phi^{A^{-1}}(I_n^A) \cap \phi^{B^{-1}}(I_m^B) \,\middle|\, \forall n, m \right\},$$

where n and m range over all intervals of A and B. In each partition, both A and B are constant.

2. Compute Expression with Constants. After partitioning, each disjoint partition contains constant values for all tensors involved in the expression. We traverse each partition and evaluate the expression using these constants. When the expression involves a reduction operation, we aggregate the computation results from each partition into the corresponding output region. If there is a reduction with Lebesgue measure over an axis, we multiply the result by the length of the partition along that axis to account for the integration over that interval. Step 2 in Figure 3.10(b) illustrates this process. Note that the partition is an n-D AAHR $\texttt{partition} = I_1 \times I_2 \times \cdots \times I_n$, where n is the number of rank variables in the expression. Let $R \subseteq \{1, \ldots, n\}$ be the set of reduced axes, each with a semiring-valued measure $\mu_i : \text{Intervals} \to \mathbb{S}$. In lines 25-26, the reduction contributes

$$\texttt{out\_constant} \, \otimes \, \mu\left(\prod_{i \in R} I_i\right) = \texttt{out\_constant} \, \otimes \, \prod_{i \in R} \mu_i(I_i).$$

For ordinary nonnegative real-valued measures, this is the standard product-measure identity for AAHRs and follows by applying Tonelli's theorem to the AAHR's indicator function [70].

To support an arbitrary semiring $(R, \oplus, \otimes, 0, 1)$ with arbitrary measures, users only need to implement lines 23-30 according to their chosen semiring and measure. To support a custom semiring, users initialize the output with the additive identity $0 \in R$, the measure weight with the multiplicative identity $1 \in R$, and define the implementations of $\oplus$ (addition) and $\otimes$ (multiplication). Users may also provide their own measure function $\mu : Interval \rightarrow R$ over intervals. For example, the *Lebesgue measure* in the real-number semiring is defined as $\mu_\lambda([a, b]) = b - a$. As a concrete example, consider the *min-product* semiring $(\mathbb{R}_{\geq 0} \cup \{\infty\}, \min, *, \infty, 1)$, where $\oplus = \min$ and $\otimes = *$. To use this semiring, one would initialize the output to ∞, and may implement an idempotent measure such as $\mu_{idem}([a, b]) = 1$, assigning uniform weight to all partitions regardless of size.

In Figure 3.9, we illustrate this process by traversing all partitions. For illustrative purposes, we only depict effectual computations involving non-zeros. For each partition, we perform the following operation with constants: $C_{I_y^{\text{Partition}}} \mathrel{+}= V_n^A \times V_m^B \times \text{length}(I_x^{\text{Partition}})$.

This semantics generalizes partitioning and computation for tensors of varying dimensions and access patterns. It ensures the partitioned iteration space is disjoint, covers the entire space, and maintains the piecewise-constant property. Each partition contributes accurately to the final result, accounting for reductions over partitioned axes.

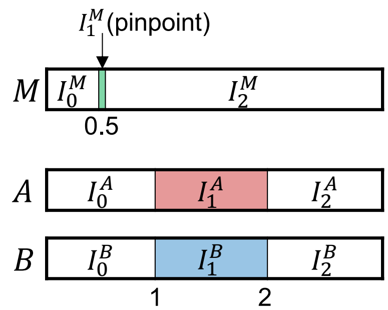
3.3.3 Program Validity

Not all programs are valid under the piecewise-constant assumption. To ensure correctness, the output must also be piecewise-constant—that is, the program must be *closed under piecewise-constant assumption*. We establish the following conditions to guarantee this closure after partitioning:

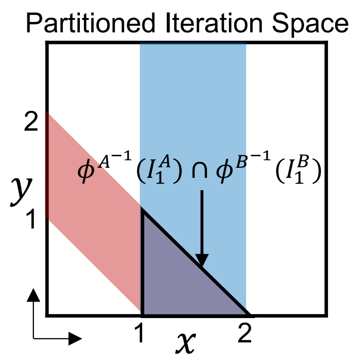
1. **Each *effectual partition* must be an AAHR.**

2. **Expressions within each effectual partition must remain constant, except when the partition reduces to a pinpoint.**

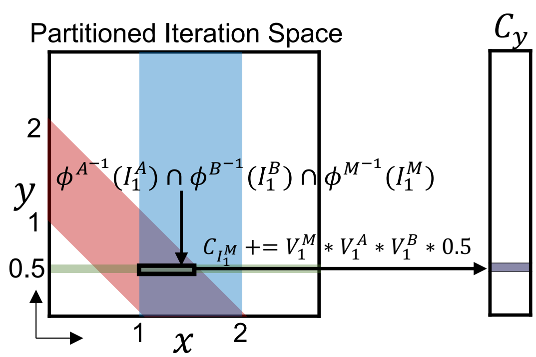
An *effectual partition* refers to a partition where effectual computations—those that contribute to the final output, such as multiplying non-zero constants—take place. A *pinpoint* refers to an N-D interval that collapses to a single point, meaning its end points are identical along all dimensions.



(a) 1D Continuous Tensors



(b) Invalid Program: $\boldsymbol{C_y = A_{y+x} * B_x * d(x, \mu_\lambda)}$



(c) Valid Program: $\boldsymbol{C_y = M_y * A_{y+x} * B_x * d(x, \mu_\lambda)}$

Figure 3.11: Illustrations of partitioned iteration spaces and program validity. (a) shows 1D continuous tensors where non-colored regions have values of zero. (b) demonstrates an invalid program because the intersected partition is a triangular region, leading to a non-piecewise-constant output. (c) shows a valid program where the pinpoint I_1^M in tensor M adjusts the partition to form AAHRs.

```
# Step1: Construct a Symbolic Partition
# For example, in the expression
# C[y] = M[y] * A[x + y] * B[x],
# where M has pinpoints /  A and B has intervals,
# the partition over (x, y) satisfies:
#   y == p1,          # pinpoint M[y]
#   p2 <= x+y <= p3, # interval A[x+y]
#   p4 <= x <= p5    # interval B[x]
partition = []
rank_vars = [x, y, z, ...] # Rank variables
for access in einsum:
  for tensor_rank in access:
    rank_expr = tensor_rank.projection(rank_vars)
    if tensor_rank.pinpoint:
      partition.add(
        rank_expr == FreshRealSymbol()
      )
    elif tensor_rank.interval:
      partition.add(
        FreshRealSymbol() <= rank_expr,
        rank_expr <= FreshRealSymbol()
      )

# Step 2: Construct a Symbolic AAHR
# aahr = [
#    p6 <= x <= p7,
#    p8 <= y <= p9,
# ]
aahr = []
for rank_var in rank_vars:
  aahr.add(
    FreshRealSymbol() <= rank_var,
    rank_var <= FreshRealSymbol()
  )

# Step 3: Check whether the partition
#         always forms an AAHR.
# Z3.check(ForAll([p1, p2, p3, p4, p5],
#          Exists([p6, p7, p8, p9],
#          ForAll([x, y], partition == aahr))))
Check1 = Exists(aahr.RealSymbols,
         ForAll(rank_vars, partition == aahr))
Check2 = ForAll(partition.RealSymbols, Check1)
Z3.check(Check2)
```

Figure 3.12: An algorithm for verifying AAHR partitions based on pinpoint/interval specifications. Step 1 defines the shape of the partition by formulating linear inequalities and equalities of rank variables based on tensor accesses. Step 2 defines the AAHR over the rank variables, and Step 3 checks whether the partition always forms an AAHR.

For the first criterion, the definition of a piecewise-constant tensor requires partitions to be AAHRs so that the output remains piecewise-constant. As illustrated in Figure 3.11(b), a standard convolution on intervals creates triangular partitions, resulting in a non-constant output. As shown in Figure 3.11(c), incorporating a pinpoint tensor M adjusts effectual partitions into AAHRs, ensuring validity. This masking guarantees that the output tensor retains the piecewise-constant property. Figure 3.12 shows an SMT-based validity-checking algorithm, illustrated with Figure 3.11(c), that tests whether effectual partitions form AAHRs given tensor dimensions with nonzero pinpoints or intervals.

For the second criterion, expressions within each partition must remain constant to ensure that the computation remains constant throughout the partition. For example, the program $C_x = A_x * (x + 1)$ with a piecewise-constant vector A is invalid because the term $(x + 1)$ varies within any partition unless all effectual partitions are pinpoints. This check is only required for rank-variable expressions outside of tensor accesses. Such expressions are allowed only when the partition is guaranteed to be a pinpoint. To verify this, we use the same code structure as the AAHR check, but replace the AAHR with a pinpoint to determine whether the partition is pinpoint.

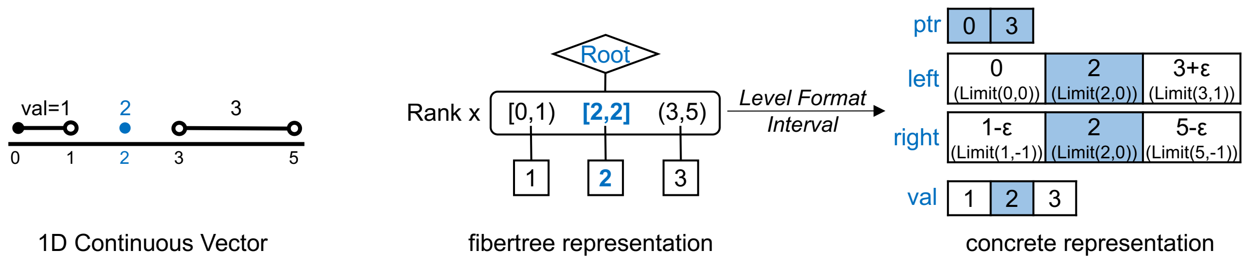


Figure 3.13: A 1D continuous vector, its interval-typed fibertree abstraction and concrete representation.

3.4 Continuous Tensor Storage

This section will explain how the continuous tensor is stored in memory under the piecewise-constant assumption using a fibertree abstraction.

3.4.1 Interval Coordinates

We introduce interval-typed coordinates in a fibertree to effectively capture piecewise-constant properties. Figure 3.13 illustrates an interval-typed fibertree for a continuous tensor `A`. *Pinpoint coordinates* are a special case, treated as intervals with both endpoints equal, collapsing to a single point; a pinpoint coordinate `2` is equivalent to the closed interval `[2,2]`. We account for the inclusiveness (open or closed) of each endpoint, resulting in four distinct interval subtypes. Similar to how traditional level formats operate on fibers with integer coordinates, we designed several level formats with interval coordinates, with the *Interval level format* being assigned for rank x in this example. Interval level format encodes the number and inclusiveness pair with `Limit` type.

Interval Level Format with `Limit` Type.

To represent the interval inclusiveness (open and closed endpoints), we introduce a new number type called `Limit` for encoding interval endpoints. The `Limit` type incorporates an infinitesimal number, denoted by ϵ, which is smaller than any positive real number but not

Table 3.1: Four inclusiveness types of intervals. In our implementation, we treat these as a single closed interval representation with the use of the infinitesimal number ϵ stored in `Limit` type.

Name	Notation	Definition	Treated as	Implemented as
Closed interval	$[a, b]$	$\{x \in \mathbb{R} \mid a \leq x \leq b\}$	$[a, b]$	`[Limit(a,0), Limit(b,0)]`
Right half-open interval	$[a, b)$	$\{x \in \mathbb{R} \mid a \leq x < b\}$	$[a, b - \epsilon]$	`[Limit(a,0), Limit(b,-1)]`
Left half-open interval	$(a, b]$	$\{x \in \mathbb{R} \mid a < x \leq b\}$	$[a + \epsilon, b]$	`[Limit(a,+1), Limit(b,0)]`
Open interval	(a, b)	$\{x \in \mathbb{R} \mid a < x < b\}$	$[a + \epsilon, b - \epsilon]$	`[Limit(a,+1), Limit(b,-1)]`

```
# Definition of Limit.
struct Limit<T>
    val::T    # Numeric type T (e.g., Int, Float32) chosen by the user
    eps::Int8 # (+ϵ, 0, -ϵ) = (+1, 0, -1)
end
# Example Operations defined on Limit.
(+)(x::Limit, y::Limit) = Limit(x.val + y.val, min(max(x.eps + y.eps, -1), +1))
(-)(x::Limit, y::Limit) = Limit(x.val - y.val, min(max(x.eps - y.eps, -1), +1))
(<)(x::Limit, y::Limit) = x.val < y.val || (x.val == y.val && x.eps < y.eps)
```

Figure 3.14: Implementation of `Limit` type. Infinitesimal number is represented by `eps` field stored in `Int8` type.

zero. This approach allows us to treat all intervals as closed by default. We emulate open endpoints by adding or subtracting ϵ as needed, transforming closed intervals into open ones. Table 3.1 illustrates how this representation works for each inclusiveness category.

Figure 3.14 depicts the definition of `Limit` and arithmetic operations implemented using this numeric type. `Limit` serves as an augmented number type for real numbers, essentially a struct comprising a regular numerical value (`val::T`) and the infinitesimal value (`eps::Int8`).

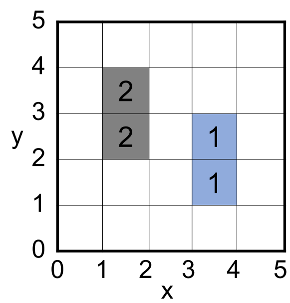
Although our compiler generates symbolic code that operates on real coordinates or intervals, actual computations are executed using finite-precision computer number types. This finite precision inevitably introduces representation and rounding errors, distinguishing practical computations from idealized continuous real-number arithmetic. The `Limit<T>` type supports various numeric types `T` (e.g., `Float32`, `Rational`, or `BigFloat` for arbitrary precision), allowing users to select the precision level that best fits their application's requirements. For consistent comparison, we matched the numeric type for real indices to each baseline in our case studies (Section 4).

3.4.2 Optimized Representation

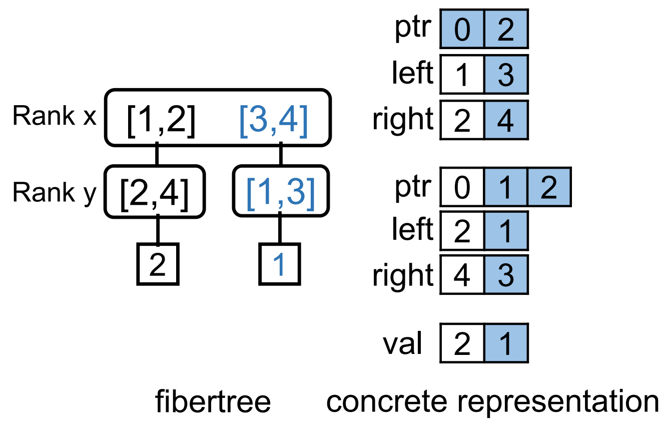
This subsection outlines optimized representations for storing tensors with specific interval patterns. Choosing appropriate level order and level formats results in notable benefits in terms of memory efficiency and the complexity of the generated code.

Impact of Level Order

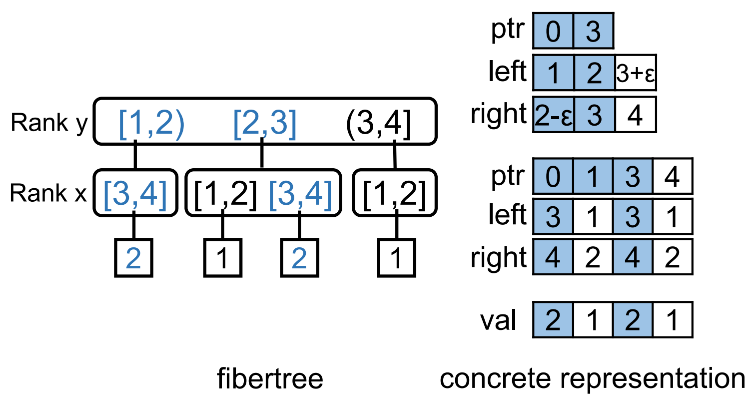
The careful selection of the level order is crucial for optimizing tensor operations in both memory consumption and performance [71]. As shown in Figure 3.15, the level order directly affects the structure of the fibertree and concrete representation. For instance, the order $x \to y$ and $y \to x$ results in different layouts. An optimal level order reduces redundancy in the storage format, minimizing memory usage by eliminating unnecessary position pointers. It also improves computation efficiency by reducing the overhead of irregular memory accesses.



(a) Boxes



(b) Level order $x \to y$



(c) Level order $y \to x$

Figure 3.15: Two concrete representations based on different level orders. With the level order $x \to y$ and $y \to x$, the alternative representation illustrates how memory usage can be reduced depending on the chosen order.

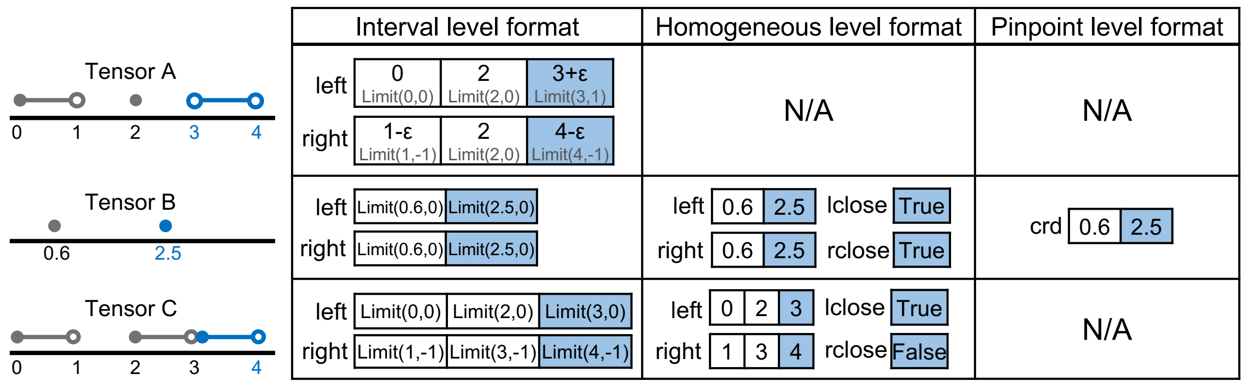


Figure 3.16: Level formats across three distinct 1D continuous tensors. Instead of storing every pair of (number, inclusiveness) in the `Limit` type, certain tensors benefit from optimized level formats.

Impact of Level Format

Figure 3.16 shows three tensors stored in various level formats. Depending on the pattern, certain tensors can benefit from a more optimized representation than storing every endpoint in the `Limit` type.

Homogeneous Level Format The need to store pairs of (number, inclusiveness) for every endpoint may vary, depending on interval inclusiveness. When all intervals within a level share the same inclusiveness, we refer to them as *homogeneous* intervals. Conversely, when intervals have inconsistent inclusiveness, they are categorized as *heterogeneous*. Tensor B and C in Figure 3.16 are homogeneous but tensor A is heterogeneous. In the case of homogeneous intervals, there is no need to store every endpoint in the `Limit` type. Instead, the level format can store endpoints in regular numeric types while keeping inclusiveness details separate using `lclose` and `rclose`.

Pinpoint Level Format If the level contains only pinpoint coordinates, there is no need to store both endpoints for each coordinate, as they share the same endpoints. Instead, pinpoints can be stored as single coordinates using the regular number type, as depicted in Tensor B in Figure 3.16.

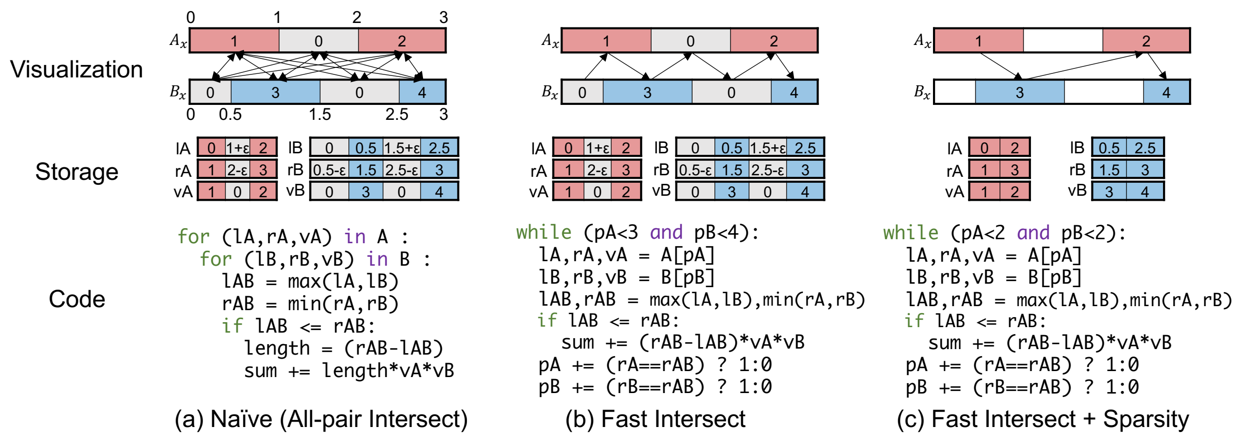


Figure 3.17: Three strategies for partitioning the iteration space for $\boldsymbol{sum} = \boldsymbol{A_x} * \boldsymbol{B_x} * \boldsymbol{d(x, \mu_\lambda)}$. `lT`, `rT`, and `vT` represent the left endpoints, right endpoints, and values of the interval of the tensor T, respectively. **(a) Naïve approach**: Computes all intersections using exhaustive pairwise comparisons. **(b) Fast Intersect**: Speeds up intersection using a two-pointer technique on sorted arrays. **(c) Fast Intersect + Sparsity**: Further improves (b) by leveraging sparsity to skip ineffectual partitions. Our compiler generates this final code.

3.5 Implementation

While the piecewise-constant evaluation semantics describe partitioning the iteration space via all-pairs intersection, we found that this process can be significantly optimized in practice. Figure 3.17 illustrates two optimizations, applied on top of the naïve baseline, that make the generated code more efficient. First, when partitioning the iteration space, it is unnecessary to compute all pairwise intersections between every interval of each tensor. Second, we can skip computations for partitions in which ineffectual operations occur, such as $a \times 0 = 0$, by storing only nonzero intervals. These two ideas are independent of any particular compiler, and both carry over to the implementation developed later in this thesis.

In the original work [72], we realized these optimizations by extending Finch [21]—a compiler originally designed for sparse tensor computation in the integer domain using Looplets [20]—into the continuous domain. Looplets partition the iteration space efficiently, and repeated rewriting and optimization passes then incorporate mathematical properties such as sparsity to enhance efficiency. We refer the reader to that work for the full code-generation details, including how Looplets are lowered into loops.

The next chapter briefly summarizes only the continuous-specific extensions to the Finch implementation before evaluating the resulting kernels. Later Chapters 5 and 7 realize the same optimization ideas through indirect Einsums, an approach we build from scratch on a different technology stack and which forms a central contribution of this thesis.

Chapter 4

Evaluation of Continuous Einsums with Finch

In this chapter, we explore diverse applications of the continuous tensor abstraction across four domains: (1) geospatial search, (2) genomic interval operations in bioinformatics, (3) interpolation in Neural Radiance Fields, and (4) 3D point cloud convolution. Our goal is to show the simplicity and clarity with which these applications can be expressed in our abstraction, particularly compared to the challenges they pose in existing tensor programming frameworks (Table 4.1). Using the Finch-based implementation from our original work, we assess the performance of the generated code by comparing it against hand-written libraries. All experiments are conducted with single-threaded execution on a MacBook Pro M2 Max with 32GB of memory. Note that our generated code may differ from the actual algorithms or data structures used in the baselines. We provide specific details in each subsection.

4.1 Finch Implementation

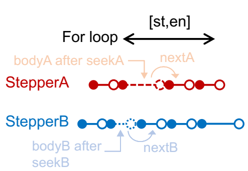
The experiments in this chapter use the prototype compiler from our original publication [72], which extends Finch [21] and its Looplet lowering framework [20]. Looplets describe an operand's data pattern by composing small iteration constructs, allowing Finch to generate

Table 4.1: Lines of code comparison between the baseline and ours.

Applications	**Baseline**	**Ours**	**LoC Saving**
Radius Search Query	501 lines	8 lines	**62**×
Genomic Interval Overlapping Query	206 lines	11 lines	**18**×
Trilinear Interpolation in NeRF	82 lines	13 lines	**6**×
Point Cloud Convolution	2,330 lines	23 lines	**101**×

co-iteration code for different storage formats. Informally, they resemble regular expressions over data patterns: a *Run* describes a region of repeated values; if R and S denote two region patterns, a *Sequence* concatenates them as RS, while a *Stepper* repeatedly applies R, analogous to the Kleene star R^*. We refer the reader to the original Looplets paper for a thorough introduction and the full lowering design. Here, we summarize only the extensions required for continuous tensors.

The compiler takes a continuous Einsum and a Looplet description of each operand. Looplets expose the endpoints at which values change, so lowering traverses finitely many pieces rather than individual real-valued points. Runs, Phases, and Sequences lower as they do in Finch. The only continuous-specific change to Looplet lowering is in the Stepper: after processing an intersection, it advances to `curr.stop` $+ \epsilon$, rather than to the next integer. The `Limit` type encodes this infinitesimal step and the inclusiveness of the shared endpoint. Figure 4.1 shows the source loop and the lowered Stepper code.



(a) Pattern.

```
A = Stepper(seekA, bodyA, nextA)
B = Stepper(seekB, bodyB, nextB)
for x = st:en
  s += A[x] * B[x]
```

(b) Source loop.

```
call seekA(st)
call seekB(st)
x = st
while x <= en
  curr = [x,en] ∩ bodyA.range ∩ bodyB.range
  for x2 = curr.start:curr.stop
    s += bodyA[x2] * bodyB[x2]
  if curr.stop == bodyA.stop; call nextA
  if curr.stop == bodyB.stop; call nextB
  x = curr.stop + ϵ
```

(c) Lowered loop.

Figure 4.1: Continuous Stepper lowering.

Continuous reductions collapse each constant piece. An integral contributes its value times the interval length, whereas a counting reduction contributes only for a pinpoint.

Because endpoint inclusiveness does not change Lebesgue measure, the compiler discards the infinitesimal component of the interval length. Figure 4.2 shows both rewrites.

```
# Rewrite a continuous reduction
rewrite_rule(for idx in interval; body) => begin
  apply_rule((op=)(lhs, rhs*d(idx, measure)) =>
  begin
    if rhs is constant with respect to idx
      reduce(idx, interval, lhs, op, rhs, measure)
    end
  end)(body)
end
```

```
st = Limit(3.0, +1) # 3.0 + epsilon
en = Limit(4.2,  0) # 4.2
for x = st:en
  s += Va * Vb * d(x, Lebesgue)
# lowers to
s += Va * Vb * drop_eps(en - st)
```

(a) Rewrite and example.

```
drop_eps(x::Limit) = x.value

# Integral mode
reduce(idx, interval, lhs, +=, rhs, Lebesgue) =
  rhs_value = remove d(idx) from rhs
  interval_length = length(interval)
  lhs += rhs_value * drop_eps(interval_length)

# Counting mode
reduce(idx, interval, lhs, +=, rhs, Counting) =
  if length(interval) == 0
    lhs += rhs
  end
```

(b) Measure-specific rules.

Figure 4.2: Continuous reduction rewriting.

Coiteration also introduces symbolic `min`, `max`, and interval-emptiness checks. The compiler uses relationships implied by the Looplet nest and Z3 [73] to prove and remove redundancies. Figure 4.3 shows the code before and after Z3 simplifies a check of the intersection length between a pinpoint and an interval.

```
ABStart = max(A.start, B.start)
ABEnd = min(A.end, B.end)
if ABStart <= ABEnd:
    Length = ABEnd - ABStart
    if Length == 0:
        Sum += val
```

(a) Before Z3.

```
ABStart = max(A.start, B.start)
ABEnd = min(A.end, B.end)
if ABStart <= ABEnd:
    Sum += val
```

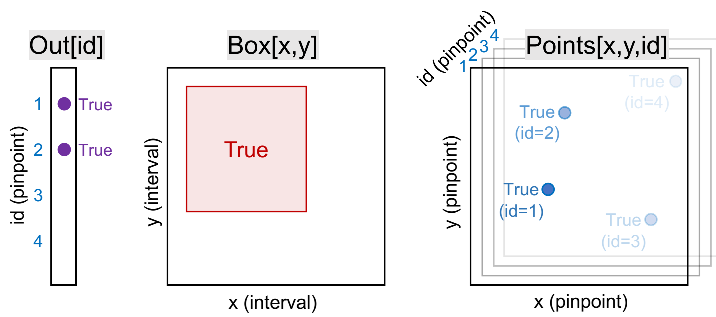
(b) After Z3.

Figure 4.3: Z3 eliminates a redundant pinpoint-length check.

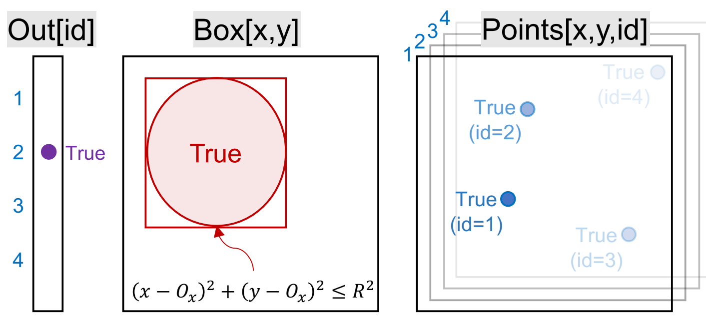
Together with Finch's existing simplifications for ineffectual computations, these changes implement the piecewise-constant evaluation semantics from Section 3.3.2 and produce the Julia kernels.

4.2 Geospatial Search

The first application we have explored is the spatial search query on 2D points, a widely used technique in applications such as geographical information systems (GIS) [74], computer-aided design (CAD) [75], and spatial databases [76]. In our study, we focused on two commonly employed queries: (1) the box search and (2) the radius search. In a box search, given a set of 2D points, this query retrieves all points within a specified box. A radius search query retrieves all points within a circle centered at $(O_x,\ O_y)$ with a radius of R.

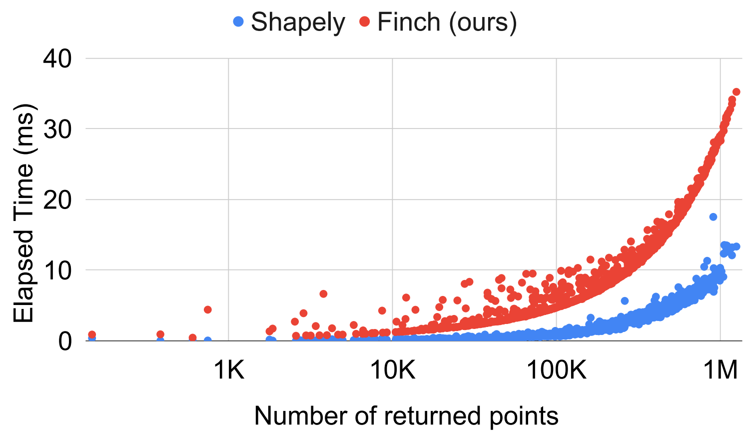


(a) Box search query

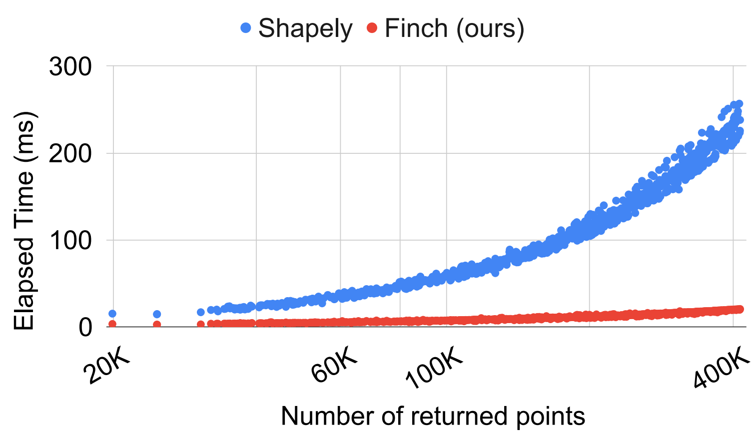


(b) Radius search query.

Figure 4.4: Illustration of two spatial searches. Points are represented as a 3D tensor `Points[x, y, id]`, with each point assigned a unique ID. `Out` retrieves the `id` of points intersecting a specified box or circle. The box search (panel (a)) is expressed as $Out_{id} = (Box_{x,y}\ \&\&\ Points_{x,y,id}) * d(x, y; \mu_{\wedge\vee}, \mu_{\wedge\vee})$, and the radius search (panel (b)) as $Out_{id} = Box_{x,y}\ \&\&\ Points_{x,y,id}\ \&\&\ (x - O_x)^2 + (y - O_y)^2 \leq R^2$. The measure on the radius search is omitted for brevity; it also uses the boolean measure $d(x, y; \mu_{\wedge\vee}, \mu_{\wedge\vee})$.



(a) Box search with increasing box size.



(b) Radius search with increasing radius.

Figure 4.5: Experimental results of spatial queries; lower values indicate better performance.

Figure 4.4 demonstrates how spatial search queries are represented in continuous tensor abstraction. We transform 2D points into a 3D continuous tensor, denoted as $Points_{x,y,id}$, by assigning a unique ID to each point (x, y). This approach accommodates multiple points that may share the same coordinates (x, y) depending on the dataset. Figure 4.4(a) illustrates a box query, which outputs IDs of points intersecting with $Box_{x,y}$. Similarly, Figure 4.4(b) presents a radius search query where the circle is defined with $(x - O_x)^2 + (y - O_y)^2 \leq R^2$.

Figure 4.5 presents the performance of our generated code in comparison to Shapely [60], a Python wrapper for GEOS [59], a well-known C++ library widely used in GIS for performing

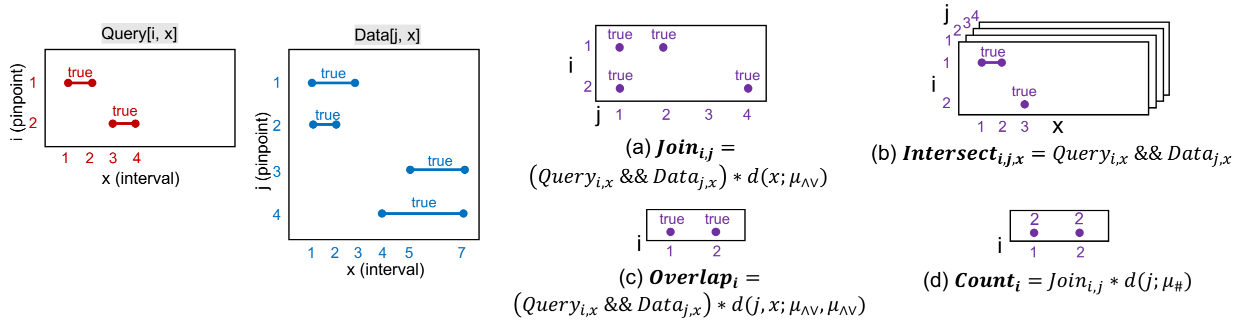


Figure 4.6: Four different genomic interval operations in continuous tensor abstraction. Genomic intervals are represented as 2D continuous tensor ($Query$ and $Data$) where white regions indicate 'false' boolean values.

operations on two-dimensional geometries. We employed a synthetic dataset that uniformly distributed 10 million points in the range [0,10000] × [0,10000]. In both experiments, we increased the size of the query shape along the x-axis to augment the number of returned output points.

Figure 4.5(a) shows that Shapely outperforms our generated code on the box search query, achieving a geometric mean speedup of 4.7×. This advantage is mainly due to Shapely's use of an advanced spatial data structure, STRtree [62], which accelerates search operations. In contrast, our code does not employ any spatial data structures. However, Figure 4.5(b) shows that our code surpasses Shapely on the radius query by a geometric mean of 9.2×. This improvement ironically stems from Shapely's reliance on STRtree, which only supports box queries. For a circular query, Shapely retrieves all points within the bounding box of a circle using STRtree and then performs a linear scan to check if each point lies within the radius R. In contrast, our generated code directly iterates within the circular region (Figure 4.4(b)), avoiding the inefficiency of Shapely's two-step approach.

4.3 Genomic Interval Operations

The second application we've explored is genomic interval operations using our continuous abstraction. In Bioinformatics, performing operations on genomic sequences [22] and applying

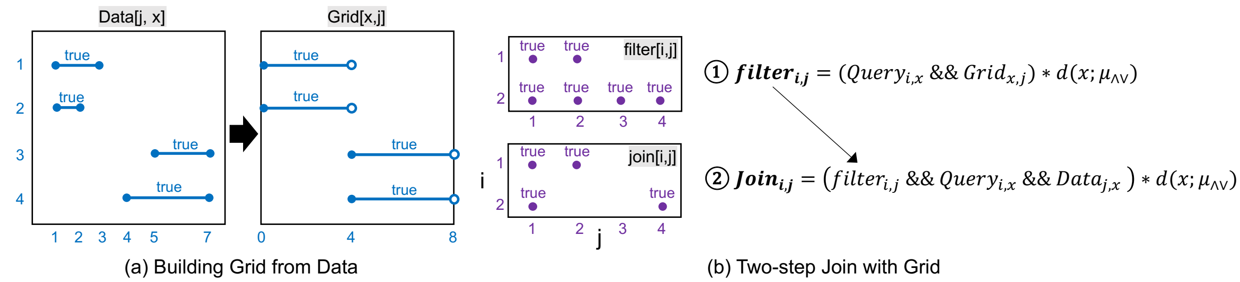
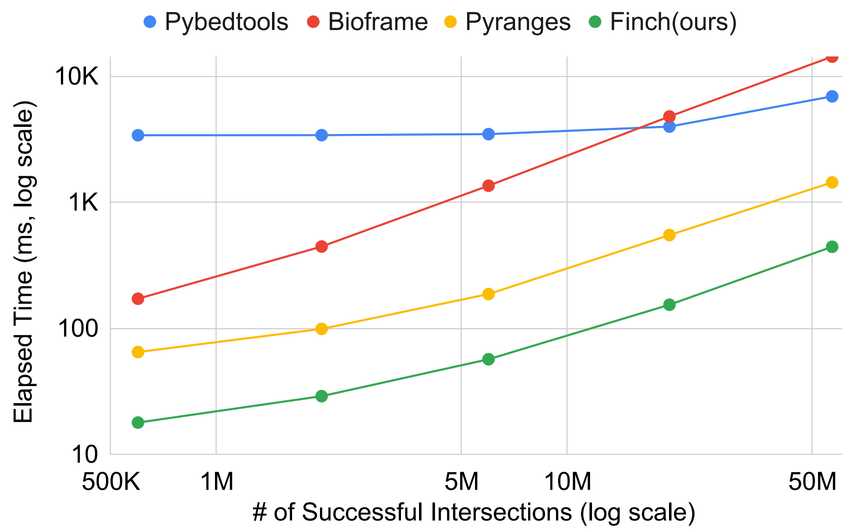


Figure 4.7: The uniform grid splits the x-dimension of 'Data' into two intervals, $[0, 4)$ and $[4, 8)$, to streamline intersection tests by eliminating irrelevant data points. (a) illustrates the construction of the grid. (b) demonstrates a two-step join operation using the grid: the first step ($filter_{i,j}$) prunes unnecessary pairs (i, j) with the grid structure, while the second step ($Join_{i,j}$) performs the actual join operation only on filtered pairs.

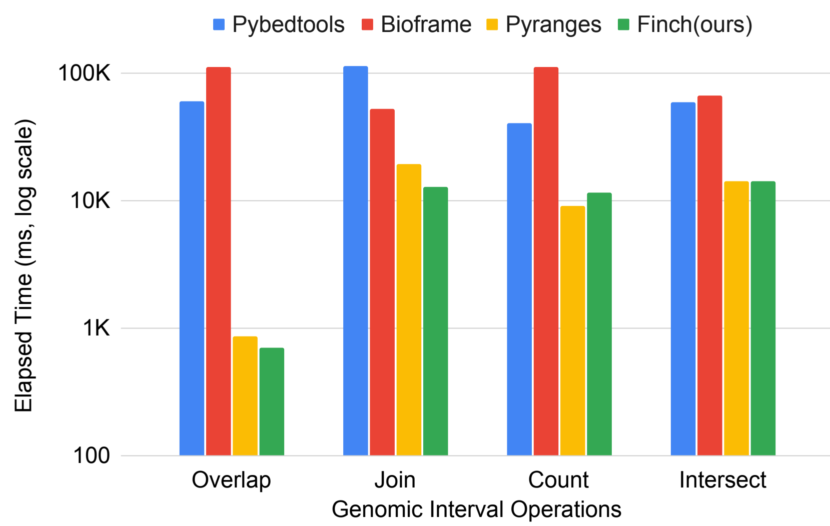
boolean operations to genomic interval data can be computationally intensive.

In Figure 4.6, genomic interval data is depicted in a 2D continuous tensor, denoted as the interval database $Data_{j,x}$ and query intervals $Query_{i,x}$, each identified by a unique interval ID. Right side of the figure illustrates four genomic operations written in the continuous tensor abstraction. For example, $Count_i$ counts the number of intervals in $Data$ intersecting with a query interval for each query ID i. These programs are expressed clearly and succinctly in the continuous tensor abstraction. However, the naive version involves pairwise comparisons $(N \times M)$ between all query and data intervals (i, j), incurring substantial computational costs when handling numerous intervals.

In practice, previous studies [22,50,63,77,78] have used an interval data structure to skip unnecessary comparisons. We demonstrated a uniform grid [77] using our abstraction to partition the continuous dimension x into exclusive partitions, encompassing the entire dimension x. The uniform grid, represented in a 2D continuous tensor as $Grid_{x,j}$, is illustrated in Figure 4.7. It divides the x domain of $Data_{j,x}$ into two halves: [0,4) and [4,8), with interval IDs j allocated to the respective partitions. The right side of the figure shows an example on a Join operation using the uniform grid. First, the uniform grid filters out irrelevant intervals in Data, reducing the need for pairwise comparisons. In the second stage, it further eliminates false positives to refine the final results.



(a) Sensitivity to the number of successful intersections.



(b) Performance on a real-world dataset.

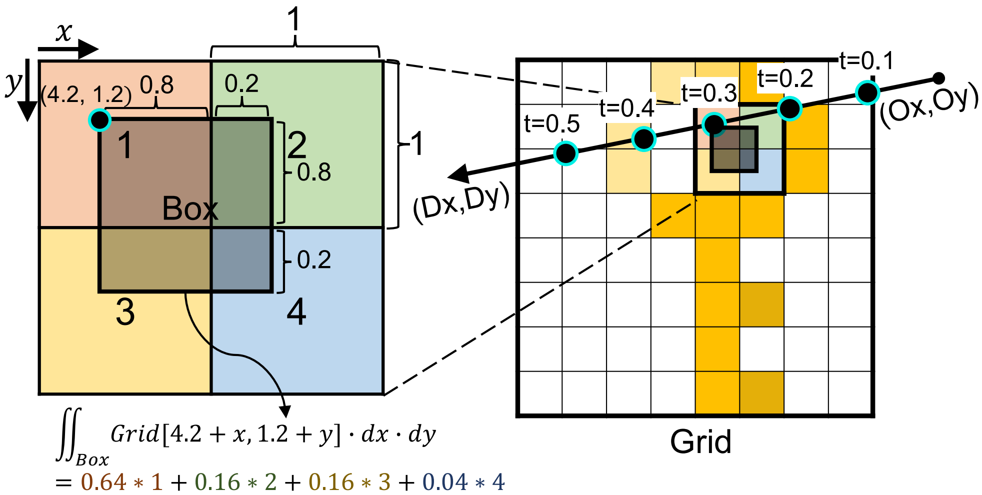
Figure 4.8: Experimental results of genomic interval operations; lower values indicate better performance.

Figure 4.8 presents a performance comparison between our generated code using uniform grid and three baseline implementations: Pybedtools [79], Bioframe [80], and Pyranges [81]. Pybedtools is a Python wrapper of Bedtools [22], a C library which utilizes a hierarchical binning data structure internally, Bioframe leverages the Pandas [82] framework for genomic interval operations, and Pyranges employs a Nested Containment List [50], a variation of the segment tree written in C. Index building time was not measured in these experiments.

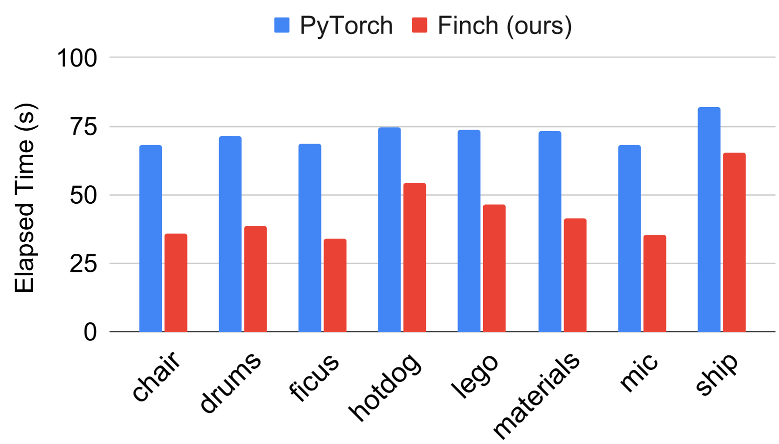
In Figure 4.8(a), a sensitivity test examines the number of successful intersections using a synthetic dataset. The intervals are uniformly distributed, maintaining a total of 100,000 intervals in both `Data` and `Query`. As the x-axis increases, the length of intervals in both `Data` and `Query` is extended to increase the number of intersections between intervals. Figure 4.8(b) presents a performance comparison on a realistic dataset [78], with `Data` containing 8,942,869 intervals and `Query` containing 1,193,657 intervals. Our generated code demonstrates superior or comparable performance in both synthetic and realistic datasets, with the advantage of being implemented in just 11 lines of code for the Overlap operation, while Bedtools implementation require 206 lines of code.

4.4 Trilinear Interpolation in Neural Radiance Field

The third application we explored is a Neural Radiance Field (NeRF) [58] in 3D deep learning. NeRF is a widely used machine learning model in computer graphics and computer vision that generates detailed 3D reconstructions from 2D images. Many NeRF models [57,83,84] use trilinear interpolation on 3D sparse voxel grids to efficiently represent the 3D scene. In NeRF, rendering a 2D image involves casting rays, sampling points along each ray, performing trilinear interpolation on the sparse voxel grid for each sampled point, and combining these results to compute the final RGB color. Our focus here is on optimizing trilinear interpolation during ray sampling in Plenoxel [57].



(a) Interpolation at sampled points on a sparse grid, shown in 2D for illustration.



(b) Performance on the NeRF Synthetic dataset [58]; lower is better.

Figure 4.9: Trilinear interpolation for Neural Radiance Fields.

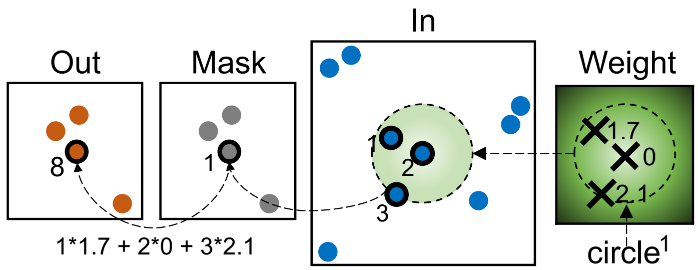
For explanatory purposes, we illustrate a 2D bilinear interpolation in Figure 4.9(a), even though the actual computations are in 3D. This interpolation is applied at each sampled point along the ray by calculating the intersected area using integral reduction:

$$Out_t = Time_t * Grid_{O_x+D_x*t+x, O_y+D_y*t+y} * Box_{x,y} * d(x, y; \mu_\lambda, \mu_\lambda),$$

where (O_x, O_y) and (D_x, D_y) are the ray's origin and direction, respectively. $Box_{x,y}$ is a binary tensor defining the interpolating region, and $Time_t$ is a binary tensor marking the specific time step for sampling along the ray. In practice, these calculations occur in 3D space.

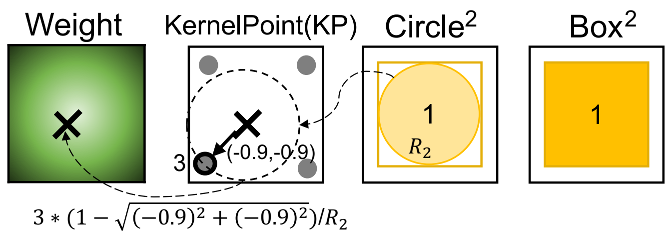
Figure 4.9(b) presents a performance comparison between our generated code and a PyTorch implementation of trilinear interpolation in Plenoxel. The evaluation was conducted while rendering a 256×256 image using the NeRF Synthetic dataset [58]. Our code achieves a speedup of 1.3× to 2.0× over the baseline. This improvement is mainly due to our efficient handling of sparse voxel grids, as we store only non-zero voxels and avoid unnecessary computations. In contrast, the PyTorch implementation uses a full voxel-grid array, storing even empty voxels and performing calculations regardless of voxel occupancy.

4.5 3D Point Cloud Convolution



$$Circle^1_{r,s} = Box^1_{r,s} * (r^2 + s^2 \leq R_1^2)$$
$$Out_{x,y} = Mask_{x,y} * In_{x+r,y+s} * (Weight_{r,s} * Circle^1_{r,s})$$

(a) Kernel point convolution.



$$Circle^2_{p,q} = Box^2_{p,q} * (p^2 + q^2 \leq R_2^2)$$
$$Weight_{r,s} = KP_{r+p,s+q} * \left(\left(1 - \sqrt{p^2 + q^2}/R_2\right) * Circle^2_{p,q} \right)$$

(b) Definition of $Weight$, which interpolates the points in $KernelPoint$ within radius R_1.

Figure 4.10: Kernel point convolution on input point clouds written in the continuous tensor abstraction. The measure is omitted for brevity; all reductions use counting measures.

The last application we explored involves 3D point cloud convolution [52,85,86]. A 3D point cloud is a set of points represented by xyz coordinates in 3D space, capturing the structure of an object. In 3D deep learning, convolutions are adapted to operate on point clouds rather than grid-like image data, necessitating specialized techniques. KPConv [54] is a notable example of such a technique, and we demonstrate its implementation using our continuous tensor abstraction.

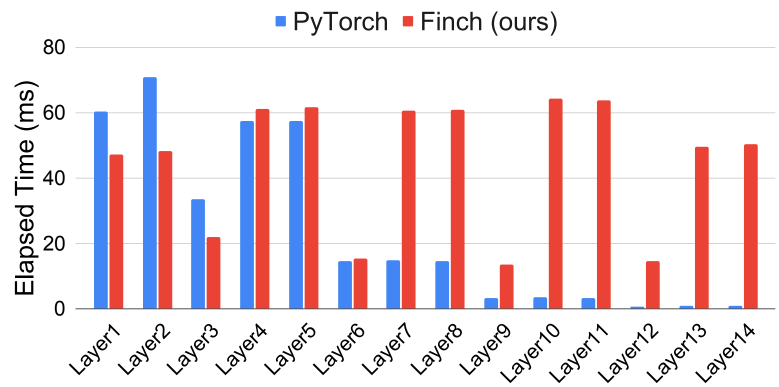
Figure 4.10 shows how KPConv, represented in 2D for simplicity, can be formulated within continuous tensor expression. In Figure 4.10(a), KPConv performs convolutions selectively on specific points, marked by a binary mask, $Mask$. For each masked point, the neighboring input points, $In_{x+r,y+s}$, within a radius R_1 are gathered and convolved with a continuously varying weight, $Weight_{r,s}$, which depends on the relative position of each neighboring point. The radius-constrained region is indicated by $Circle_{r,s}$, which determines which neighboring points are gathered.

As shown in Figure 4.10(b), the continuous weight $Weight_{r,s}$ is defined by interpolating values at fixed set of "kernel points" within a radius R_2. This interpolation is distance-based, relying only on kernel points within the radius. Notably, the interpolated weight $Weight_{r,s}$ is not a piecewise-constant, but is evaluated only on pinpoints selected by the mask $Mask_{x,y}$ in the main convolution.

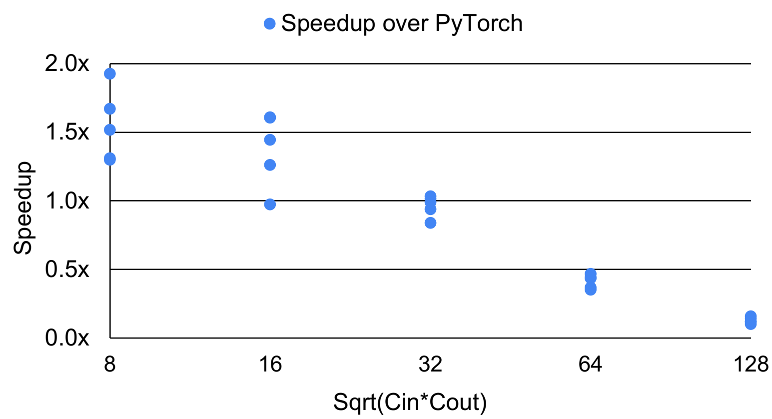
By using continuous tensor abstraction, complex point cloud convolutions that are challenging to implement in traditional tensor programming models can be expressed in a simplified manner. In contrast, the original PyTorch implementation of KPConv requires 2,330 lines of code, including dependencies for neighborhood search libraries [87]. With our abstraction, users can specify the whole KPConv operation as a single Einsum:

$$Out_{x,y,m} = Mask_{x,y} * In_{x+r,y+s,c} * (KP_{r+p,s+q,m,c} * (1 - \sqrt{p^2+q^2}/R_2^2) * Circle^2_{p,q}) * Circle^1_{r,s} * d(r,s,c; \mu_\#, \mu_\#, \mu_\#)$$

where m and c are indices for the output and input channels, respectively, along with the traversal order of indices and the choice of storage format.



(a) Performance by convolutional layer; lower is better.



(b) Speedup over PyTorch by channel size; higher is better.

Figure 4.11: Experimental results of 3D point cloud convolution.

Figure 4.11 presents the experimental results comparing our generated code with the PyTorch implementation of KPConv. In Figure 4.11(a), the elapsed time of each KPConv layer in the 3D shape classification model architecture using the ModelNet40 [88] dataset is shown. In the initial layers (layers 1-6), our code either outperforms or matches the performance of the PyTorch implementation. However, starting from layer 7, PyTorch begins to outperform our generated code.

We found that the later layers have larger channel sizes, resulting in more dense computation. PyTorch leverages highly optimized BLAS routines for this large tensor processing. Figure 4.11(b) further illustrates this difference by showing the speedup over PyTorch as channel size increases. In the initial layers, where the channel sizes are small ($\sqrt{C_{in} \cdot C_{out}} \leq 32$), our generated code performs better. However, in the later layers with larger channel sizes, where dense computation becomes more significant, PyTorch's implementation excels. Consequently, our code performs better in the early layers, where the radius query is the primary computational component, while the PyTorch implementation dominates in the later layers, where dense computation is the primary workload.

Chapter 5

Compiling Discrete Einsums with Format-Aware Computation

5.1 Introduction

The previous chapter demonstrated that continuous tensor programs can be compiled to executable code by extending an existing sparse tensor compiler: generalizing the Looplet mechanism of Finch from integer coordinates to intervals yields correct and reasonably efficient CPU implementations. In building on this infrastructure, however, we also inherited its limitations, and these limitations point at a broader problem in sparse tensor compilation.

The first limitation concerns hardware. Sparse tensor compilers, including Finch, have largely targeted CPUs, and for good reason: their central contribution is compiling coordinate intersection and union—two-finger merges and irregular loop nests—into correct, efficient co-iteration code, and the CPU's flexible control flow is a natural fit for that task. Achieving high performance on GPUs, by contrast, demands a largely orthogonal set of efforts: staging data through shared memory, invoking tensor cores, coalescing memory accesses, and keeping thousands of threads uniformly busy. None of this falls within the scope of sparse–sparse co-iteration, so existing sparse compilers have generally treated it as secondary. Figure 5.1

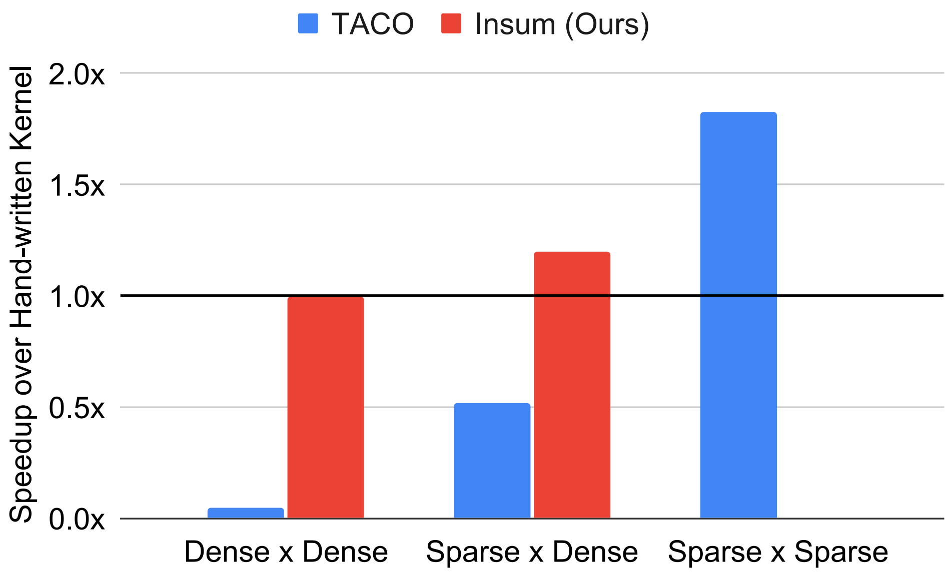


Figure 5.1: Speedup over the best hand-written vendor kernel (cuBLAS GEMM, cuSPARSE SpMM, cuSPARSE SpMSpM) across dense, single-sparse, and fully sparse matmuls. TACO wins on fully sparse but degrades as density grows; Insum (ours) targets the single-sparse regime, matching or beating the baseline while retaining full dense performance.

shows the consequence: TACO beats the vendor baseline on SpMSpM, where fast intersection code is what matters, but falls well short of hand-written kernels as computation becomes denser and these GPU-specific optimizations dominate performance.

The second limitation concerns the structure of real workloads. Many important sparse applications—sparse convolution on point clouds, sparse-dense matrix multiplication (SpMM), equivariant tensor products—are not sparse-sparse computations at all. They are *sparse-dense*: a single sparse operand determines *which* computations to perform, while the bulk of the arithmetic is dense work over feature dimensions. This dense work is precisely what modern dense tensor compilers such as PyTorch's already optimize extremely well, with mature kernel fusion, tensor-core code generation, and GPU targeting. Sparse compilers, focused on the machinery of coordinate iteration, fail to exploit these dense components; dense tensor compilers, in turn, cannot express the irregular accesses that sparsity introduces. The result is that practitioners fall back on hand-written CUDA libraries—thousands of lines of code per application, each a research effort in its own right (Table 5.1).

Name	Struct. SpMM	Unstruct. SpMM	Equiv. Prod.	Sparse Conv.
`TorchBSR` [89]	202 LoC	✗	✗	✗
`Sputnik` [46]	✗	1918 LoC	✗	✗
`e3nn` [90]	✗	✗	225 LoC	✗
`TorchSparse` [23]	✗	✗	✗	4491 LoC
Indirect Einsums	1 LoC	1 LoC	1 LoC	1 LoC
LoC Saving	**202**×	**1918**×	**225**×	**4491**×
Insum Speedup	**1.95**×	**1.20**×	**3.81**×	**1.14**×

Table 5.1: Comparison of different libraries across applications. We develop a compiler, Insum, that reformulates format-agnostic Einsums into format-aware *indirect Einsums*. While each library is tailored to a specific application, our compiler-based approach supports all of them, achieving better speedups with significantly fewer lines of code.

This chapter develops a new code generation strategy that resolves this tension by reusing dense compiler infrastructure rather than rebuilding it. The key idea is a *format-aware reformulation* of format-agnostic Einsums: instead of hiding a tensor's storage format behind the compiler, we encode the format's data and metadata directly into the expression through indirect indexing, producing an *indirect Einsum* such as $C_{AI_p} = AV_p \times B_{AI_p}$. Once the format is expressed this way, a sparse computation becomes an array program with gather and scatter operations—a program that dense tensor compilers can fuse, optimize, and lower onto GPUs.

We develop this strategy in two chapters. The first chapter (this chapter) presents the approach in its simplest setting: discrete, integer coordinate iteration spaces with a single sparse operand. We show how format-agnostic Einsums over sparse tensors are rewritten into indirect Einsums, and present **Insum**, a compiler that lowers indirect Einsums onto PyTorch's FX graph representation, extended with native tensor-core matrix multiplication and a *lazy broadcasting* code generation technique. The chapter 7 generalizes the strategy along both axes at once: to an arbitrary number of sparse operands, and from discrete to continuous piecewise-constant tensors. This brings the continuous tensor abstraction of Chapter 3 onto GPU hardware, connecting the abstraction and the efficient execution.

5.2 Format-Aware Reformulation of Format-Agnostic Einsums

Sparse tensor compilers are built around a strict separation of concerns. The user writes a *format-agnostic* Einsum, such as $C_{m,n} = A_{m,k} \times B_{k,n}$ for matrix multiplication, that describes the computation over the logical tensors with no reference to how they are stored. The storage formats are declared separately, and the compiler bridges the two: given the expression and the formats, it synthesizes iteration code that walks the compressed representations, intersects coordinates, and skips ineffectual operations. This separation is one of the central achievements of the field [18]: a single expression can be compiled against any combination of formats, and the format can be changed without touching the algorithm.

The cost of this separation is that the compiler must absorb the entire complexity of the format into the code it generates. Turning compressed metadata into iteration code requires complex control flow, and as argued in the introduction, this binds sparse compilers to CPU-style code generation and prevents them from fully utilizing GPUs.

This chapter therefore uses a two-step approach. First, we reformulate the format-agnostic Einsum and its format declarations as a *format-aware* Einsum stated directly over the arrays that form the compressed representation. In this format-aware expression, every operand is an ordinary array, and the intersection logic is made explicit through indirect accesses. Second, we lower this format-aware Einsum to a computation composed entirely of operations that dense tensor frameworks already execute efficiently on GPUs.

5.2.1 A Walkthrough: SpMV in COO Format

To make the reformulation concrete, consider a format-agnostic matrix–vector multiply, $C_i = A_{i,k} \times B_k$, where A is sparse and B and C are dense. The most naïve implementation ignores sparsity altogether: store A as a two-dimensional array and invoke a dense matrix–vector product. This is trivially format-aware, since the expression operates directly on the

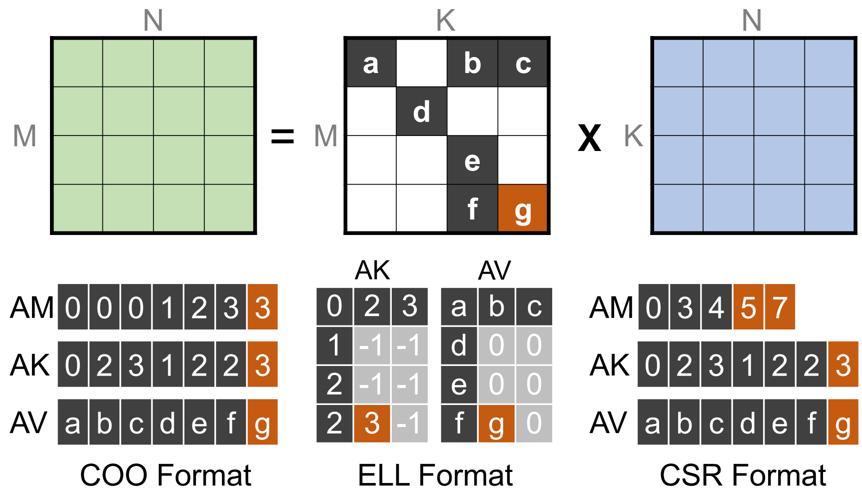


Figure 5.2: SpMM example ($C_{m,n} = A_{m,k} \times B_{k,n}$) and various sparse formats for the matrix A: COO, ELL, and CSR.

array, but it performs an ineffectual multiplication ($a \times 0 = 0$) for every zero in A.

To skip this wasted work, sparse tensors are stored in compressed formats that record only the nonzero values along with metadata describing their coordinates, as shown in Figure 5.2. In the COO format, the matrix A is represented by three flat arrays: `AV` holds the nonzero values, while `AM` and `AK` hold their row and column coordinates. The computation then proceeds nonzero by nonzero: for each position p, we *gather* the element of B selected by the column coordinate $\mathtt{AK}_p$, multiply it by the value $\mathtt{AV}_p$, and *scatter-add* the product into the output row selected by $\mathtt{AM}_p$. Figure 5.3 shows this format-aware computation as a PyTorch program when using the COO format.

Two observations follow from this small example. First, the format-aware computation requires no sparse compiler at all: it is three array operations, executable as-is by any dense tensor framework, and thus immediately portable to GPUs. All of the structure that a sparse compiler would have turned into specialized loops has instead become coordinate arrays that ordinary gather and scatter operations consume. Second, writing format-aware PyTorch programs by hand does not scale. Each application requires a separate implementation specialized to a particular format. Exploring alternative sparse formats therefore requires

```python
AM = torch.tensor([  0,   0,   2,   3])
AK = torch.tensor([  0,   3,   2,   0])
AV = torch.tensor([1.0, 2.0, 3.0, 4.0])
B  = torch.randn((4,))
C  = torch.zeros((4,))

def SpMV_COO(AM, AK, AV, B, C):
    BV = B[AK]                # Gather
    CV = AV * BV              # Multiply
    return torch.index_add( # Scatter
        C, dim=0, index=AM, source=CV
    )
```

Figure 5.3: Format-aware SpMV in COO format, expressed in PyTorch. The format's data (AV) and metadata (AM, AK) are ordinary arrays; the format's logic is expressed as a gather, an elementwise multiply, and a scatter-add.

tediously rewriting the same computation as a new sequence of gathers, scatters, and framework-specific operations each time. What is missing is a clean notation that captures format-aware computation, much as Einsums capture format-agnostic computation.

5.2.2 Extending Einsums with Indirect Indexing

We obtain such a notation by extending Einsums with *indirect indexing*: indexing a tensor via values drawn from another tensor. We call the resulting expressions *indirect Einsums*. For example, we allow $C_{X_i} = A_{Y_{i,j}} \times B_j$, where C_{X_i} and $A_{Y_{i,j}}$ are indirect accesses. All tensors appearing in an indirect Einsum are stored in arrays. Indirect indexing on the right-hand side corresponds to a gather, and on the left-hand side to a scatter. As in traditional Einsums, the output is implicitly initialized to zero, and multiple writes to the same output location are resolved by accumulation, consistent with the operational semantics of Einsums in prior work [26].

Under this notation, the format-aware SpMV of Figure 5.3 collapses to a single line:

$$C_{AM_p} = AV_p \times B_{AK_p}.$$

The expression is read as iterating over nonzero positions p: the access B_{AK_p} is the gather,

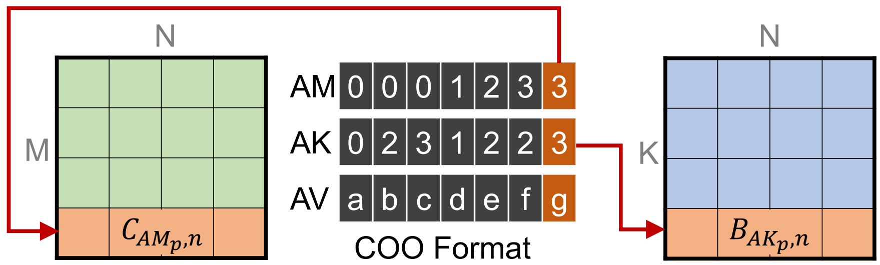


Figure 5.4: COO SpMM: $C_{AM_p,n} = AV_p \times B_{AK_p,n}$.

the product with AV_p is the elementwise multiply, and the indirect output access C_{AM_p} is the scatter with accumulation. The one-line indirect Einsum and the PyTorch program of Figure 5.3 denote exactly the same computation; the notation simply makes the format-aware structure explicit, just as classical Einsum notation makes format-agnostic structure explicit.

The same reformulation extends directly to SpMM, $C_{m,n} = A_{m,k} \times B_{k,n}$, the running example of Figure 5.2. Storing A in COO format and applying the rewriting yields the indirect Einsum shown in Figure 5.4:

$$C_{AM_p,n} = AV_p \times B_{AK_p,n},$$

in which the column rank n simply rides along with each gathered and scattered row. This example illustrates why the reformulation is a natural fit for the sparse–dense workloads motivating this chapter: the sparse operand contributes only the indirection pattern, while the ranks of the dense operands remain contiguous, regular work that existing dense tensor compilers can optimize on GPU.

In summary, the reformulation takes as input a format-agnostic Einsum together with the formats of its operands, and produces an indirect Einsum over the arrays underlying those formats. The remainder of this chapter develops this strategy into a compiler: we first show how the rewriting is derived systematically for a range of formats beyond COO, and then present how Insum lowers the resulting indirect Einsums onto PyTorch's FX graph representation, extended with native tensor-core matrix multiplication and a lazy broadcasting code generation technique.

Although indirect Einsums can express COO, they cannot yet express every sparse format. Widely used formats such as CSR, block CSR (BCSR), and CSF require loops whose lengths vary from one row or group to another. The next section explains this limitation and focuses on formats compatible with fixed-length loops.

5.3 Fixed-Length Sparse Formats

While the previous section explains how to express sparse computations in COO format using indirect Einsums, not all sparse formats are directly compatible with this approach. A fundamental limitation of the Einsum-based expression model is that the loop bounds for each rank variable must be fixed and cannot depend on data values. Many sparse formats, such as the CSR format, require data-dependent loop bounds. For instance, SpMM in CSR (Figure 5.2) iterates over rows, and within each row, it iterates over a variable number of nonzeros encoded in `AM`, as shown below:

```
for m in 0..M:
  for p in AM[m]..AM[m+1]: # variable-length loop
    for n in 0..N:
      C[m,n] += AV[p] * B[AK[p],n]
```

Because Einsums require fixed iteration spaces, variable-length loop bounds must be instead represented using a fixed-length encoding. COO does so by enumerating every nonzero, while ELL pads each row to a uniform length [91]. We refer to formats whose traversal can be expressed with fixed bounds as *fixed-length formats*. Their regular iteration structure maps naturally to indirect Einsums and enables efficient parallel execution on GPUs. Variable-length formats such as CSR offer a complementary tradeoff: they avoid padding and can perform better when row lengths are highly irregular, but require data-dependent iteration spaces that are not currently expressible using indirect Einsums. Therefore, our current format-aware approach focuses on fixed-length formats, and we show that this expressive subset, together with careful GPU optimization, is sufficient to outperform most state-of-the-art baselines. Extending indirect Einsums with data-dependent iteration spaces to support variable-length formats is left to future work.

The ELL format has the advantage of avoiding explicit storage of coordinates along the row rank, thereby eliminating the need for scatter operations. However, it can introduce significant padding overhead when the nonzero distribution is uneven. To address this, we propose a fixed-length sparse format called **GroupCOO**, which strikes a balance between COO and ELL by grouping nonzeros along a chosen rank. This reduces padding compared to ELL and requires fewer scatter operations than COO. We also show that GroupCOO extends naturally to block-sparse formats.

5.3.1 Grouping

We introduce **grouping**, a technique that derives the family of GroupCOO formats. To illustrate it, we walk through an example of SpMM, $C_{m,n} = A_{m,k} \times B_{k,n}$, where `A` is sparse.

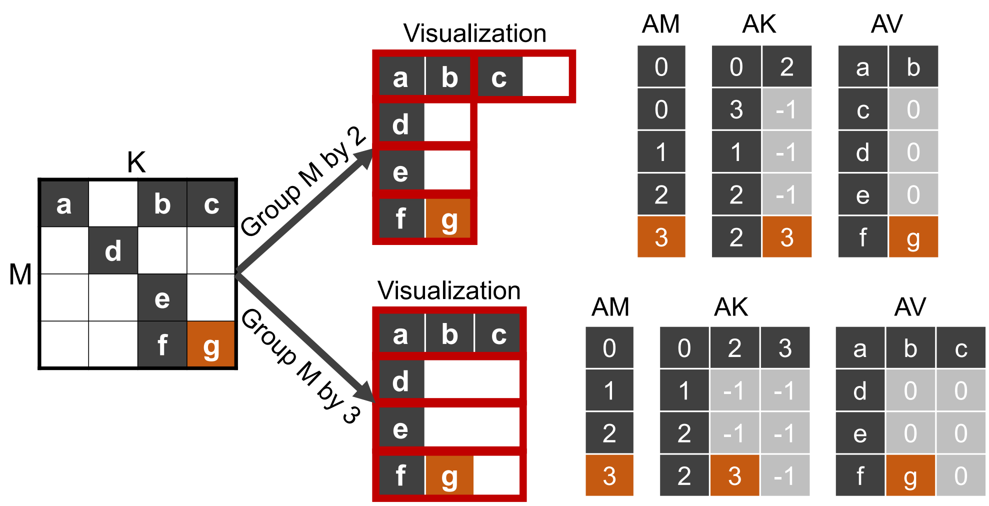


Figure 5.5: GroupCOO format with grouping on the `M` rank by group sizes 2 and 3. Grouping with the maximum number of nonzeros per row (size = 3) yields the ELL format.

Grouping: While the COO format is simple, it can be inefficient when the same row or

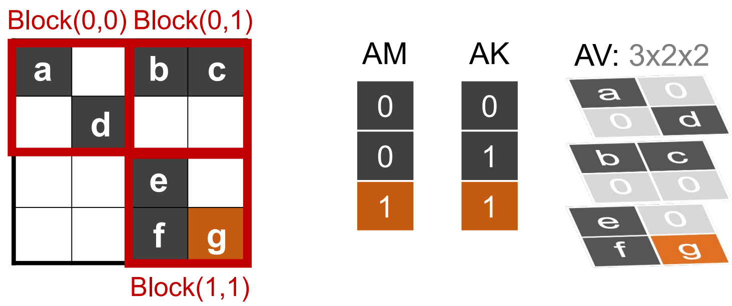


Figure 5.6: BlockCOO SpMM: $C_{AM_p,bm,n} = AV_{p,bm,bk} \times B_{AK_p,bk,n}$, where `bm` and `bk` are rank variables ranging over coordinates within each block.

column coordinates appear repeatedly. To reduce this redundancy, GroupCOO partitions nonzeros into **groups** along a chosen rank (e.g., rows) and stores the group coordinate only once per group. For example, in Figure 5.5, we group the nonzeros along the `M` rank (rows) using group sizes of 2 and 3. Within each group, we pad the data if necessary: the padding makes the format fixed-length, while sharing one coordinate per group reduces coordinate redundancy.

The GroupCOO SpMM can be written as follows, where `p` iterates over groups and `q` over elements within each group:

$$C_{AM_p,n} = AV_{p,q} \times B_{AK_{p,q},n}$$

GroupCOO sits between the COO format and the ELL format: setting the group size to 1 degenerates to the original COO format, while setting the group size to the maximum number of nonzeros per row yields the ELL format (Figure 5.5).

Group+Blocking: Blocking is a common technique in sparse computation to exploit local dense patterns by dividing a sparse matrix into dense blocks [19,71,89]. The BlockCOO format stores the coordinates of each nonzero block in `AM` (row block) and `AK` (column block), and the block values in `AV` as an array of shape `[num_blocks, bM, bK]`. SpMM in BlockCOO is expressed as shown in Figure 5.6.

Grouping can also be applied to block formats. In **BlockGroupCOO**, we group along

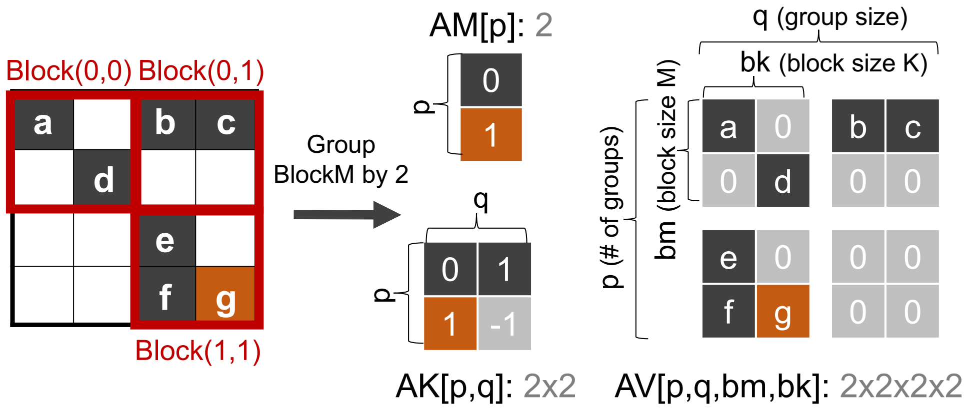
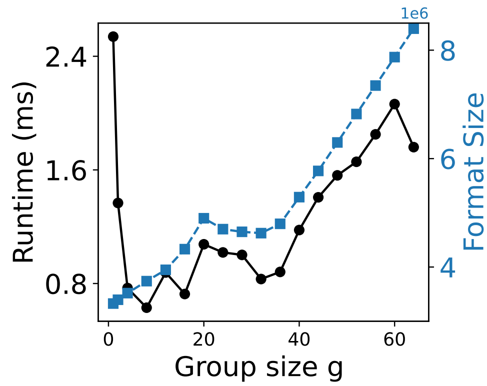


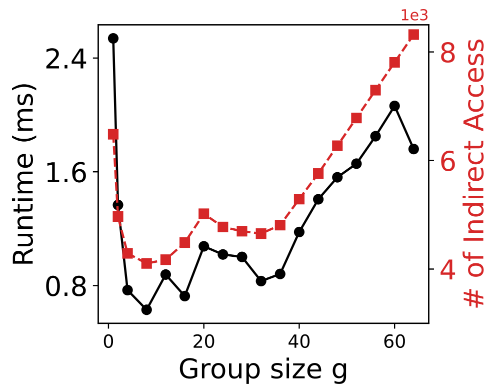
Figure 5.7: BlockGroupCOO SpMM: $C_{AM_p, bm, n} = AV_{p, q, bm, bk} \times B_{AK_{p, q}, bk, n}$, where `p` ranges over groups, `q` over entries within each group, and `bm`, `bk` over coordinates within each block.

one block-coordinate rank, so that `AV` has shape `[num_groups, group_size, bM, bK]`, and `AM`/`AK` store group-level and block-level coordinates. SpMM in BlockGroupCOO is expressed as shown in Figure 5.7.

5.3.2 Choosing Group Size



(a) Runtime vs. Format Size.



(b) Runtime vs. # of Ind. Access.

Figure 5.8: Runtime correlation with indirect access and format size under grouping. The number of indirect accesses (i.e., gathers and scatters) shows a stronger correlation with runtime than the total memory required.

Choosing an appropriate group size g is crucial for both storage footprint and compute efficiency. If g is too large, each row is padded to the next multiple of g, leading to wasted memory. If g is too small, the format degenerates into standard COO, losing the benefits of grouping. Ideally, we want to choose the group size g that yields the best runtime.

Our experiments suggest that minimizing *indirect memory accesses*, i.e., gathers and scatters, is the key to achieving high performance. These indirect accesses often result in expensive DRAM transactions due to poor spatial locality. To study this, we run Block-

GroupCOO SpMM on a 4096×4096 block-sparse matrix with 32×32 dense blocks at 80% sparsity, sweeping various group sizes g (Figure 5.8).

One proxy we tested for the optimal g is the one that minimizes total storage (i.e., the size of `AM`, `AK`, and `AV` in Figure 5.7), thereby reducing memory footprint. However, as shown in Figure 5.8(a), the format memory increases almost monotonically with larger g, especially in BlockGroupCOO, where padding in `AV` is amplified by the block size. Thus, runtime does not correlate well with the format size.

In contrast, Figure 5.8(b) demonstrates that runtime is closely aligned with the total number of *gathers and scatters* (i.e., accesses to `AM` and `AK`), which we define as $F(g)$:

$$F(g) = \underbrace{\sum_{i=0}^{n-1} \left\lceil \frac{occ_i}{g} \right\rceil}_{\text{AM: Scatter}} + \underbrace{g \sum_{i=0}^{n-1} \left\lceil \frac{occ_i}{g} \right\rceil}_{\text{AK: Gather}} = (g+1) \sum_{i=0}^{n-1} \left\lceil \frac{occ_i}{g} \right\rceil .$$

Here, n is the number of rows in the matrix and occ_i is the number of nonzeros in row i (Figure 5.5 gives $occ = [3, 1, 1, 2]$). With group size g, each row i is divided into $\lceil occ_i/g \rceil$ groups.

Finding the exact optimal g over the range $[1, \max_i occ_i]$ requires brute-force evaluation of $F(g)$, leading to $O(n \cdot \max_i occ_i)$ complexity. To obtain a much faster, yet nearly optimal estimate, we replace the ceiling with a conservative approximation: $\lceil occ_i/g \rceil \approx occ_i/g + 1$.

Letting $S = \sum_i occ_i$, the relaxed cost becomes:

$$\tilde{F}(g) = (g+1)\left(\frac{S}{g} + n\right) = S + \frac{S}{g} + ng + n.$$

Treating g as a real variable and differentiating:

$$\frac{d\tilde{F}}{dg} = -\frac{S}{g^2} + n = 0 \quad \Rightarrow \quad g^\star = \sqrt{\frac{S}{n}}.$$

This estimate provides a good approximation to the group size that minimizes indirect

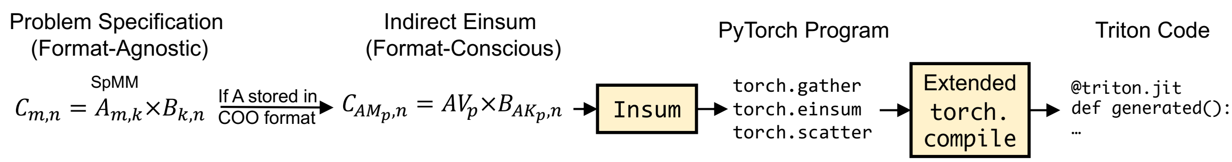


Figure 5.9: Overview of our Insum compiler. Instead of expressing sparse computations using format-agnostic Einsums as in prior approaches, users write indirect Einsums in a format-aware manner. This expression is translated into a PyTorch program and then lowered to a Triton kernel via our extended `torch.compile` pipeline.

accesses and likely achieves the best runtime. In practice, we round $g^\star$ to the neighboring power-of-two values and select the one with the better runtime. This choice is motivated by the fact that Triton, our backend, performs best with power-of-two block sizes, as evidenced by the downward spikes in Figure 5.8. This heuristic consistently yields efficient configurations across applications (Section 6).

5.4 Compiler Implementation

This section describes how we generate efficient GPU code from the indirect Einsum expressions produced by the reformulation of the preceding sections. We leverage a dense tensor compiler that operates on arrays, produces high-performance code, and supports gather/scatter operations. Among available options, we selected the PyTorch compiler [27] for four reasons: (1) it is Python-native and user-friendly; (2) it supports automatic differentiation, essential for sparse deep learning; (3) it targets Triton, enabling tensor core support without manual scheduling [40]; and (4) it delivers state-of-the-art performance among existing dense tensor compilers [27]. Figure 5.9 shows an overall pipeline of our Insum compiler.

5.4.1 Insum: Rewriting Indirect Einsums in PyTorch

To use the PyTorch compiler effectively, we first need to translate indirect Einsum expressions into an equivalent PyTorch program. To facilitate this, we introduce the `Insum` compiler, which converts indirect Einsum strings into standard PyTorch operations. The interface for `Insum` is as follows:

```
Insum(expression: str, **tensors: dict)
```

Here, `expression` is a string representing the indirect Einsum computation, and `tensors` is a dictionary mapping tensor names to their corresponding PyTorch tensors. For example, an indirect Einsum $C_{D_y,x} = A_{y,E_r} \times B_{r,x}$ can be expressed as `Insum("C[D[y],x] += A[y,E[r]] * B[r,x]", C = C, A = A, B = B, D = D, E = E)`.

The `Insum` function works by parsing the provided string expression and constructing an intermediate representation known as an FX graph [92]. In the PyTorch compiler stack, an FX graph is a functional representation of a PyTorch program that clearly encodes tensor operations and their dependencies. Since PyTorch lacks a native primitive for indirect Einsums, `Insum` translates the expression into equivalent PyTorch operations in three main steps:

1. **Gather Inputs**: If the right-hand side involves indirect indexing, gather elements from the required tensors using operations like `torch.index_select` or `torch.gather`.

2. **Execute Einsum**: Perform the Einsum (i.e., `torch.einsum`) computation using the gathered data.

3. **Scatter Outputs**: If the left-hand side involves indirect indexing, scatter the computed results to their target locations using operations like `torch.index_add`.

For the above example, `Insum` will create an FX graph equivalent to the following PyTorch program:

```
# (1) Gather: Atmp[y,r] = A[y,E[r]]
Atmp = torch.index_select(A, dim=1, index=E)
# (2) Einsum: Ctmp[y,x] = Atmp[y,r] * B[r,x]
Ctmp = torch.einsum("yr,rx->yx", Atmp, B)
```

```
# (3) Scatter: C[D[y],x] += Ctmp[y,x]
C.index_add_(dim=0, index=D, source=Ctmp)
```

5.4.2 Native Matrix Multiply Support in TorchInductor

Once we construct an FX graph, we pass it to TorchInductor, the code generation backend of the PyTorch compiler. TorchInductor lowers the FX graph into optimized code by first translating it into an intermediate representation called *InductorIR*, a loop-level IR that describes how operations are executed through nested loops. At this level, TorchInductor performs optimizations such as loop tiling, reordering, and fusion before generating the final Triton code.

Limitation: A core optimization in TorchInductor is the *fusion* of multiple PyTorch operations into a single loop nest. TorchInductor can fuse many pointwise and reduction operations into a single Triton kernel. However, there is a significant exception: matrix multiplication.

Due to its critical role in deep learning workloads, matrix multiplication is not generated natively by TorchInductor. Instead, TorchInductor relies on a hand-optimized Triton template. While this template allows limited fusion of pointwise operations, it prevents complex operations, such as gather and scatter, from being fused. This lack of fusion can result in increased memory traffic, reduced locality, and degraded performance for computations involving matrix multiplication.

When the FX graph for `"C[D[y],x] += A[y,E[r]] * B[r,x]"` is passed to TorchInductor, it generates three separate (i.e., not fused) Triton kernels: one for gathering `A`, one using a matrix multiplication template (`Ctmp[y,x] = Atmp[y,r] * B[r,x]`), and one for scattering to `C`.

To overcome the limitations of fixed templates, we extend TorchInductor's codegen pass to emit the Triton `tl.dot` intrinsic, which maps directly to tensor core operations. This enables the compiler to natively generate tensor-core-optimized kernels without relying on hand-written templates. Implemented directly on TorchInductor's loop-level IR, our approach integrates seamlessly with its fusion infrastructure, allowing fully fused kernels for indirect Einsums.

Forcing TorchInductor to Generate Matmul

While TorchInductor uses a hand-written Triton template for matrix multiplication, it can also be configured to generate matrix multiplication natively. We build on this capability in our compiler extension.

Consider the PyTorch matrix multiplication example (`C = torch.matmul(A,B)`) with matrices of shape $(2048, 2048)$. To prompt TorchInductor to generate matrix multiplication kernels without a template, a useful trick is to rewrite the operation explicitly as replicated multiplication (i.e., batched Cartesian product) followed by summation:

```
# Prod[y,r,x] = A[y,r] * B[r,x]
Prod = A.unsqueeze(2) * B.unsqueeze(0)
# C[y,x] = Prod[y,r,x]
C = Prod.sum(dim=1)
```

When provided with this rewritten computation, TorchInductor generates the Triton kernel depicted in Figure 5.10. However, the resulting kernel suffers from performance limitations due to two critical issues:

- **Lack of Tensor Core Utilization:** TorchInductor generates naive multiplication and summation (as seen in the green-highlighted code) instead of leveraging Triton's optimized `tl.dot` operation for tensor cores.
- **Absence of Proper Tiling:** TorchInductor currently flattens all pointwise ranks (those not involved in the reduction) into a single loop variable. This often helps exploit the GPU's abundant parallelism by enlarging the parallelizable loop. However, it can also

```
@triton.jit
def triton_v1(A, B, C):
  YXBLOCK : tl.constexpr = 32*32
  RBLOCK : tl.constexpr = 32
  yxoffset = tl.program_id(0) * YXBLOCK
  yx = yxoffset +
       tl.arange(0, YXBLOCK)[:,None]
  r_base =
       tl.arange(0, RBLOCK)[None,:]

  y = yx // 2048
  x = yx % 2048

  # (YX,R)
  acc = tl.full([YXBLOCK, RBLOCK], 0.0)
  for r_offset in range(0,2048,RBLOCK):
    r = r_offset + r_base
    # (YX,R)
    A_yr = tl.load(A + 2048*y + r)
    # (YX,R)
    B_rx = tl.load(B + 2048*r + x)
    acc += A_yr * B_rx

  # (YX,R) -> (YX,1)
  acc = tl.sum(acc, 1)[:, None]
  tl.store(C + yx, acc)
```

Figure 5.10: Triton code generated by default TorchInductor (without our compiler extension) for the matrix multiply between two 2048 × 2048 matrices, `C[y,x] = A[y,r] * B[r,x]`. The green-highlighted lines show naive multiplication and summation in place of tensor core operations, and the yellow-highlighted lines show the flattened `yx` loop variable.

negatively impact memory locality. In particular, the output ranks y and x, which are critical for memory access patterns in matrix multiplication, are flattened together. This behavior is evident in the loop-level InductorIR:

```
for yx:
  y, x = yx // 2048, yx % 2048
  for r:
    Prod[y,r,x] = A[y,r] * B[r,x] # multiply
    C[y,x] = Prod[y,r,x] # summation
```

Natively Invoking Tensor Cores with `tl.dot`

To achieve optimal performance on GPUs, the generated code must leverage Triton's intrinsic `tl.dot` and apply efficient two-dimensional tiling over the output matrix. To support this, we introduce a new IR node in InductorIR called `ops.dot`, which explicitly represents matrix multiplication.

Although `ops.dot` has the same semantics as broadcast multiplication followed by summation, it is lowered differently during code generation: it maps directly to `tl.dot`. We implemented a rewrite pass that detects patterns of broadcast multiplication followed by summation, and replaces them with an `ops.dot` node.

When this node is present, Inductor explicitly tiles the computation along the pointwise variables y and x, producing the following tiled IR:

```
for y:
  for x:
    for r:
      # Equivalent to C[y,x] += A[y,r] * B[r,x]
      ops.dot(C[y, x], A[y, r], B[r, x])
```

```python
1 @triton.jit
2 def triton_v2(A, B, C):
3   YBLOCK : tl.constexpr = 32
4   XBLOCK : tl.constexpr = 32
5   RBLOCK : tl.constexpr = 32
6   yoffset = tl.program_id(1) * YBLOCK
7   xoffset = tl.program_id(0) * XBLOCK
8   y = yoffset +
9       tl.arange(0,YBLOCK)[:,None,None]
10  x = xoffset +
11      tl.arange(0,XBLOCK)[None,:,None]
12  r_base =
13      tl.arange(0,RBLOCK)[None,None,:]
14
15  # (Y,X)
16  acc = tl.full([YBLOCK,XBLOCK], 0.0)
17  for r_offset in range(0,2048,RBLOCK):
18    r = r_offset + r_base
19    # (Y,1,R)
20    A_yxr = tl.load(A + 2048*y + r)
21    # (1,X,R)
22    B_yxr = tl.load(B + 2048*r + x)
23    # (Y,1,R) -> (Y,R)
24    A_yr=tl.view(A_yxr,[YBLOCK,RBLOCK])
25    # (1,X,R) -> (X,R)
26    B_xr=tl.view(B_yxr,[XBLOCK,RBLOCK])
27    # (Y,R) x (R,X) -> (Y,X)
28    acc += tl.dot(A_yr,tl.trans(B_xr))
29  acc = acc[:, :, None] #(Y,X)->(Y,X,1)
30  tl.store(C + 2048*y+x, acc)
```

Figure 5.11: Triton code generated with native matmul support for the same matrix multiply as Figure 5.10. The new `ops.dot` IR node enables tensor cores (`tl.dot`, green) and 2D tiling over the output ranks. The yellow-highlighted lines show eager broadcasting, which forces the `tl.view` and `tl.trans` calls before `tl.dot`.

Figure 5.11 shows the generated code using the tiled IR with `ops.dot`. Compared to the naive version in Figure 5.10, there are two notable differences:

1. **Tiled y and x (Lines 8–13):** The computation is now tiled over three explicit axes: y, x, and r, with y and x parallelized. This replaces the earlier flattened `yx` loop variable.

2. **Tensor Core Invocation via `tl.dot` (Lines 23–29):** Instead of manually multiplying and summing over the reduction axis, Inductor now emits a `tl.dot` operation from

`ops.dot`, enabling the use of tensor cores. The `tl.dot` intrinsic expects its operands to have shapes (Y, R) and (R, X), producing an output of shape (Y, X). To meet these shape requirements, the input tensors are first reshaped to (`YBLOCK`, `RBLOCK`) and (`XBLOCK`, `RBLOCK`) using `tl.view`, and the second operand is transposed with `tl.trans`. The accumulator, initialized in line 16, excludes the reduction rank `RBLOCK`, following an output-stationary dataflow that retains the partial sums directly in the output tile.

Lazy Broadcasting

While adding `ops.dot` and tiling the pointwise ranks already allows us to generate a fully fused gather–scatter kernel with tensor cores, we found that an additional optimization, *Lazy Broadcasting*, is necessary to reach peak performance.

By default, when Inductor lowers IR to Triton code, it uses what we refer to as *Eager Broadcasting*: every loop variable is assigned a unique axis in the Triton block ranks up front. For example, in the yellow-highlighted code in Figure 5.11, `YBLOCK`, `XBLOCK`, and `RBLOCK` correspond to y, x, and r, shaped as `[:,None,None]`, `[None,:,None]`, and `[None,None,:]`, respectively. This eager broadcasting is convenient for code generation, as it simplifies indexing logic: each tensor can be accessed easily without needing to reason about broadcasting behavior at load time.

However, a major downside is that `tl.dot` expects operands in a very specific layout: $(Y, R) \times (R, X)$. Eager broadcasting breaks this layout, requiring explicit reshaping and transposing before the `tl.dot` call. We found that this introduces runtime overhead from reshaping and transposing.

To eliminate this overhead, we introduce *Lazy Broadcasting*. Rather than broadcasting every loop variable with a unique axis from the start, we delay broadcasting until it is actually required. This allows us to match the expected layout of `tl.dot` without any reshaping or transposing.

```
1 @triton.jit
2 def triton_v3(A, B, C):
3   YBLOCK : tl.constexpr = 32
4   XBLOCK : tl.constexpr = 32
5   RBLOCK : tl.constexpr = 32
6   yoffset = tl.program_id(1) * YBLOCK
7   xoffset = tl.program_id(0) * XBLOCK
8   y = yoffset +
9       tl.arange(0, YBLOCK)[:,None]
10  x = xoffset +
11      tl.arange(0, XBLOCK)[None,:]
12  r_base =
13      tl.arange(0, RBLOCK)
14
15  # (Y,X)
16  acc = tl.full([YBLOCK, XBLOCK], 0.0)
17  for r_offset in range(0,2048,RBLOCK):
18    r = r_offset + r_base
19    # (Y,R)
20    A_yr = tl.load(A+2048*y+r[None,:])
21    # (R,X)
22    B_rx = tl.load(B+2048*r[:,None]+x)
23    # (Y,R) x (R,X) -> (Y,X)
24    acc += tl.dot(A_yr, B_rx)
25
26  tl.store(C + 2048*y+x, acc)
```

Figure 5.12: Triton code generated with Lazy Broadcasting for the same matrix multiply as Figure 5.10. Delaying layout expansion (yellow) lets the operands arrive at `tl.dot` (green) already in the correct shapes, eliminating the reshaping and transposing of Figure 5.11.

Figure 5.12 shows the generated code after applying Lazy Broadcasting. There are two key differences from Figure 5.11:

1. **Minimal Eager Broadcasting (Lines 8–13):** Loop variables whose broadcasting ranks stay constant are expanded once at the beginning. Only y and x are shaped as `tl.arange(0, YBLOCK)[:, None]` and `tl.arange(0, XBLOCK)[None, :]`, while r remains 1D.

2. **Lazy Broadcasting (Lines 19–22):** We apply broadcasting to r on demand, depending on its usage. For example, when loading `A[y, r]`, we use `r[None, :]` to form shape `(YBLOCK,RBLOCK)`; when loading `B[r, x]`, we use `r[:, None]` to form `(RBLOCK,XBLOCK)`.

```
@triton.jit
def generated_triton(A, B, C):
 yoffset = tl.program_id(1) * YBLOCK
 xoffset = tl.program_id(0) * XBLOCK

 y = yoffset + tl.arange(0, YBLOCK)[:, None] # (Y,1)
 x = xoffset + tl.arange(0, XBLOCK)[None, :] # (1,X)
 r_base = tl.arange(0, RBLOCK) # (R,)

 acc = tl.full([YBLOCK, XBLOCK], 0.0) # (Y,X)
 for r_offset in range(0, 2048, RBLOCK):
  r = r_offset + r_base # (R,)
  E_r = tl.load(E + r) # (R,)
  A_yr = tl.load(A + (E_r[None,:] + 2048*y)) # (Y,R)
  B_rx = tl.load(B + (x + 2048*r[:,None])) # (R,X)
  acc += tl.dot(A_yr, B_rx) # (Y,X)

 D_y = tl.load(D + y) # (Y,1)
 tl.atomic_add(C + (x + 2048 * D_y), acc) #(Y,X)
```

Figure 5.13: Generated Triton code for `C[D[y],x] += A[y,E[r]] * B[r,x]`, showing full fusion of gather, matmul, and scatter using tensor cores. Yellow-highlighted lines indicate where lazy broadcasting occurs.

This ensures that the operands passed to `tl.dot` are already in the correct shape, removing the need for views or transposes.

To implement lazy broadcasting, we maintain block shape metadata for every Triton variable during codegen. At kernel entry, only the output indices (`y` and `x`) are eagerly materialized as a 2D block, while the reduction variable `r_base` is kept as a 1D block. As new variables are generated, their block shape is inferred from the operands they depend on. Comments in Figure 5.13 show the block shape of each variable.

As we track block shapes during code generation, whenever a variable with 1D shape `(RBLOCK,)` interacts with a 2D variable shaped with `YBLOCK` or `XBLOCK`, we apply lazy broadcasting to the 1D variable along the required axis. For instance, when compiling $C_{D_y,x} = A_{y,E_r} \times B_{r,x}$ in Figure 5.13, we defer broadcasting of the 1D variables `r` and `E_r` until they are needed in loading `A[y,E[r]]` or `B[r,x]`. As a result, the inputs to `tl.dot` naturally take the shapes `(YBLOCK,RBLOCK)` and `(RBLOCK,XBLOCK)`, enabling direct use of tensor cores without requiring extra reshaping or transposition.

Chapter 6

Evaluation of the Format-Aware Compiler on Discrete Einsums

In this chapter, we evaluate the GPU kernels generated by the `Insum` compiler and our PyTorch compiler extension developed in this chapter. We focus on the following five research questions:

- **Q1:** How universal is our indirect Einsum model across different sparse applications?
- **Q2:** How does the performance of our generated code compare to hand-written, state-of-the-art kernels?
- **Q3:** How does grouping or blocking in a fixed-length sparse format affect performance?
- **Q4:** How does our new code generation, including native matrix multiplication and lazy broadcasting, affect performance?
- **Q5:** How does our compiler compare to existing sparse GPU compilers?

We evaluate **Q1** and **Q2** by comparing our generated code against hand-optimized kernels across five case studies. We address **Q3** and **Q4** through an ablation study in Section 6.7. Finally, we answer **Q5** by comparing our compiler to existing compilers across various dimensions in Section 6.8.

6.1 Experimental Setup

The `Insum` compiler comprises approximately 500 lines of code, and the corresponding modifications to TorchInductor add about 1,600 lines. Our implementation is based on a patched version of PyTorch (commit `e8304f0`).[1] All experiments are conducted on an RTX 3090 (24 GB, Ampere). All group sizes are selected using the heuristic described in Section 5.3.2.

Case Studies: We evaluate our compiler across five sparse kernels commonly used in deep learning; the first four correspond to the applications summarized in Table 5.1. For each application, we compare our generated code against state-of-the-art implementations, which often involve significant engineering effort and expert-written Triton or CUDA.

- **Structured SpMM:** A sparse matrix–dense matrix multiplication (SpMM), where the sparse matrix exhibits structured sparsity, specifically block sparsity.
- **Unstructured SpMM:** Similar to Structured SpMM, but the sparse matrix features unstructured sparsity.
- **Point Cloud Sparse Convolution:** A sparse convolution operation over 3D point clouds or sparse voxels.
- **Equivariant Tensor Product:** A fully connected tensor product used in equivariant neural networks.
- **Jagged Batch Matrix Multiply:** A batch matrix multiplication between a jagged tensor and a dense matrix, outputting a jagged tensor. This operation is widely used in recommendation systems where sequence lengths vary across the batch.

[1] Our implementation has been upstreamed to PyTorch 2.10 and can be enabled by `torch._inductor.config.triton.native_matmul = True`.

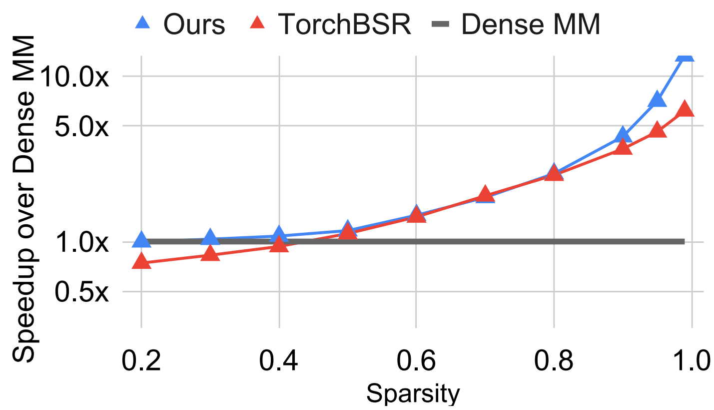


Figure 6.1: Structured SpMM: speedup of our kernel and TorchBSR over dense matmul on FP16, 4096×4096 matrices with 32×32 blocks.

6.2 Case Study: Structured SpMM

In this case study, we compare our compiler-generated block-sparse SpMM kernels with the state-of-the-art TorchBSR kernel [89], a hand-written Triton implementation integrated into PyTorch. TorchBSR requires 202 lines of Triton code, while our approach expresses the same computation in a single-line Einsum. We use the BlockGroupCOO format (Section 5.3) with 32×32 blocks and grouping along block rows. Figure 6.1 shows FP16 performance on 4096×4096 matrices, reported as speedup over dense matmul.

Our approach shifts the crossover point, where sparse outperforms dense, from about 40% to 25% sparsity. Our kernels consistently match or surpass TorchBSR, with a larger advantage at high sparsity. This is mainly because TorchBSR relies on the BCSR format, a block variant of CSR. Like CSR, BCSR incurs an $O(N)$ row-pointer overhead, where N is the number of rows; even empty rows require storage and traversal. In hypersparse matrices, where most rows contain no nonzeros, this overhead becomes dominant [93]. In contrast, BlockGroupCOO is COO-based and stores only nonzero blocks, eliminating per-row overhead and scaling more efficiently in the hypersparse regime.

6.3 Case Study: Unstructured SpMM

In this study, we compare our compiler-generated unstructured SpMM kernel with two state-of-the-art hand-written CUDA kernels: Sputnik [46] and cuSPARSE [45]. While Sputnik implements SpMM in approximately 2,000 lines of code, and cuSPARSE's implementation details are unavailable because the library is proprietary, our approach requires only a single Einsum expression (using the GroupCOO format).

The operation we evaluate is $C_{m,n} = A_{m,k} \times B_{k,n}$, where A is an unstructured sparse matrix. We selected real-world sparse matrices (Table 6.1) from the TC-GNN datasets [94] with various sparsity patterns, including skewed distributions of nonzeros per row and matrices with large numbers of rows and nonzeros. For our experiments, we set the number of columns in C to 128.

Data	**Rows**	**NNZ**	**Avg**	**Med**	**Max**
soc-BlogCatalog	88784	2093195	23.6	6	9429
com-amazon	334863	1851744	5.5	4	549
Yeast	1710902	3636546	2.1	2	6
artist	50515	1638396	32.4	13	1469
PROTEINS_full	43466	162088	3.7	4	25
DD	334925	1686092	5.0	5	19
citeseer	3327	9228	2.8	2	99
OVCAR-8H	1889542	3946402	2.1	2	5
ppi	56944	818716	14.4	8	582
amazon0505	410236	4878874	11.9	10	2760
amazon0601	403394	3387388	8.4	5	2751
cora	2708	10556	3.9	3	168
YeastH	3138114	6487230	2.1	1	6
pubmed	19717	88651	4.5	2	171

Table 6.1: Sparse matrix statistics: number of rows (= columns), number of nonzeros (NNZ), and NNZ distribution per row.

Figure 6.2 compares the performance of different kernels on FP32 inputs, reported as speedup relative to cuSPARSE. Our kernel achieves the highest average performance, running 1.2× faster than cuSPARSE, compared to 1.09× for Sputnik. For unstructured sparsity, however, no single kernel dominates across all test cases, since performance depends heavily

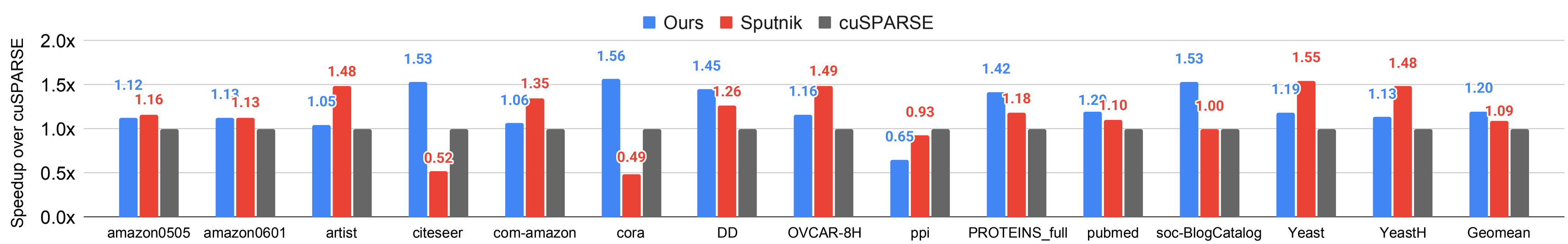


Figure 6.2: Comparison of our compiler-generated unstructured SpMM kernel, Sputnik, and cuSPARSE on various sparse matrices. On average, our compiler-generated kernel outperforms Sputnik by 1.09× and cuSPARSE by 1.20×

on the distribution of nonzeros. Sputnik benefits from its row-permutation strategy that groups rows with similar nonzero counts, which gives it an advantage on datasets with highly skewed distributions (e.g., artist).

We also evaluated FP16 inputs. These results are omitted from the figure because Sputnik provides only limited FP16 support. It can process matrices with fewer than 2^{16} rows, but cannot handle the large matrices in our test suite. In contrast, our kernel runs robustly across all benchmarks and, on average, performs 1.18× better than cuSPARSE in FP16.

Our kernel consistently delivers the best average performance. Importantly, it achieves this with a succinct, compiler-generated implementation, in contrast to Sputnik's complex, hand-optimized code. Furthermore, our approach supports a broader range of problem specifications, including FP16, highlighting the effectiveness of a compiler-based approach.

6.4 Case Study: Point Cloud Sparse Convolution

In this case study, we evaluate sparse convolution on point clouds and compare our generated kernel against TorchSparse [23], a state-of-the-art library. TorchSparse implements sparse convolution using two different algorithms: (1) ImplicitGEMM [95] and (2) Fetch-on-Demand [96], each relying on distinct sparse formats and separate CUDA codebases.

The point cloud sparse convolution can be expressed as:

$$Out_{x,m} = Map_{x,y,z} \times In_{y,c} \times Weight_{z,c,m}$$

Here, c and m denote input and output channels, respectively. In and $Weight$ are dense tensors and Map is a 3D sparse tensor. If we store Map in COO format, with its nonzero indices stored in $MAPX$, $MAPY$, $MAPZ$, and corresponding values in $MAPV$, the sparse convolution becomes:

$$Out_{MAPX_p,m} = MAPV_p \times In_{MAPY_p,c} \times Weight_{MAPZ_p,c,m}$$

We can further optimize this by grouping entries by $MAPZ$, yielding the following Einsum form:

$$Out_{MAPX_{p,q},m} = MAPV_{p,q} \times In_{MAPY_{p,q},c} \times Weight_{MAPZ_{p},c,m}$$

This exposes a batched matrix multiply among Out, In, and $Weight$ (i.e., $pqm \leftarrow pqc \times pcm$), enabling the use of tensor cores.

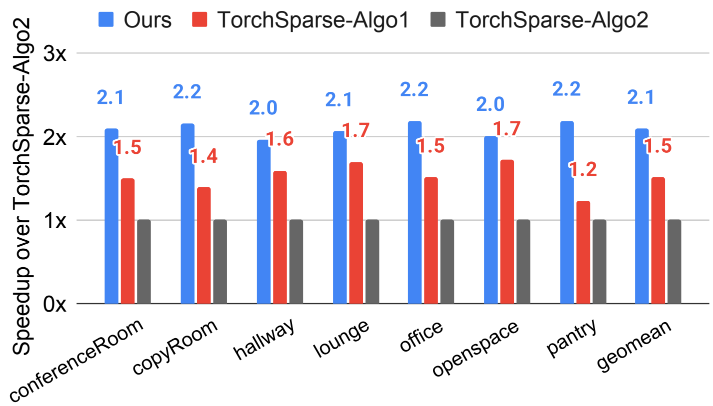
While TorchSparse requires over 4000 lines of CUDA code to implement and optimize these sparse convolution kernels across data types and channel sizes, our approach achieves the same computation using a single Einsum expression.

Figure 6.3 shows performance results for channel size 128 on seven real-world indoor point clouds from the S3DIS dataset (Area 6). We used a voxel size of 5 cm for quantization, as described in prior work [85]. Our method consistently outperforms both TorchSparse algorithms on FP16. In Figure 6.3(b), we evaluate our generated code on an HBM-equipped GPU (NVIDIA H100) and observe consistent performance improvements over hand-written kernels. Our kernel achieves higher speedups on Hopper than on Ampere, primarily because TorchSparse is optimized for the Ampere architecture but not for Hopper, whereas our compiler-generated Triton code can automatically adapt and autotune for Hopper. This highlights the power of the compiler-based approach compared to highly specialized hand-written implementations.

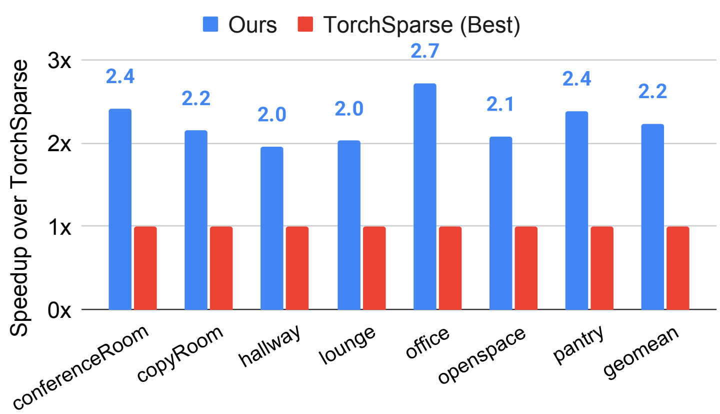
6.5 Case Study: Equivariant Tensor Product

In this case study, we implement a fully connected tensor product in an equivariant neural network [90] using our compiler. The fully connected tensor product, also referred to as the `uvw` mode, can be written as:

$$Z_{b,i,w} = CG_{i,j,k,l} \times X_{b,j,u} \times Y_{b,k} \times W_{b,l,u,w}$$



(a) Speedup of our method vs. TorchSparse-Algo1 and TorchSparse-Algo2 for FP16 inputs on RTX3090 (Ampere). On average, our compiler-generated kernel outperforms TorchSparse-Algo1 by 2.1× and TorchSparse-Algo2 by 1.5×



(b) Speedup of our method vs. the best of TorchSparse-Algo1 and Algo2 for FP16 inputs on H100 (Hopper). On average, our compiler-generated kernel outperforms TorchSparse by 2.2×

Figure 6.3: Comparison of our compiler-generated sparse convolution kernels with TorchSparse on two different GPUs. TorchSparse implements sparse convolution in two variants: TorchSparse-Algo1 (referred to as ImplicitGEMM) and TorchSparse-Algo2 (referred to as Fetch-on-Demand).

$\ell_{\max}$	**Channels**	**Ours**	**cuequivariance**	**e3nn**
1	16	8.3×	2.6×	1.0×
	32	4.2×	1.5×	1.0×
	64	2.3×	0.9×	1.0×
2	16	5.2×	1.1×	1.0×
	32	5.4×	1.1×	1.0×
	64	3.3×	0.5×	1.0×
3	16	2.6×	0.5×	1.0×
	32	3.6×	0.6×	1.0×
	64	2.5×	0.3×	1.0×

Table 6.2: Speedup of our method vs. cuequivariance and e3nn. Speedups are reported relative to e3nn (normalized performance). All experiments are conducted in FP32.

Here, CG is a sparse 4D tensor, while all other tensors (X, Y, W, Z) are dense tensors. The coordinates selected by the rank variables i, j, k, l in CG determine the interaction pattern among the input tensors, while u and w denote input/output channels and b denotes the batch rank. We store CG in COO format using coordinate arrays CGI, CGJ, CGK, CGL and corresponding nonzero values CGV, which allows us to rewrite the computation as:

$$Z_{b,CGI_p,w} = CGV_p \times X_{b,CGJ_p,u} \times Y_{b,CGK_p} \times W_{b,CGL_p,u,w}$$

To further optimize, we group entries by CGL, leading to:

$$Z_{b,CGI_{p,q},w} = CGV_{p,q} \times X_{b,CGJ_{p,q},u} \times Y_{b,CGK_{p,q}} \times W_{b,CGL_p,u,w}$$

This exposes a batched matrix multiply pattern among Z, X, and W of the form $(bp)qw \leftarrow (bp)qu \times (bp)uw$. Here, the batch rank variable is bp and the reduction rank variable is u, allowing our compiler to apply tensor core acceleration.

We compare our GroupCOO Einsum formulation against two hand-written libraries: `cuequivariance` [97] and `e3nn` [90]. All experiments are conducted with a batch size of 10,000. To control the sparsity pattern of the CG tensor, we use real Clebsch–Gordan (CG) coefficients corresponding to different values of a hyperparameter $\ell_{\max}$.

As shown in Table 6.2, while `cuequivariance` occasionally outperforms `e3nn`, our method consistently achieves higher speedups across all tested channel sizes and $\ell_{\max}$ values, reaching up to 8.3× and remaining above 2× in every setting.

6.6 Case Study: Jagged Batch Matrix Multiply

The final case study evaluates a batch matrix multiplication (BMM) between a jagged tensor and a dense tensor. This operation is commonly used in recommendation systems [98], where models depend heavily on categorical features such as user histories or item sets. These features often differ in length from one sample to another; for example, one user may have clicked on 5 items, while another clicked on 50. This leads to *jagged tensors*, where the size of one rank (typically a sequence length) varies across the batch.

Jagged BMM can be written as the following Einsum:

$$C_{b,m,n} = A_{b,m,k} \times B_{b,k,n}$$

Formally, a jagged tensor can be thought of as a tensor where a given rank (e.g., sequence length) is not uniform across batches. In this case study, the input tensor $A_{b,m,k}$ is jagged along the m rank variable for each batch coordinate b, meaning that for each b, the number of entries along the m (e.g., sequence entries) can vary. Note, however, that the k is dense, meaning that for each jagged entry (b, m), there exists a fully dense k-dimensional feature vector. The output tensor $C_{b,m,n}$ shares the same jagged pattern on b and m, while the second input $B_{b,k,n}$ remains dense.

To handle the jaggedness in A and C, we treat jagged tensors as sparse and store them in a COO-like format. Specifically, we represent a jagged tensor A using coordinate arrays AB and AM, which store the batch coordinate b and the jagged coordinate m of each nonzero, respectively. The corresponding feature vectors are stored in a value tensor AV of shape `#NNZ` $\times$ `K`. This reformulates the Einsum as:

$$CV_{p,n} = AV_{p,k} \times B_{AB_p,k,n}$$

To expose more structure and parallelism, we group the COO representation by AB (i.e., batch coordinate), resulting in:

$$CV_{p,q,n} = AV_{p,q,k} \times B_{AB_p,k,n}$$

This reveals a batch matrix multiplication pattern of the form $pqn \leftarrow pqk \times pkn$, enabling our compiler to generate optimized tensor core operations.

B \ M	**64**	**128**	**256**	**512**
100	1.62×	1.80×	1.70×	1.82×
1000	1.52×	1.81×	1.65×	1.37×
10000	1.52×	1.44×	1.47×	1.98×

Table 6.3: Normalized speedup of our compiler-generated jagged BMM relative to FBGEMM for varying batch sizes (`B`) and jagged ranks (`M`) of tensor $A_{b,m,k}$.

We compare our GroupCOO (group size 32) jagged BMM with FBGEMM's implementation [99], which is hand-written in approximately 100 lines of Triton code. We vary the batch size (`B`) and jagged rank (`M`) of the jagged tensor $A_{b,m,k}$, while fixing the feature rank variables `K` and `N` to 128. To simulate realistic variation, we randomly assign sequence lengths in the jagged rank variable m for each batch, resulting in approximately 50% overall sparsity. As shown in Table 6.3, our compiler-generated jagged BMM consistently outperforms the hand-optimized Triton baseline across all settings in FP32. This case study demonstrates that our compiler and the indirect Einsum abstraction generalize effectively to non-uniform, real-world tensor structures like jagged inputs.

6.7 Ablation Study

Figure 6.4 shows an ablation study on structured SpMM to evaluate the impact of format selection and compiler optimizations. All matrices are of size 4096×4096, with the sparse matrix having 90% uniform sparsity over 32×32 dense blocks.

Impact of Format: We store the sparse matrix A using three different formats, each explained in Section 5.3:

- **COO:** A is stored in coordinate list (COO) format, where the row and column indices of nonzeros are stored as `AM` and `AK`. The indirect Einsum is: $C_{AM_p,n} = AV_p \times B_{AK_p,n}$.

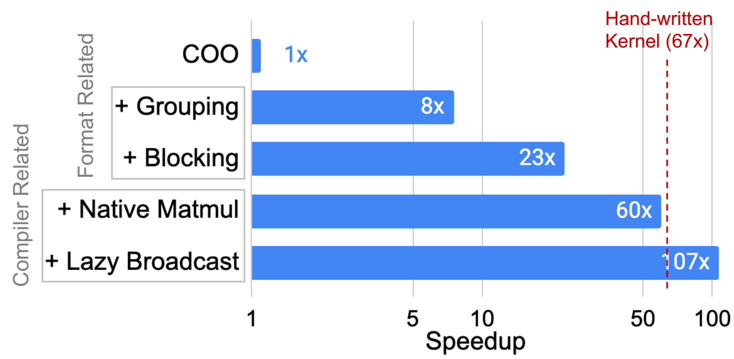


Figure 6.4: Ablation study on structured SpMM, showing how each optimization contributes to speedup over the baseline (COO without fusion). When all format and compiler optimizations are enabled, the performance surpasses that of the hand-written kernel (TorchBSR).

- **COO + Group:** We group the row indices `AM` in groups of 16. The indirect Einsum is: $C_{AM_{p},n} = AV_{p,q} \times B_{AK_{p,q},n}$.
- **COO + Group + Block:** Once we block the matrix with 32×32 dense blocks, we then group the block rows `AM` (by 4). The Einsum is: $C_{AM_{p},bm,n} = AV_{p,q,bm,bk} \times B_{AK_{p,q},bk,n}$.

In the top three rows of Figure 6.4, we compare these formats without our compiler optimizations. In this setting, the PyTorch compiler separately launches gather, matrix multiplication, and scatter operations.

Grouping alone provides a substantial speedup, 8× over the baseline. This comes partly from reduced memory consumption: GroupCOO uses only 69% of the memory required by COO. More importantly, grouping improves data reuse and reduces the number of scatter operations.

```
# COO SpMM
parallel-for p in [0, NNZ):
  for n in [0, N):
    C[AM[p], n] += AV[p] * B[AK[p], n] # NNZ * N atomics

# GroupCOO SpMM (group size = g)
parallel-for p in [0, NNZ / g):
```

```
for n in [0, N):
  acc = 0 # accumulate in register
  for q in [0, g): # nonzeros in a group
    acc += AV[p, q] * B[AK[p, q], n]
  C[AM[p], n] += acc # NNZ / g * N atomics
```

COO SpMM indirectly accesses `B` and `C` for every nonzero loop `p`, issuing an atomic update per element of `C`. In contrast, GroupCOO reuses the same row of `C` across all g nonzeros in a group, accumulating their contributions in registers before issuing an atomic update. This improves locality for both `C` and `B`, and reduces the number of atomic calls by a factor of g, which explains much of the observed speedup.

Finally, we observe that combining grouping and blocking delivers even greater performance, up to 20× over the baseline. The blocked formats achieve this by enabling tensor cores, while grouping improves data reuse.

Impact of Compiler Optimizations: In the bottom two rows of Figure 6.4, we apply our compiler optimizations on top of the COO + Group + Block format. Enabling native matrix multiplication with tensor cores delivers a 2.6× speedup over the default PyTorch compiler by fusing gather, matmul, and scatter into a single Triton kernel. This eliminates the need to materialize large intermediate tensors (which exceed 1.5 GB when fusion is disabled) and avoids costly reloads from global memory. With fusion, the gathered data remains in shared memory and is fed directly into the tensor cores without unnecessary trips to global memory. Finally, Lazy Broadcasting further improves memory efficiency by eliminating the transposing and reshaping overhead in the Triton kernel, increasing the instruction-level parallelism feeding into tensor cores.

Metric	**Insum**	**TACO** [18]	**SparseTIR** [48]
Compile (s)	9.9	**0.01**	0.32
Autotune (s)	**4.9**	N/A (10 LoC)	N/A (860 LoC)
FormatConvert (ms)	0.55	**0.47**	13.47
Runtime (ms)	**0.47**	253.53	1.05

Table 6.4: Performance, compilation cost, and conversion cost for point cloud convolution on the `conferenceRoom` input with FP16 and channel size 128. TACO and SparseTIR lack autotuners, requiring users to manually search for the best schedule, which often demands substantial time and code: 10 lines of schedule for TACO and 860 lines for SparseTIR, respectively.

6.8 Comparison Against Other Sparse Compilers

While our compiler generates kernels that outperform hand-written kernels, two sources of overhead must be considered before kernel launch: (1) compilation and (2) format conversion. Focusing on these aspects, we compare `Insum` with other sparse GPU compilers, TACO [18] and SparseTIR [48], using the point cloud convolution example (Table 6.4).

Compilation overhead typically consists of two parts: scheduling (tuning) to select tile sizes and the format to store, followed by code generation and compilation. In deep learning workloads, this cost is easily amortized since compiled operators are cached and reused many times during inference. The difficulty arises because these compilers lack an auto-scheduler, forcing developers to manually craft schedules. Using TACO, we spent hours identifying a schedule that still underutilized the GPU (no shared memory or tensor core usage), and SparseTIR required adopting a schedule of 860 lines provided by its authors. In contrast, our compiler requires only a single indirect Einsum. We integrate the PyTorch compiler's autotuning infrastructure to automatically discover efficient Triton configurations for our fused kernel, eliminating the need for manual schedule engineering. Although our compile and autotuning time is longer (14.8 seconds), the generated code is both the fastest and the only compiler-based approach to surpass the hand-written implementation (0.66 ms, TorchSparse). This process is fully automated, and its cost is easily amortized over repeated execution.

For the majority of applications, the format conversion overhead can also be amortized [48,71]. Many sparse workloads are static in structure, such as fixed graphs in GNNs [94] and pruned weight matrices [85], allowing conversion to be done once offline and reused throughout inference. Even in dynamic scenarios such as point cloud convolution, the format conversion is done once and reused across multiple layers [23]. Our framework shows moderate conversion cost (0.55 ms). The conversion kernel of SparseTIR is implemented on the CPU, resulting in significantly higher overhead (13.47 ms). TACO achieves the fastest conversion (0.47 ms), but its advantage is offset by prohibitively slow runtime performance. Thus, our approach strikes a balance: reasonable conversion time combined with the best kernel runtime.

6.9 Summary

The five case studies confirm the claims of this chapter: a single indirect Einsum, compiled by Insum, matches or outperforms hand-written kernels that each required hundreds to thousands of lines of expert CUDA or Triton (Q1, Q2). The ablation study shows that both halves of the approach are necessary, with fixed-length formats supplying data reuse and tensor-core-friendly structure, and the compiler extensions supplying fusion and efficient code generation (Q3, Q4). Compared to existing sparse compilers, Insum is the only one whose generated code surpasses hand-written kernels, at a compilation cost that is amortized in practice (Q5). These results establish the format-aware reformulation in the discrete, single-sparse setting; the next chapter extends it to multiple sparse operands and to the continuous tensors of Chapter 3.

Chapter 7

Compiling Continuous Einsums with Format-Aware Computation

Two previous chapters (Chapter 5 and 6) developed the format-aware reformulation of sparse computation for Einsums with a single sparse operand. In that setting, the format-agnostic Einsum converts directly into gathers and scatters over the sparse operand's positions, the leader-follower intersection strategy in TACO's terminology: the sparse operand leads the iteration, and every dense operand is simply read or written at the leader's coordinates. This chapter extends the format-aware approach to Einsums with arbitrarily many sparse operands. The extension is non-trivial for two reasons. First, with several sparse operands no single operand can lead: the effectual multiplications are exactly those between nonzeros whose coordinates intersect, and this intersection between the operands' coordinate sets must itself be computed. Second, the output generally becomes sparse, and its nonzero structure is not known before the computation runs, so results cannot simply be scattered into preallocated uncompressed arrays.

The chapter proceeds in three stages. First, it supports arbitrarily many sparse operands over a discrete iteration space, writing into an uncompressed output. Second, it supports outputs in sparse formats. Third, it generalizes both to piecewise-constant tensors and continuous iteration space, where pinpoint coordinates become intervals and coordinate equality becomes interval intersection.

7.1 Format-Aware Computation on Multiple Sparse Operands

Two running examples are used throughout this section: a two-dimensional sparse elementwise multiplication and a sparse matrix–sparse vector multiplication,

$$\text{Example 1:} \quad C_{i,j} = A_{i,j} \times B_{i,j}, \qquad\qquad \text{Example 2:} \quad C_i = A_{i,k} \times B_k,$$

where A, B, and C are all sparse. The evaluation strategy of format-aware computation has three steps. *Mask creation* identifies which combinations of nonzeros can interact and therefore create effectual computations. *Product* generates every candidate output contribution permitted by the mask, together with its product coordinates, duplicates included. *Merge* reduces the candidates into the output according to their coordinates. This is a generalization of the expansion-sort-compress strategy [93] for sparse–sparse matrix multiply: expansion corresponds to mask creation and product, and sort-compress is one way to realize the reduction step. In this section the output is uncompressed, so the merge is a scatter-add; the sparse-output section that follows generalizes the merge into the coalesce operator.

7.1.1 Tensors and Formats

Throughout the chapter, every sparse operand is stored in COO format. In the fibertree abstraction, COO flattens all ranks of the tensor into one rank and stores that flattened rank in a compressed level: a tensor of R ranks is held in $R + 1$ flat arrays, one coordinate array per rank and one value array. COO is among the most widely used sparse formats, and much real-world data arrives in it, including point clouds and the matrices of the SuiteSparse collection. We assume the inputs are deduplicated, holding at most one entry per tensor point; Section 7.1.3 notes where this assumption matters.

In this section the output is stored uncompressed as an ordinary array whose extents are given and in which explicit zeros occupy storage. The two example instances, as PyTorch arrays:

```
# Example 1: C_ij = A_ij * B_ij
# A: (0,1):2 (1,0):3 (2,2):4
# B: (0,1):10 (1,2):30 (2,2):20
A0, A1, AV = tensor([0, 1, 2]), tensor([1, 0, 2]), tensor([2., 3., 4.])
B0, B1, BV = tensor([0, 1, 2]), tensor([1, 2, 2]), tensor([10., 30., 20.])

# Example 2: C_i = sum_k A_ik * B_k
# A: (0,1):2 (0,3):3 (2,3):4
# B: (1):10 (3):20
A0, A1, AV = tensor([0, 0, 2]), tensor([1, 3, 3]), tensor([2., 3., 4.])
B0, BV = tensor([1, 3]), tensor([10., 20.])
```

Figure 7.1 shows the overview of how we compute format-aware computation on example 2: $C_i = A_{i,k} \times B_k$. The left side shows the instance in dense form. Only three products are effectual: $2 \cdot 10$ and $3 \cdot 20$ at row $i = 0$, and $4 \cdot 20$ at row $i = 2$. The right side shows how format-aware SpMSpV computes the same result from the COO arrays. Here a and b are rank variables that point to positions of the coordinate array in COO. Mask creation (a) compares $A1_a$ with $B0_b$ over all nine position pairs and accepts the three where the k coordinates agree. The nonzero b_1 matches nothing because no nonzero of A has $k = 2$. The product step (b) multiplies the values of the accepted pairs and reads their product coordinates from $A0$. The merge step (c) scatter-adds each product into the output at its coordinate.

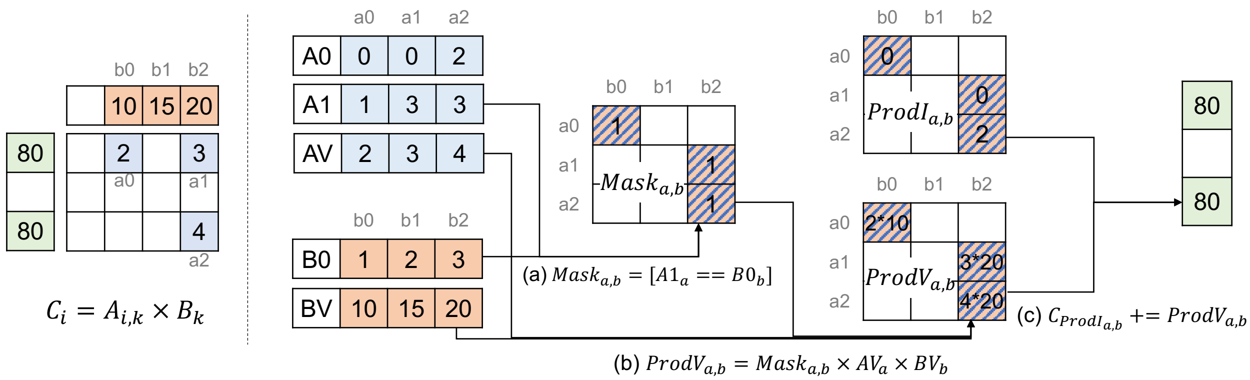


Figure 7.1: Format-aware computation of $C_i = A_{i,k} \times B_k$ with sparse A and B in COO format. Left: the instance in dense form. Right: the same computation on the COO arrays, indexed by nonzero positions a and b. (a) The mask accepts the three position pairs whose k coordinates agree. (b) The product multiplies the values of the accepted pairs and reads their product coordinates from $A0$. (c) The merge scatter-adds the products into the output, giving $C = [80, 0, 80]$.

7.1.2 Mask Creation

The mask is a 0/1 binary tensor over the nonzero positions of the sparse operands, with one rank per operand; its shape is the tuple of nonzero counts, $|A| \times |B|$ for the examples. Entry (a, b) tells whether nonzero a of A and nonzero b of B actually interact. Which coordinates must match (i.e., How to construct the mask) is decided by the Einsum expression: every rank variable shared between two sparse operands contributes one equality condition between the corresponding coordinate arrays, and the mask is the conjunction of all such conditions,

Example 1: $\text{Mask}_{a,b} = [\,A0_a = B0_b\,]$[1]$\times[\,A1_a = B1_b\,]$, Example 2: $\text{Mask}_{a,b} = [\,A1_a = B0_b\,]$.

In Example 1 both i and j are shared, so both coordinate pairs must agree; in Example 2 only the reduced k is shared, carried by A's second rank and B's only rank. Each factor is a single broadcast comparison:

```
# Mask (Example 1): i and j both shared, two equality factors
M = (A0[:,None] == B0[None,:]) & (A1[:,None] == B1[None,:])
```

[1] $[x] = 1$ if x is True, otherwise 0

```
# [[1,0,0], [0,0,0], [0,0,1]]

# Mask (Example 2): only k shared
M = A1[:,None] == B0[None,:]
# [[1,0], [0,1], [0,1]]
```

7.1.3 Product

The product step forms one candidate for every pair (a, b) of COO positions. The mask determines whether that pair contributes to the output, and the candidate value is

$$\mathrm{ProdV}_{a,b} = \mathrm{Mask}_{a,b} \times AV_a \times BV_b.$$

We call this value ProdV. Each candidate also records the coordinates at which it will be merged, which we call its product coordinates ProdI and ProdJ.

$$\mathrm{ProdI}_{a,b} = \mathrm{Mask}_{a,b} \times \mathrm{intersect}(A0_a,\ B0_b), \qquad \mathrm{ProdJ}_{a,b} = \mathrm{Mask}_{a,b} \times \mathrm{intersect}(A1_a,\ B1_b) \qquad \text{(Example 1)},$$

In Example 1, the mask accepts (a, b) only when $A0_a = B0_b$ and $A1_a = B1_b$. An accepted pair therefore has one i coordinate and one j coordinate, which we read from A. In Example 2, the output has only the i coordinate, which is also read from A. The mask multiplies these coordinates as well as the candidate value:

$$\begin{aligned}
&\text{Example 1:} \quad \mathrm{ProdI}_{a,b} = \mathrm{Mask}_{a,b} \times A0_a, \quad \mathrm{ProdJ}_{a,b} = \mathrm{Mask}_{a,b} \times A1_a, \\
&\text{Example 2:} \quad \mathrm{ProdI}_{a,b} = \mathrm{Mask}_{a,b} \times A0_a.
\end{aligned}$$

For an accepted pair, $\mathrm{Mask}_{a,b} = 1$, so its product coordinates are unchanged. For a rejected pair, $\mathrm{Mask}_{a,b} = 0$, so both $\mathrm{ProdV}_{a,b}$ and its product coordinates become zero. Thus, although the rejected candidate is later reduced at coordinate zero, its contribution to the reduction is ineffectual because the mask has set its value to zero.

When do duplicate product coordinates appear? If the Einsum has no reduced rank variable, as in Example 1, they do not: each input coordinate appears at most once (Section 7.1.1), so each output coordinate can also be produced at most once. A reduced variable changes this. In Example 2, the sum over k means several candidates can land on the same output coordinate, one for each matching k. The tuples (a_0, b_0) and (a_1, b_1) both produce a candidate at $i = 0$: the first through $k = 1$ and the second through $k = 3$. These duplicates are the terms of the reduction , waiting to be added in the next step.

```
# Product (Example 1): no reduced variable
ProdV = M * AV[:,None] * BV[None,:]
ProdI = M * A0[:,None].expand(-1, len(BV))
ProdJ = M * A1[:,None].expand(-1, len(BV))
# ProdV = [[20,0,0], [0,0,0], [0,0,80]]

# Product (Example 2): k reduced
ProdV = M * AV[:,None] * BV[None,:]
ProdI = M * A0[:,None].expand(-1, len(BV))
# ProdV = [[20,0], [0,60], [0,80]] two candidates at i = 0
```

7.1.4 Merge

The merge step adds up the candidates that share the same output coordinates. Formally, the output is

$$C_{i,j} \;=\; \sum_{a,b} [\,\mathrm{ProdI}_{a,b} = i\,] \times [\,\mathrm{ProdJ}_{a,b} = j\,] \times \mathrm{ProdV}_{a,b}.$$

Masked-out tuples have $\mathrm{ProdV}_{a,b} = 0$ and add nothing. Because the output is an uncompressed array, this sum can be written directly as a scatter-adding indirect assignment,

$$\text{Example 1:}\quad C_{\mathrm{ProdI}_{a,b},\,\mathrm{ProdJ}_{a,b}} \mathrel{+}= \mathrm{ProdV}_{a,b}, \qquad\qquad \text{Example 2:}\quad C_{\mathrm{ProdI}_{a,b}} \mathrel{+}= \mathrm{ProdV}_{a,b},$$

which is an indirect Einsum in the sense of the previous chapter and compiles to a single scatter-add over the flattened tuples. In Example 2 the two candidates at $i = 0$ have values 20 and 60, and the scatter-add produces $C_0 = 80$.

```
# Merge (Example 1): scatter-add into the 3x3 array output
C = torch.zeros(3, 3)
C.index_put_((ProdI.ravel(), ProdJ.ravel()), ProdV.ravel(),
             accumulate=True)
# C = [[0,20,0], [0,0,0], [0,0,80]]

# Merge (Example 2): scatter-add into the length-3 array output
C = torch.zeros(3)
C.index_put_((ProdI.ravel(),), ProdV.ravel(), accumulate=True)
# C = [80, 0, 80]
```

This works only because the output is uncompressed: any coordinate can be addressed directly, and the zeros simply remain in the array. The cost is that every zero occupies storage. The next section removes this limitation by producing the output directly in a sparse format.

7.2 Supporting Sparse Masks and Sparse Outputs

The previous section evaluates a format-aware computation in three steps: mask creation, product, and merge. Every tensor in those steps has one rank per sparse operand, so its size is the product of the operands' nonzero counts, $|A| \times |B|$ in the examples, and the computation costs $\Theta(|A| \cdot |B|)$ in time and memory even when few nonzeros actually interact. This section removes that cost by exploiting two tensors that the previous section left uncompressed but that are in fact sparse and can be compressed: the mask, which speeds up the product step, and the output, which turns the merge step into the coalesce operator.

7.2.1 Fast Product with a Sparse Mask

The mask is itself a sparse tensor, and usually a very sparse one: entry (a, b) is 1 only when the coordinates of nonzero a and nonzero b match, and most pairs do not match. In Example 1, each nonzero of A matches at most one nonzero of B, so the mask holds at most $\min(|A|, |B|)$ ones out of its $|A| \cdot |B|$ entries. The exception is an Einsum whose sparse operands share no rank variable, such as the Cartesian product $D_{i,j,k} = A_i \times B_j \times C_k$: there every combination of nonzeros is effectual, the mask is all ones, and the $|A| \cdot |B| \cdot |C|$ cost is inherent to the problem rather than an artifact of the method. Because the mask is $0/1$, its COO representation needs no value array; it is nothing but its coordinate arrays, a list of position pairs $(M0_t, M1_t)$ for $t = 1, \ldots, |M|$, where $|M|$ is the number of successful intersections.

Storing the mask sparsely does more than save memory: it turns the rest of the computation into a single-sparse-operand problem, the case the previous chapter already solved. Consider the product step again,

$$\mathrm{ProdV}_{a,b} = \mathrm{Mask}_{a,b} \times AV_a \times BV_b.$$

Over the position dimensions a and b, the value tensors AV and BV are dense; the mask is the only sparse operand. The leader-follower strategy of the previous chapter therefore applies directly, with the mask as the leader: iterate over the mask's nonzeros and gather every other array at their positions,

$$\mathrm{ProdV}_t = AV_{M0_t} \times BV_{M1_t}, \qquad \mathrm{ProdI}_t = A0_{M0_t}, \qquad \mathrm{ProdJ}_t = A1_{M0_t},$$

$$\mathrm{C}_{\mathrm{ProdI_t},\mathrm{ProdJ_t}} \mathrel{+}= \mathrm{ProdV}_t$$

Every downstream array now has length $|M|$ instead of $|A| \cdot |B|$, and the merge is the same scatter-add as before, over $|M|$ tuples. Turning the mask into a sparse format has expressed the product step as an indirect Einsum: the gathers through $M0$ and $M1$ are exactly the indirect accesses of the previous chapter.

```
# Sparse mask (Example 1)
M = (A0[:,None] == B0[None,:]) & (A1[:,None] == B1[None,:])
M0, M1 = M.nonzero(as_tuple=True) # Turning Mask to COO
ProdV = AV[M0] * BV[M1] # gather at the mask positions
ProdI, ProdJ = A0[M0], A1[M0]
C = torch.zeros(3, 3)
C.index_put_((ProdI, ProdJ), ProdV, accumulate=True)
# |M| = 2 C = [[0,20,0], [0,0,0], [0,0,80]]

# Sparse mask (Example 2)
M = A1[:,None] == B0[None,:]
M0, M1 = M.nonzero(as_tuple=True)
ProdV = AV[M0] * BV[M1]
ProdI = A0[M0]
C = torch.zeros(3)
C.index_put_((ProdI,), ProdV, accumulate=True)
# |M| = 3 C = [80, 0, 80]
```

Fast Intersection for Mask Construction

Storing the mask in COO form turns the product step into an indirect Einsum and cuts its cost from $|A| \cdot |B|$ to $|M|$. One bottleneck remains: the listings above still build the dense mask and extract its nonzeros through the calls to `nonzero`, so mask creation itself still costs $|A| \cdot |B|$. What is needed is a way to construct $(M0, M1)$ directly from the coordinate arrays, without materializing the mask in an array.

Constructing the sparse mask is a join: it finds the pairs of entries whose coordinates satisfy the equalities extracted from the Einsum. Joins have a long history in computer science and have been studied in fields like databases and algorithms. No single strategy wins everywhere; the best choice depends on the architecture, on the rank-variable expression of the Einsum, and on the data distribution. We therefore implement three strategies:

- **Dense mask creation.** The naive all-pair comparison of the previous section. Its asymptotic cost is quadratic, but the GPU is abundantly parallel, and in practice the dense mask is often the fastest choice when the operands are small.
- **Binary search.** Mask creation reformulates into a series of binary searches with `torch.searchsorted`: sort one operand's coordinates, linearizing them into a single key when the Einsum matches on several, and search the other operand's coordinates into them. This reduces the cost from quadratic to log-linear.
- **Database join.** Finding the tuples of entries that satisfy a relation is precisely a database join, so we port the entire mask creation step onto a GPU database library.

The binary-search strategy on the running examples replaces the two mask-construction lines of the listing above; everything downstream, the gathers, the product coordinates, and the merge, is unchanged. When the Einsum matches on several coordinates, as in Example 1, the search runs on one coordinate and the remaining equalities are applied afterwards as a filter, through indirect accesses.

```
# Sparse mask via binary search (Example 1): search on i, filter on j
ord_B = torch.argsort(B0)
lo = torch.searchsorted(B0[ord_B], A0)
hi = torch.searchsorted(B0[ord_B], A0, side='right')
cnt = hi - lo # i-match candidates per A entry
Ma = torch.repeat_interleave(torch.arange(len(A0)), cnt)
off = torch.arange(int(cnt.sum())) - (torch.cumsum(cnt,0) - cnt)[Ma]
Mb = ord_B[lo[Ma] + off]
keep = A1[Ma] == B1[Mb] # filter on j by indirect access
M0, M1 = Ma[keep], Mb[keep]
```

```
# pairs (0,0) and (2,2), as before

# Sparse mask via binary search (Example 2): k is the only key
ord_B = torch.argsort(B0)
lo = torch.searchsorted(B0[ord_B], A1)
hi = torch.searchsorted(B0[ord_B], A1, side='right')
hit = hi > lo
M0 = torch.arange(len(A1))[hit]
M1 = ord_B[lo[hit]]
# pairs (0,0), (1,1), (2,1), as before
```

In both cases the dense $|A| \cdot |B|$ mask never exists. New cost becomes the sort, binary searches, and the candidates the filters generate. We evaluate all three strategies in the experiment section and choose the best one per workload.

7.2.2 Supporting Sparse Outputs

We now let the output be stored in COO format as well. With the uncompressed output of Section 7.1, the merge is a scatter-add,

$$C_{\mathrm{ProdI}_t,\,\mathrm{ProdJ}_t} \mathrel{+}= \mathrm{ProdV}_t,$$

whose time cost is only the number of successful intersections $|M|$; its space cost, however, is the full shape of C, since the uncompressed array stores every zero. This is problematic for two reasons. First, the full shape can far exceed the available memory. Second, continuous tensors have real-valued coordinates, so for them an uncompressed array does not even exist; supporting sparse outputs is a prerequisite for supporting continuous Einsums.

The idea is simple. After the product step, every candidate already carries its product coordinates and its value. Building the COO output therefore only requires merging the candidates that share the same coordinates and summing their values. The merge has two parts. First, `torch.unique` removes duplicate coordinates and records where each original coordinate appears:

$$(C0, \mathrm{inv}) = \texttt{torch.unique}(\mathrm{ProdI}).$$

For example,

$$\mathrm{ProdI} = [2, 5, 2] \implies C0 = [2, 5], \qquad \mathrm{inv} = [0, 1, 0].$$

Here, $\mathrm{inv} = [0, 1, 0]$ means that the three candidates contribute to unique entries 0, 1, and 0, respectively. Their values are therefore accumulated as

$$CV_{\mathrm{inv}_t} \mathrel{+}= \mathrm{ProdV}_t.$$

Thus, `torch.unique` constructs the sparse output coordinates, while the existing indirect Einsum performs the accumulation using inverse.

```
# Sparse-output merge (Example 2): unique, then scatter-add
C0, inv = torch.unique(ProdI, return_inverse=True)
CV = torch.zeros(len(C0)).index_add_(0, inv, ProdV)
# C0 = [0, 2] CV = [20, 80]
```

The generalization to N dimensions changes only the unique: it runs over coordinate tuples rather than scalars,

$$(C0_o,\, C1_o) = \text{the unique pairs of } (\mathrm{ProdI}_t,\, \mathrm{ProdJ}_t), \qquad (C0_{\mathrm{inv}_t},\, C1_{\mathrm{inv}_t}) = (\mathrm{ProdI}_t,\, \mathrm{ProdJ}_t),$$

while the indirect Einsum is literally unchanged, $CV_{\mathrm{inv}_t} \mathrel{+}= \mathrm{ProdV}_t$.

```
# Sparse-output merge (Example 1): unique over coordinate pairs
coords = torch.stack([ProdI, ProdJ])
C01, inv = torch.unique(coords, dim=1, return_inverse=True)
```

```
C0, C1 = C01
CV = torch.zeros(C01.shape[1]).index_add_(0, inv, ProdV) # CV[inv] += ProdV
# C0 = [0, 2] C1 = [1, 2] CV = [20, 80]
```

In Example 1 there is no reduced variable, so the candidates already have distinct coordinates and the unique merges nothing. But we showed this listing to show the N-D generalized version of the merge.

7.3 Extending to Continuous Einsums

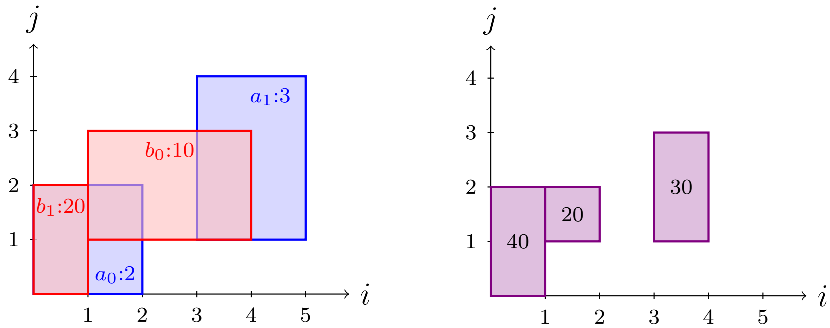
We now extend the mask-product-merge strategy to continuous Einsums. The core change from the discrete setting is the shape of a nonzero: a piece of a piecewise-constant tensor covers a half-open interval $[s, e)$ in each rank, not a single point. The storage stays in COO form, one entry per piece. In the structure-of-arrays layout used here, each rank has separate coordinate arrays for interval starts and ends.

The two running examples keep their equations, but every tensor is now piecewise constant and every rank is an interval. In Example 1 the pieces are AAHRs in the (i, j) plane. In Example 2 the reduction over k ranges over a continuum, so the sum becomes an integral,

$$\text{Example 1:} \quad C_{i,j} = A_{i,j} \times B_{i,j}, \qquad\qquad \text{Example 2:} \quad C_i = A_{i,k} \times B_k \ \ (\int_k A_{i,k} \times B_k \, dk).$$

Their instances, as PyTorch arrays:

```
# Example 1: C_ij = A_ij * B_ij (all dimensions intervals)
# A: a0 (i=[0,2), j=[0,2), v=2) a1 (i=[3,5), j=[1,4), v=3)
# B: b0 (i=[1,4), j=[1,3), v=10) b1 (i=[0,1), j=[0,2), v=20)
AIs, AIe = tensor([0., 3.]), tensor([2., 5.])
AJs, AJe = tensor([0., 1.]), tensor([2., 4.])
AV = tensor([2., 3.])
BIs, BIe = tensor([1., 0.]), tensor([4., 1.])
```



inputs A (blue) and B (red) output C

Figure 7.2: Example 1. Left: the pieces of A and B in the (i, j) plane, labeled with their values. Right: the output $C_{i,j} = A_{i,j} \times B_{i,j}$, one piece per intersecting pair of input pieces; a_1 and b_1 do not intersect.

```
BJs, BJe = tensor([1., 0.]), tensor([3., 2.])
BV = tensor([10., 20.])

# Example 2: C_i = int_k A_ik * B_k dk
# A: a0 (i=[0,2), k=[1,4), v=2) a1 (i=[3,5), k=[0,6), v=3)
# B: b0 (k=[0,1), v=10) b1 (k=[2,5), v=20)
AIs, AIe = tensor([0., 3.]), tensor([2., 5.])
AKs, AKe = tensor([1., 0.]), tensor([4., 6.])
AV = tensor([2., 3.])
BKs, BKe = tensor([0., 2.]), tensor([1., 5.])
BV = tensor([10., 20.])
```

Figures 7.2 and 7.3 draw the two instances together with the outputs they produce. In Example 1 the output pieces are the pairwise intersections of the input boxes, three of them, since a_1 and b_1 do not overlap. In Example 2 the output is a piecewise-constant function of i: on $[3, 5)$ the piece a_1 meets both pieces of B, and the two contributions $3 \cdot 10 \cdot 1 = 30$ and $3 \cdot 20 \cdot 3 = 180$ merge to 210.

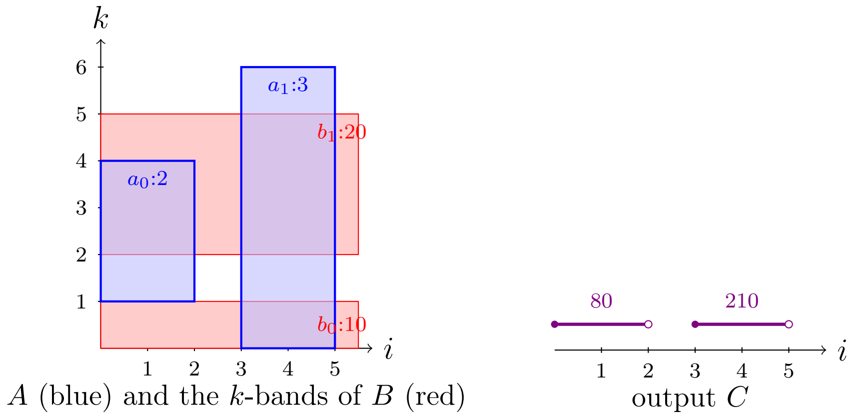


Figure 7.3: Example 2. Left: the pieces of A in the (i, k) plane and the pieces of B, which depend only on k, drawn as horizontal bands. Right: the output $C_i = \int_k A_{i,k} \times B_k \, dk$, two pieces on the i axis labeled with their values; on $[3, 5)$ the two contributions 30 and 180 merge to 210.

7.3.1 Constructing the Mask with Continuous Tensors

In the discrete setting, the mask is a binary tensor over the nonzero positions: entry (a, b) tells whether nonzero a of A and nonzero b of B interact, with one coordinate-equality factor per shared variable. Two things change in the continuous setting. First, the equality test becomes an interval intersection test. Second, the mask is no longer binary: when the Einsum reduces over a continuous variable, the mask entry stores the *length* of the intersection on that variable instead of a 0 or 1. The reason is the integral. Over the region where two constant pieces overlap, the integral of their product is the product of the two values times the length of the overlap; the values are multiplied in the product step, so the mask is the natural place to carry the length.

Both pieces are elementary. Two intervals $[s, e)$ and $[s', e')$ overlap exactly when each starts before the other ends,

$$s < e' \quad \text{and} \quad s' < e,$$

and when they do, the length of their intersection is $\min(e, e') - \max(s, s')$. The mask is then built variable by variable: every shared variable contributes its two overlap inequalities, and a reduced variable additionally contributes its intersection length. Written out in full for the two examples,

$$\text{Example 1:}\quad \text{Mask}_{a,b} = [\,AIs_a < BIe_b\,]\times[\,BIs_b < AIe_a\,]\times[\,AJs_a < BJe_b\,]\times[\,BJs_b < AJe_a\,],$$

$$\text{Example 2:}\quad \text{Mask}_{a,b} = [\,AKs_a < BKe_b\,]\times[\,BKs_b < AKe_a\,]\times\big(\min(AKe_a, BKe_b) - \max(AKs_a, BKs_b)\big).$$

In Example 1 both shared variables, i and j, appear on the output, so the mask is a product of four indicators and stays binary as before. In Example 2 the shared k is reduced, so the two inequalities are joined by the overlap length, and the mask holds the k-overlap lengths directly.

```
# Mask (Example 1): i and j output -> two inequalities per variable
M = (AIs[:,None] < BIe[None,:]) & (BIs[None,:] < AIe[:,None]) \
  & (AJs[:,None] < BJe[None,:]) & (BJs[None,:] < AJe[:,None])
# [[1,1], [1,0]]

# Mask (Example 2): k reduced -> inequalities times overlap length
M = ((AKs[:,None] < BKe[None,:]) & (BKs[None,:] < AKe[:,None])) \
  * (torch.minimum(AKe[:,None], BKe[None,:])
   - torch.maximum(AKs[:,None], BKs[None,:]))
# [[0,2], [1,3]]
```

The masks match Figures 7.2 and 7.3. In Example 1 the only zero is (a_1, b_1), the pair of boxes that do not overlap. In Example 2 the pair (a_0, b_0) is dead because $[1, 4) \cap [0, 1)$ is empty, and the live entries hold the lengths the integral needs: for (a_1, b_1), the overlap $[0, 6) \cap [2, 5) = [2, 5)$ has length 3. As in the discrete case, a mask entry of 0 means no interaction; the difference is only that live entries can now carry a weight other than 1.

In practice, as in the discrete case, we do not compare all pairs to build this mask. The three strategies of Section 7.2.1 carry over, dense mask creation, binary search, and the database port, with the equality join replaced by an inequality join: the join condition is the pair of overlap inequalities above rather than an equality on keys.

7.3.2 Product with Continuous Tensors

The product step again does two things: it computes each candidate's product value and its product coordinates.

The value is

$$\text{ProdV}_t \;=\; MV_t \times AV_{M0_t} \times BV_{M1_t},$$

where $(M0, M1, MV)$ is the mask in sparse form. The one change from the discrete case is the factor MV_t: the mask may now carry values, the intersection lengths of the reduced variables, so its COO form gains a value array. In Example 2 this factor is the k-overlap length, and multiplying it in performs the integral: over the region where two constant pieces overlap, $\int A \times B\, dk$ is the two values times the length. In Example 1 the mask is binary, MV is all ones, and the factor drops out as in the discrete case.

The product coordinates now have to be explicitly computed. In the discrete case the mask had forced the pinpoint coordinates to be equal, so the intersection reduced to reading either one. An intersection between two overlapping intervals is a new interval, whose endpoints are the maximum of the starts and the minimum of the ends, gathered through the mask positions. For Example 1, both output variables i and j are intersected,

$$\text{ProdIs}_t = \max(AIs_{M0_t},\, BIs_{M1_t}), \qquad \text{ProdIe}_t = \min(AIe_{M0_t},\, BIe_{M1_t}),$$

$$\text{ProdJs}_t = \max(AJs_{M0_t},\, BJs_{M1_t}), \qquad \text{ProdJe}_t = \min(AJe_{M0_t},\, BJe_{M1_t}),$$

For Example 2, the output variable i is carried by A alone, so its interval is simply gathered,

$$\text{ProdIs}_t = AIs_{M0_t}, \qquad \text{ProdIe}_t = AIe_{M0_t}.$$

Structurally the step has not changed: it is gathers through $M0$ and $M1$ followed by elementwise maximum, minimum, and multiply, an indirect Einsum as before.

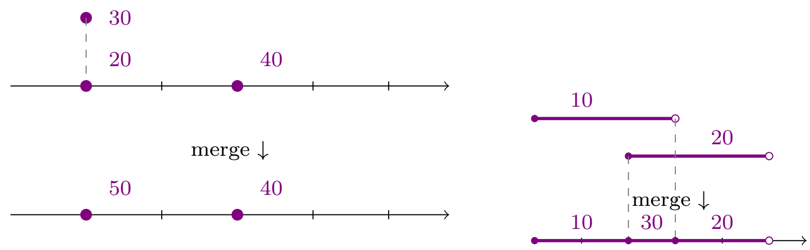
```
# Product (Example 1)
M0, M1 = M.nonzero(as_tuple=True) # mask values are all 1
ProdIs = torch.maximum(AIs[M0], BIs[M1])
ProdIe = torch.minimum(AIe[M0], BIe[M1])
ProdJs = torch.maximum(AJs[M0], BJs[M1])
ProdJe = torch.minimum(AJe[M0], BJe[M1])
ProdV = AV[M0] * BV[M1]
# ([1,2)x[1,2)):20 ([0,1)x[0,2)):40 ([3,4)x[1,3)):30

# Product (Example 2)
M0, M1 = M.nonzero(as_tuple=True)
MV = M[M0, M1] # k-overlap lengths
ProdIs, ProdIe = AIs[M0], AIe[M0]
ProdV = MV * AV[M0] * BV[M1] # k integrated by the mask
# ([0,2)):80 ([3,5)):30 ([3,5)):180
```

The candidates are the intersections drawn in Figures 7.2 and 7.3. In Example 1 the three candidate boxes happen to be disjoint, so they are already the output. In Example 2 the two candidates at $i = [3, 5)$ have identical coordinates and coexist, with values 30 and 180; merging them is the subject of the next subsection.

7.3.3 Merge with Continuous Tensors

When the Einsum has a reduction, the product step can generate several candidates on the same output region, and the merge step has to combine them. The idea is the same as in the discrete case. What changes is what "same region" means: coordinates are now intervals, so two candidates do not only collide when their coordinates are equal — they can also overlap partially. The merge therefore has to cut overlapping intervals into disjoint pieces and sum



(a) pinpoints (b) intervals

Figure 7.4: Merging candidates with pinpoint versus interval coordinates. (a) Pinpoints collide only at equal coordinates: two candidates at the same point sum. (b) Intervals overlap partially: $[0, 3)$:10 and $[2, 5)$:20 are cut at their endpoints into $[0, 2)$:10, $[2, 3)$:30, and $[3, 5)$:20.

the values on each piece. Figure 7.4 shows the two situations side by side.

We implement this merge with one simple idea: work with the position of each endpoint in the sorted list of all endpoints, instead of the coordinate values themselves. One flat unique per dimension,

$$B_o = \text{the unique values of ProdIs} \cup \text{ProdIe}, \qquad B_{\text{rs}_t} = \text{ProdIs}_t, \quad B_{\text{re}_t} = \text{ProdIe}_t,$$

gives the breakpoints B_o, which divide the line into segments $[B_o, B_{o+1})$. Because the breakpoints are the candidates' own endpoints, candidate t covers exactly the segments $\text{rs}_t, \ldots, \text{re}_t - 1$. After this step everything is integers: equal candidates have equal runs of segments, and overlapping candidates have runs that intersect. The rest of the merge is built from sorts, running sums, gathers, and scatter-adds over flat arrays, which is also the reason it is fast; we come back to this at the end of the section.

One Output Dimension: a Single Sweep

With a single interval dimension we can cut and sum in one sweep along the axis. Each candidate should place its value on every segment it covers and we implement the sweep with prefix-sum: deposit the value only at the two ends, with opposite signs,

$$\text{dval}_{\text{rs}_t} \mathrel{+}= \text{ProdV}_t, \qquad \text{dval}_{\text{re}_t} \mathrel{+}= -\text{ProdV}_t,$$

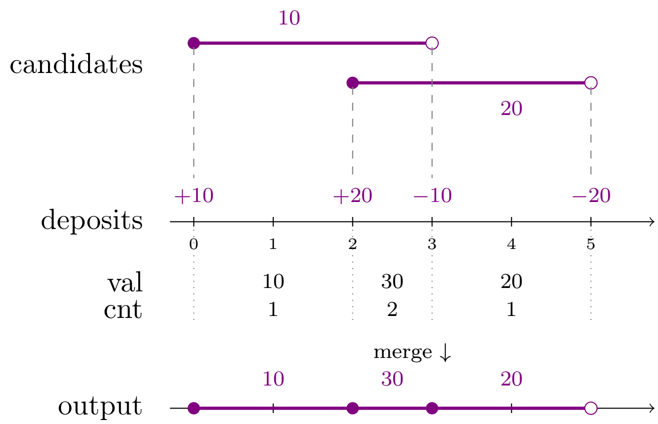


Figure 7.5: The one-dimensional sweep on two partially overlapping candidates, $[0, 3)$:10 and $[2, 5)$:20. The three segments are exactly the cut of Figure 7.4(b).

and take a running sum. The sum picks the value up where the candidate starts and drops it where it ends, so every segment in between sees it. Doing the same with ± 1 instead of the values gives a coverage count, which identifies the segments that no candidate covers — the gaps — so they can be removed from the output.

In one dimension the unique itself is not needed. We sort the $2M$ endpoints once, keep each endpoint's deposit attached, and run the sums in sorted order. Between two consecutive sorted endpoints there is either a real segment, whose value and coverage are the running sums at that point, or nothing, when the two endpoints are duplicates. A width test removes the empty ones, which is the only thing the unique was contributing. Figure 7.5 shows this sweep.

```
def merge_1d(ProdIs, ProdIe, ProdV):
    # Disjoint sorted intervals from overlapping ones, summing
    # values. One sort of 2M endpoints + two running sums.
    x = torch.cat([ProdIs, ProdIe])
    dval = torch.cat([ProdV, -ProdV]) # deposits, interleaved
    ones = torch.ones(M, dtype=torch.long)
    dcnt = torch.cat([ones, -ones]) # coverage deposits

```

```
    order = torch.argsort(x)
    xs = x[order]
    val = torch.cumsum(dval[order], dim=0)[:-1] # value between
    cnt = torch.cumsum(dcnt[order], dim=0)[:-1] # consecutive pts

    keep = (cnt > 0) & (val != 0) & (xs[1:] > xs[:-1])
    return xs[:-1][keep], xs[1:][keep], val[keep]
```

Multi-dimensional Merge

With several output dimensions, we cannot sweep one dimension at a time: two candidates can overlap in i but be disjoint in j, and then they must not be summed. Merge only happens between candidates that overlap in every dimension, so we *cut* every dimension first and merge at the end. Figure 7.6 shows the whole computation on two boxes with values 10 and 20.

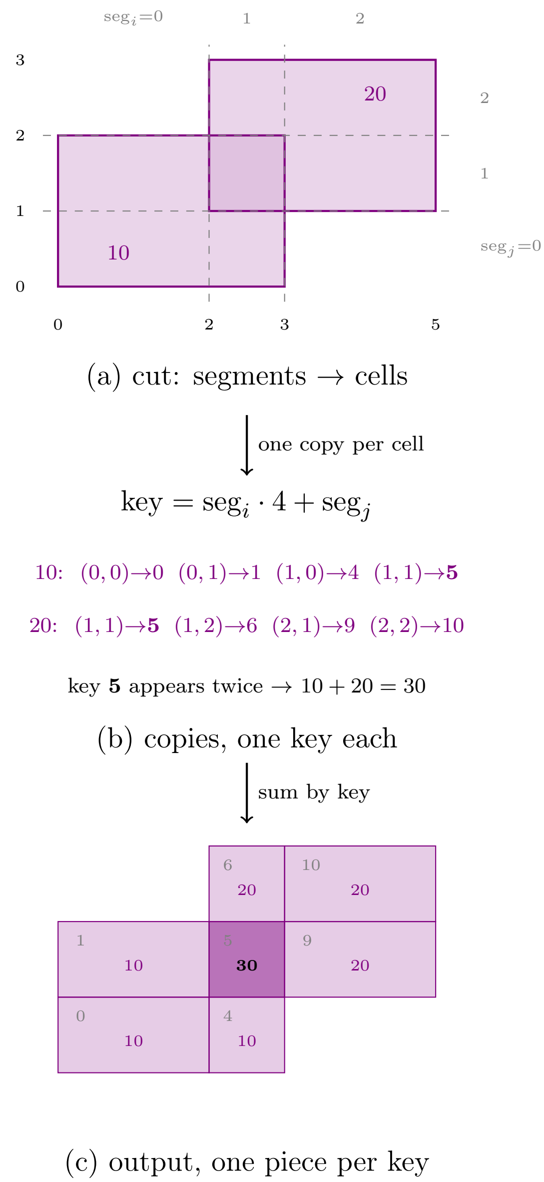


Figure 7.6: The multi-dimensional merge.

(a) Cut. The candidates' endpoints slice each dimension into segments: i is cut at $\{0, 2, 3, 5\}$ and j at $\{0, 1, 2, 3\}$, giving three segments per dimension. The segments form a grid, and each candidate covers a block of its cells.

(b) Cells and keys. Cutting replaces each candidate by one copy per covered cell, each carrying the candidate's value. A copy is identified by its cell, the pair of segment indices

$(\text{seg}_i, \text{seg}_j)$. To compare cells quickly, we squeeze the pair into a single integer,

$$\text{key} \;=\; \text{seg}_i \cdot R_j + \text{seg}_j,$$

where R_j is the number of segments along the j dimension. In the figure, the shared cell $[2, 3) \times [1, 2)$ gets key 5 from both candidates.

(c) Sum by key. One flat unique over the keys and one scatter-add finish the merge: key 5 appears twice and sums to $10 + 20 = 30$, and the seven distinct keys become the seven output pieces.

```
# Segment space: endpoint positions per dimension (one flat unique each).
B0, r = torch.unique(torch.cat([ProdIs, ProdIe]), return_inverse=True)
rs0, re0 = r[:M], r[M:] # candidate t covers i-cells [rs0_t, re0_t)
B1, r = torch.unique(torch.cat([ProdJs, ProdJe]), return_inverse=True)
rs1, re1 = r[:M], r[M:]

# Cut along i: one copy per covered i-segment.
cnt = re0 - rs0
rep = torch.repeat_interleave(torch.arange(M), cnt)
off = torch.cumsum(cnt, 0) - cnt
seg0 = rs0[rep] + torch.arange(int(cnt.sum())) - off[rep]
ProdV, rs1, re1 = ProdV[rep], rs1[rep], re1[rep]

# Cut along j: identical, yielding seg1 and replicating (ProdV, seg0).

# Discrete merge over the integer cells, packed into one key.
key = seg0 * B1.numel() + seg1
cell, inv = torch.unique(key, return_inverse=True)
CV = torch.zeros(cell.numel(), dtype=ProdV.dtype)
CV.index_add_(0, inv, ProdV)
```

```
# Unpack the kept cells back into interval coordinates.
keep = CV != 0
s0, s1 = cell[keep] // B1.numel(), cell[keep] % B1.numel()
CIs, CIe = B0[s0], B0[s0 + 1]
CJs, CJe = B1[s1], B1[s1 + 1]
CV = CV[keep]
```

The listing follows the figure line by line: the endpoints come from one flat unique per dimension, the copies from `repeat_interleave`, and the final sum from a flat unique plus `index_add`. With more than two dimensions, the key is packed two columns at a time, renumbered through unique in between so it stays small.

Chapter 8

Evaluation of the Format-Aware Compiler on Continuous Einsums

This chapter evaluates the mask–product–merge pipeline developed in the previous chapter. It proceeds in four parts. After describing the experimental setup, we first compare the three mask-creation strategies of Section 7.2.1 in isolation, then benchmark the full pipeline end to end against Finch's reference implementation of continuous Einsums, and finally break the end-to-end time into its three stages to see where the time goes.

8.1 Experimental Setup

Hardware. All experiments run on a single machine with an Intel Core i9-10900K CPU (10 cores, 20 hardware threads, 3.70 GHz) with 128 GB of RAM, and an NVIDIA GeForce RTX 3090 GPU with 24 GB of memory.

Implementation. We implement the continuous Einsums in PyTorch (version 2.9), exactly as in the listings of the previous chapter: the mask, product, and merge steps are composed from the PyTorch operations, so the same code runs unchanged on CPU and GPU. The dense and binary-search mask creation strategies are implemented in PyTorch; the database-join mask creation runs on Polars (version 1.35), using its GPU engine when the pipeline runs on the GPU. Unless stated otherwise, the end-to-end pipeline uses the binary-search builder. We evaluate three configurations of our implementation: *CPU-single* (one thread), *CPU-multi* (all 20 threads), and *GPU*.

Baseline. We compare against the original implementation of continuous Einsums, an extension of the Finch compiler [21], which generates single-threaded CPU code in Julia. Finch does not operate on COO inputs: each operand must first be converted into Finch's nested run-length (SparseRLE) levels, and multi-dimensional interval operands must additionally be fragmented at the outer dimension's endpoints to make the nesting disjoint. Because our implementation consumes COO directly while Finch requires this separate format, the full benchmark reports Finch's format-conversion time alongside its kernel time; Finch's one-time kernel compilation is excluded, as it is amortizable across runs.

Notation. Each benchmark is written as an Einsum whose superscripts give the coordinate type of every dimension: P for a pinpoint dimension and I for an interval dimension. For example, $C_x^I = \int_r A_{xr}^{II} B_r^I \, dr$ is a matrix–vector contraction in which every dimension of every operand is an interval. Whereas, $C_x^P = A_x^P \times B_x^P$ is an elementwise vector-vector multiplication in which A and B have pinpoints. All intervals in this evaluation are half-open, $[s, e)$.

Dataset. We use synthesized operands: each operand's n pieces are sampled without replacement from a regular grid of roughly $2n$ cells over $[0, 64)^D$, with each piece's intervals (or pinpoints) drawn inside its own cell, so that two operands generated from different seeds interact exactly where their sampled cells collide. A skew parameter in $[0, 1]$ biases the sampling toward one corner of the grid, which controls how many pieces of the two operands interact; unless noted, we report skew 0.5, and piece values are drawn uniformly from $[0.5, 1.5)$. Every measurement is the median of five timed runs after two warmup runs (the first warmup absorbs `torch.compile` overhead), with the GPU synchronized around every timed region.

8.2 Comparing Mask-Creation Strategies

Figure 8.1 compares the three mask builders on the GPU for the three types of join conditions generated by continuous Einsums: pure *equality* joins, where every shared dimension is represented by pinpoint coordinates on both operands; pure *inequality* joins, where every shared dimension is represented by intervals on both operands; and the *mixed* case of a pinpoint operand against an interval operand (a point-in-box test). All three builders produce identical masks.

First, the dense table wins at small n. Up to a few thousand pieces, the all-pairs comparison is implemented as a single fused kernel with no sorting or data-dependent control flow. At $n = 1{,}000$, the dense builder takes about 0.2ms across all three join types, outperforming binary search (0.3–0.7ms) and significantly outperforming the database join (2–7ms). Although the algorithm has quadratic complexity, it maps naturally to dense GPU computation, exposing abundant parallelism with regular memory access. However, the quadratic cost eventually dominates: the dense builder becomes slower than binary search around $n = 3{,}000$ and is the first approach to run out of memory, beyond approximately 10^4–10^5 pieces.

Second, the database join performs extremely well on equality joins but poorly on inequality joins. Equality joins are the workload that database engines are heavily optimized for: Polars executes them as hash joins, and for the all-pinpoint case it overtakes binary search at large n (11.9ms versus 61.3ms at $n = 10^6$). Inequality joins reverse this trend. Since the overlap predicates cannot be evaluated using a hash table, Polars's `join_where` falls back to pairwise predicate evaluation. Consequently, inequality joins perform the all pairwise comparisons, leading to substantially higher execution time than equality joins.

Although both the dense builder and Polars perform all-pairs comparisons on inequality join, the dense builder is substantially faster because it maps the computation to a dense tiled 2D tensor operation, exposing significantly more parallelism and data reuse. In contrast, Polars implements the inequality join as a parallel nested-loop kernel, assigning one thread to

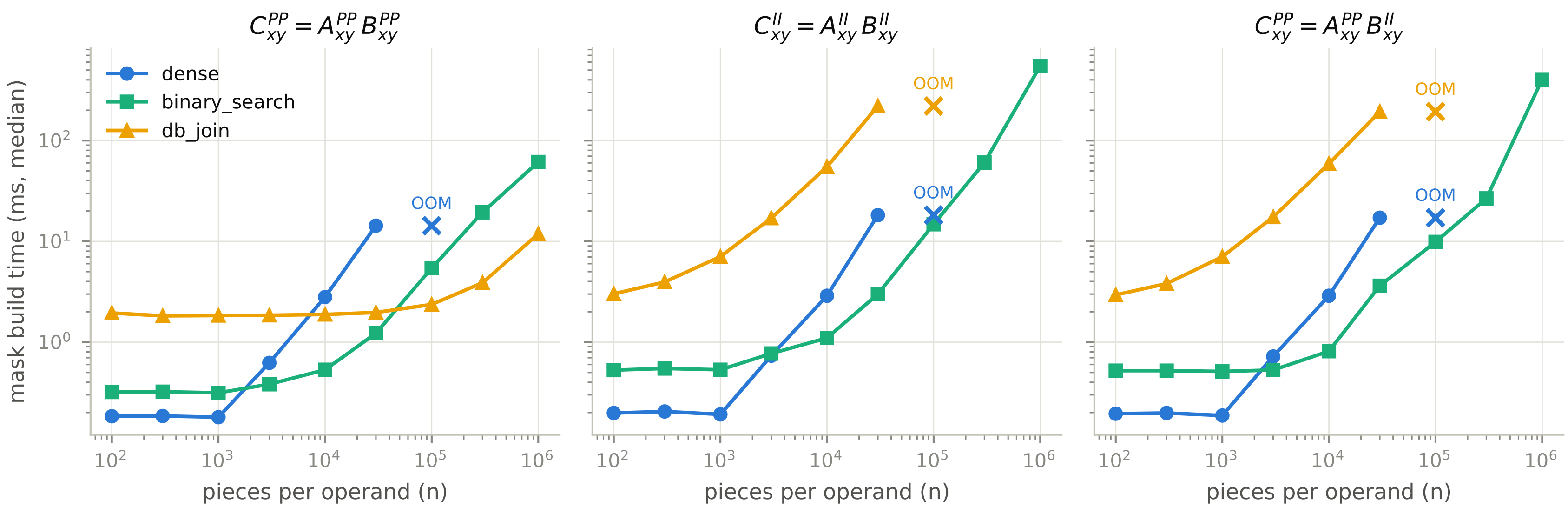


Figure 8.1: Mask creation on the GPU: the three builders of Section 7.2.1 on the three join types, as the number of pieces n grows (log–log). Left: composite equality (all pinpoints). Middle: interval–interval overlap. Right: mixed point-in-box. A line ends where the mask creation fails due to an out of memory.

Table 8.1: The seven Einsums of the full benchmark. Superscripts give each dimension's coordinate type (P pinpoint, I interval); $\int$ marks a contraction over a continuous variable.

Case	Einsum	Description
pointwise 1-D	$C_x^I = A_x^I\, B_x^I$	elementwise product of interval vectors
pointwise 2-D	$C_{xy}^{II} = A_{xy}^{II}\, B_{xy}^{II}$	box–box intersection (Example 1)
diagonal	$C_x^I = A_{xx}^{II}\, B_x^I$	diagonal extraction times a vector
matrix–vector	$C_x^I = \int_r A_{xr}^{II}\, B_r^I\, dr$	contraction, 1-D output (Example 2)
matrix–matrix	$C_{xy}^{II} = \int_r A_{xr}^{II}\, B_{ry}^{II}\, dr$	contraction, 2-D output
sampled matrix–matrix	$D_{xy}^{PP} = \int_r A_{xy}^{PP}\, B_{xr}^{II}\, C_{ry}^{II}\, dr$	three operands, mixed types
box search	$C_{xy}^{PP} = A_{xy}^{II}\, B_{xy}^{PP}$	one query box against n points

each outer-table row while scanning the inner table serially. Consequently, the dense builder achieves much higher GPU utilization despite performing the same asymptotic amount of work.

Third, binary search provides the most robust middle ground. It is never the fastest approach at small n, and on the largest equality joins it loses to the hash join, but it is the only method that completes every benchmark across all input sizes. More importantly, for interval-overlap joins (the dominant case in continuous Einsums), it is the fastest method for every problem size beyond the practical range of the dense builder. For this reason, we use binary search as the default mask builder in the full benchmark that follows.

8.3 Full Benchmark

Table 8.1 lists the seven continuous Einsums used in the full benchmark. They cover the main dimensions of the design space: with and without reductions, up to three operands, 1-D and 2-D outputs, and pinpoint, interval, and mixed coordinate types. For the contraction Einsums, we align operand intervals with the grid cells produced by the generator. With arbitrary endpoints, the exact piecewise output can grow toward the full $(2n)^2$ breakpoint grid. At our largest scale, this would mean roughly 10^{10} output pieces, which is too large for any implementation to materialize.

Figure 8.2 shows the results for all seven Einsums at $n = 10^5$ pieces per operand. We

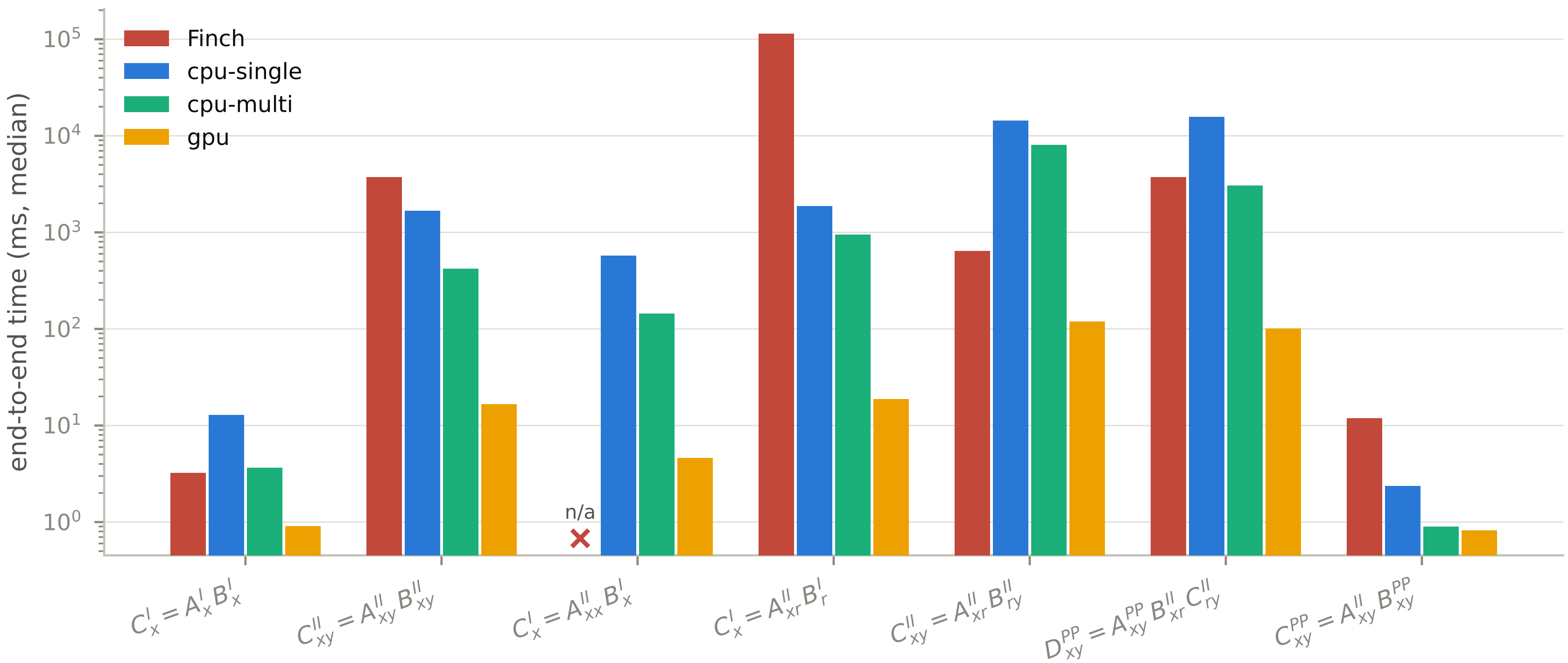


Figure 8.2: End-to-end time of the seven Einsums of Table 8.1 at $n = 10^5$ pieces per operand (log scale). The Finch bar stacks the COO-to-Finch format conversion on top of the kernel time; the diagonal case is n/a for Finch.

compare Finch with three configurations of our implementation: CPU-single, CPU-multi, and GPU. All four start from the same COO input. The Finch bars also include the COO-to-Finch format conversion described in Section 8.1. Sometimes the conversion alone takes 3.2 s, which is longer than Finch's kernel time for the Einsum $C^{II}_{xy} = A^{II}_{xy}\,B^{II}_{xy}$ (0.58 s). The Einsum $C^{I}_{x} = A^{II}_{xx}\,B^{I}_{x}$ is marked n/a for Finch because Finch cannot lower the repeated rank variable in A^{II}_{xx} through its nested continuous levels.

With one CPU thread, our Finch prototype is often faster than our format-aware compiler. Finch's generated code streams through sorted operands and merges them directly, while our compiler first materializes candidate pairs from the COO input. For the contraction Einsum $C^{II}_{xy} = A^{II}_{xr}\,B^{II}_{ry}$, Finch takes 0.61 s, while CPU-single takes 14.4 s. This is not true for every Einsum. For the contraction Einsum $C^{I}_{x} = A^{II}_{xr}\,B^{I}_{r}$, CPU-single takes 1.9 s, compared with 112 s for Finch. CPU-single was also faster for the Einsums $C^{II}_{xy} = A^{II}_{xy}\,B^{II}_{xy}$ and $D^{PP}_{xy} = A^{PP}_{xy}\,B^{II}_{xr}\,C^{II}_{ry}$. CPU-multi gives another 2–5× speedup, although Finch is still faster for some Einsums, including $C^{I}_{x} = A^{I}_{x}\,B^{I}_{x}$ and $C^{II}_{xy} = A^{II}_{xr}\,B^{II}_{ry}$.

Finally the GPU results are very different. Every stage of our implementation is built from data-parallel primitives, and the GPU version is 100–155× faster than CPU-single across all seven Einsums. It is the fastest configuration in every benchmark. Even for the contraction Einsum $C^{II}_{xy} = A^{II}_{xr}\,B^{II}_{ry}$, where Finch performs best, our GPU implementation is 5.4× faster (119 ms versus 0.64 s). For the other Einsums, the gap is often two to three orders of magnitude. For example, the Einsum $C^{II}_{xy} = A^{II}_{xy}\,B^{II}_{xy}$ takes 16.6 ms on our GPU implementation and 3.8 s in Finch.

This is not a direct apples-to-apples hardware comparison, since Finch runs on a CPU and our fastest configuration runs on a GPU. Still, it shows the main benefit of the mask–product–merge formulation. It turns continuous Einsums into regular data-parallel operations that map well to GPUs, instead of the sequential pointer-chasing merge loops generated by Finch.

8.4 Stage Breakdown

Finally, Figure 8.3 breaks down the GPU pipeline into its three stages on three representative Einsums: $C_{xy}^{II} = A_{xr}^{II}\, B_{ry}^{II}$ with free interval endpoints (a contraction whose output grows rapidly), $C_x^{I} = A_{xr}^{II}\, B_r^{I}$ (a contraction with a small output), and $C_{xy}^{II} = A_{xy}^{II}\, B_{xy}^{II}$ (pure pointwise). The figure shows where the time goes for each type of Einsum.

For the pointwise Einsum, almost all of the time is spent building the mask. At $n = 10^5$, mask creation takes 17.6 ms out of the 17.7 ms total. The product stage consists of a few gather operations (0.2 ms), and no merge is needed because the output pieces are already disjoint (Section 7.3.3).

Once a contraction is introduced, the merge stage becomes important. For $C_x^{I} = A_{xr}^{II}\, B_r^{I}$, the mask and merge take nearly the same time (15.2 ms and 15.0 ms, respectively, at $n = 10^5$). The mask produces 1.0×10^7 candidate products, which the merge reduces to 1.9×10^5 output pieces.

For $C_{xy}^{II} = A_{xr}^{II}\, B_{ry}^{II}$ with free interval endpoints, the merge dominates the runtime. At $n = 3{,}000$, it takes 132 ms out of the 174 ms total. The cut step described in Section 7.3.3 must split each candidate over every segment it overlaps, causing both the intermediate data and the final output to grow rapidly. This is why we aligned interval endpoints in the full benchmark to test $n = 100,000$.

This breakdown also explains where optimization matters. The product stage is never the bottleneck. The runtime is dominated by mask creation and merging. Section 8.2 showed that different join algorithms are preferable in different regimes for mask construction. The merge stage, meanwhile, is implemented entirely with data-parallel primitives such as sorting, prefix sums, and scatter-adds, allowing it to scale efficiently on the GPU.

Overall, these results highlight the power of the format-aware approach. Our Finch extension provides a strong single-threaded baseline through its specialized sequential code, sometimes even outperforming our multithreaded CPU implementation. The format-aware compiler goes further by operating directly on COO arrays and lowering continuous Einsums

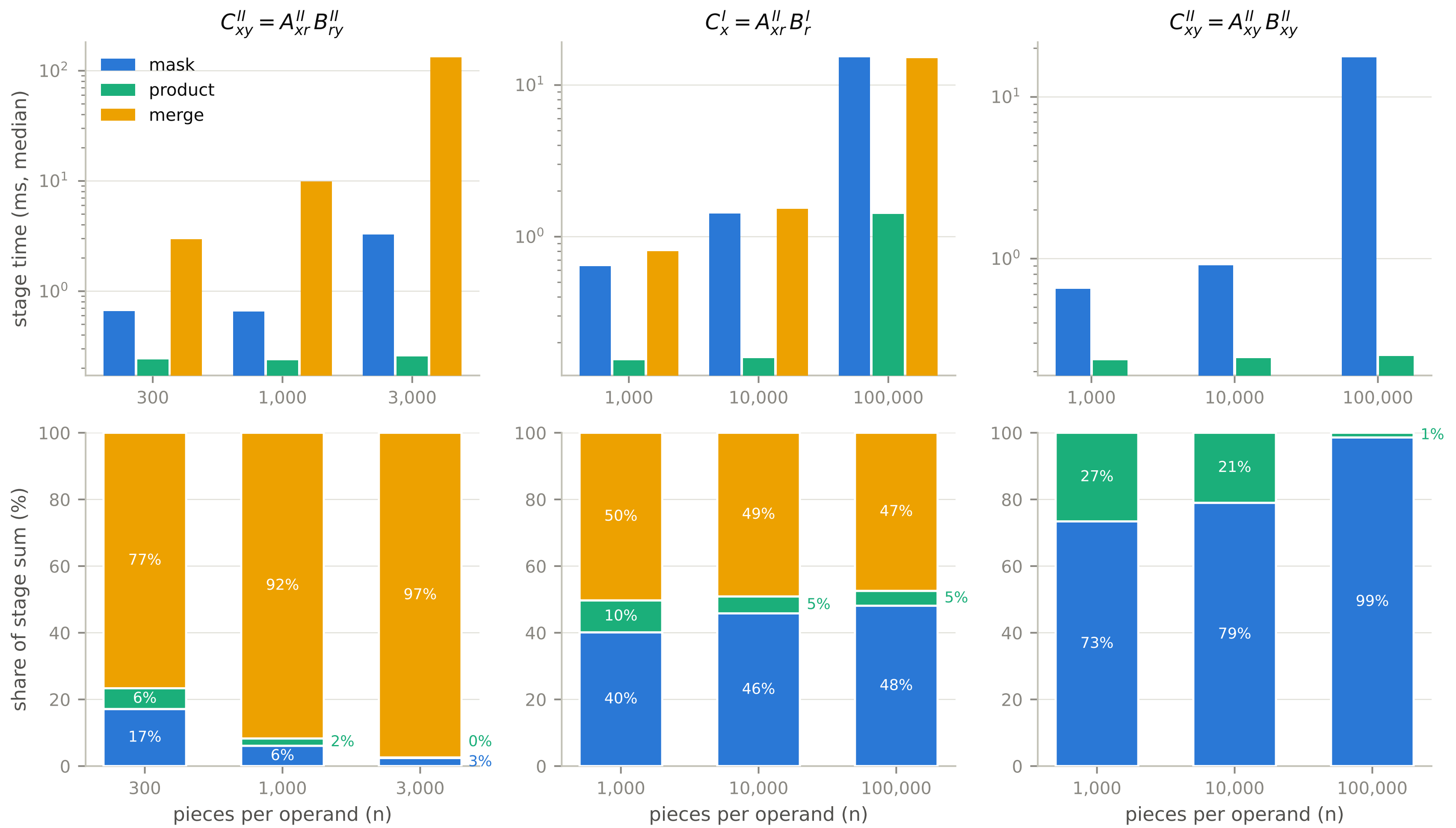


Figure 8.3: GPU time per pipeline stage for three representative cases as n grows. Mask creation and merge dominate; the product step is negligible throughout. The left most case uses free (unaligned) interval endpoints and is swept only to $n = 3{,}000$: its merged output already reaches 2.6×10^7 pieces there, and larger sizes exhaust GPU memory.

into regular mask–product–merge operations that leverage highly optimized dense tensor infrastructure. With GPU parallelism, it is the fastest implementation across all seven benchmarks, achieving a 100–155× speedup over CPU-single and at least a 5.4× speedup over Finch. Moreover, the breakdown identifies clear opportunities for further improvement in mask creation and output merging, while the product stage is already consistently negligible.

Chapter 9

Related Work

This chapter surveys the work this thesis builds on: dense tensor compilers, sparse tensor compilers, and the scattered fields that handled continuous data before it had a tensor compiler. We will study all of them in one taxonomy, so we set that up first.

9.1 Taxonomy: Compilation of Computational Graphs

Throughout this chapter, a program is a computational graph: every node is an operation, and every edge is the tensor (or a table in database) flowing from one operation to the next. This view of programs gives us three questions to ask of any system, and the whole chapter is organized around them.

- **Node language.** How does the user describe what one operation computes: an Einsum-like notation, SQL, a symbolic expression?
- **Edge language.** How does the system describe the tensor on an edge: its domain, its layout, its storage format?
- **Lowering.** How does the declarative graph become imperative code? Lowering happens in two steps:

			Lowering	
	Node language	**Edge language**	**Graph rewrites**	**Codegen**
Dense	pointwise, reduction, reshape operations; Einsum notation	dope vectors; CuTe; linear layouts	TTGT; contraction paths; algebraic rewrites	pre-optimized BLAS calls; loop generation; **native matmul**
Sparse	Einsums and their extensions; Message passing	coordinate trees; fibertree; level formats	Finch Logic; **format-aware reformulation**	merge lattices; Looplets
Continuous	symbolic expressions; spatial SQL; **continuous Einsums**	spatial indexes; **fibertree on ℝ**	symbolic simplification; SQL query planning; **format-aware reformulation**	hand-written joins; plane sweep

Table 9.1: The map of this chapter. Rows are families of systems; columns are the two languages and the two lowering steps. Bold entries are the contributions of this thesis.

1. *Graph rewrites* turn the graph into a different graph that has the same meaning but a different structure.

2. *Codegen* turns each remaining node into machine work. There are two styles here: the *library style*, which calls a pre-optimized kernel for each node, and the *loop style*, which generates a fresh loop nest specialized to the node and to the formats on its edges.

Table 9.1 is the map of this chapter, and the bold entries mark what this thesis contributes. In the dense row, native matmul lets loop-style codegen fuse computation around tensor-core matmuls. In the sparse row, format-aware reformulation (our Insum) compiles formats away into graphs of array operations. In the continuous row, the same ideas come together: continuous Einsums as the node language, level formats on ℝ as the edge language, and reformulation to reach dense tensor frameworks. The rest of the chapter walks each row and shows where every other entry came from.

9.2 Dense Tensor (Array) Compilers

Node language A dense tensor framework hands the programmer a menu of tensor operations: pointwise operations, reductions, broadcasts, reshapes, and so on. APL [5] set this declarative pattern, Matlab [6] and NumPy [11] carried it into interpreter-based interactive computing, and TensorFlow [2], PyTorch 2.0 [1], and JAX [14] made the underlying graph explicit. Almost every operation can be expressed in Einsum-like notation (the matrix sum $A + B$ is in fact $C_{ij} = A_{ij} + B_{ij}$), and the user can also state the basic Einsum explicitly through APIs such as `np.einsum`. The Tensor Contraction Engine [3], Halide [12], and TVM [13] adopt the Einsum-like expression as their algorithm language.

Edge language The classic descriptor for an array-valued edge is the dope vector [100]: a shape, a dtype, and strides. Popular tensor frameworks such as NumPy, PyTorch, and JAX still work this way, with strides as a runtime property of the tensor. Modern GPUs, however, need more than this: tensor cores want special layouts, shared memory wants swizzles to avoid bank conflicts, and registers want their own tiling. So recent work has started building serious layout languages for arrays: CuTe [101], the layout algebra behind CUTLASS, and Triton's linear layouts [102], which model a layout as a linear map over bits. Even edges carrying arrays now need a rich language as the hardware gets complicated.

Graph rewrites Graph rewriting has a long history. The IBM APL compilers [5] were already composing away chains of reshapes and transposes and fusing runs of pointwise operations in the 1980s. SPIRAL [103] built the entire compiler around rewriting: FFT-class kernels are derived by algebraic rules under measured-runtime search. TCE [3] splits a many-tensor Einsum into pairwise contractions; picking that order is NP-hard [33]. TTGT [104] rewrites a single contraction into transposes around a GEMM. TASO [105] searches verified substitutions over deep-learning graphs, and XLA [106] bakes similar rewrites, including layout assignment, into fixed passes.

Codegen Both codegen styles, calling a pre-optimized library and generating loops from scratch, are alive and well in the dense tensor world. The library style pairs a graph rewriter with tuned kernels. TTGT exists precisely so a contraction can reach pre-optimized BLAS: it transforms an arbitrary Einsum into a batched matmul. The loop style generates the nest instead. Polyhedral compilers [38,39] model loops and array accesses mathematically, allowing them to optimize loop order, fusion, tiling, and parallelism. For example, Tiramisu [39] exposes a polyhedral scheduling language for mapping computations to affine iteration space. Halide [12] famously split the algorithm from the schedule, TVM [13] extended that design, and TorchInductor [27] now writes the schedule automatically, with TorchInductor lowering the captured PyTorch graph to Triton kernels. Beyond compiler code generation, Fast and Fusiest [107] addresses accelerator design and evaluation by efficiently finding optimal fused mappings of a graph of Einsums onto a candidate hardware architecture. Each style has its own tradeoff. Generated loops can fuse and tile the operations into the right dataflow, but performance-sensitive kernels such as matmul and FlashAttention [108] still call for pre-optimized implementations, because loop codegen alone cannot yet reach that level of complexity. The native matmul support in this thesis pushes that boundary: loop-level codegen can fuse the surrounding operations into a matmul while still using the tensor cores on the GPU.

9.3 Sparse Tensor Compilers

Node language Sparse compilers mostly reuse the dense tensor compiler's node language. TACO [18] first adopted Einsum notation, with a clean division of labor: the user writes a format-agnostic Einsum, and sparsity lives entirely on the edges. The extensions came gradually. TACO-UCF [85] added affine rank-variable expressions such as A_{i+j}, which is exactly what a sparse convolution needs, and Finch [20,21] generalized the language further with indirect indexing, compound bodies, and custom reductions. Our Insum compiler takes a different approach by introducing format-aware indirect Einsums, in which the user embeds all format information directly into the computation. Graph DSLs such as GraphIt [109] and DGL [110] use the message-passing paradigm to express graph computations. The message step computes a value for each edge, the aggregation step reduces incoming edge values at each vertex, and the update step uses the aggregated result to update the vertex value.

Edge language The sparse edge language is the level format, introduced by TACO [19]. The idea is to describe a tensor as a stack of per-rank levels (called dense, compressed, and singleton in TACO), so that CSR, COO, and CSF all fall out as combinations rather than as special cases. The fibertree abstraction [25] generalizes the same idea, and Taichi [111] follows the same recipe from a different direction: for each rank of a tensor you pick a level type (dense, pointer, bitmask), and the stack defines the structure.

Graph rewrites We discuss two closely related systems here. Galley [112] plans formats and fusion together: fusing two nodes changes the iteration order, and the iteration order decides which formats are even legal, so the planning must happen on the graph before any loop exists. Our work, Insum [24], goes in a different direction: it compiles the format away. A format-agnostic Einsum becomes an array-operation DAG with format-aware indirect Einsums. We call this pattern *format-aware reformulation*: read the format-agnostic Einsums, and convert into an equivalent graph that now all tensors are stored in arrays.

Codegen The core codegen problem for sparse tensors is co-iteration: walking the intersection or union of several position lists, each stored in its own format. TACO [18] and Finch [21] developed different sparse code-generation techniques that start from format-agnostic Einsums and explicit format descriptions. TACO compiles co-iteration with merge lattices, a lattice of loop cases, one for every combination of formats that can still be active. Finch compiles it with Looplets, where an iterator is composed from small fragments such as dense runs, sparse runs, and constant runs, and codegen is rewriting on that algebra. Insum, interestingly, needs no sparse codegen at all: after its format-aware reformulation, indirect Einsums contains only array operations, so the dense tensor compiler's code generation, tensor cores included, does the work, which is exactly the payoff of format-aware computation.

9.4 Continuous Computation

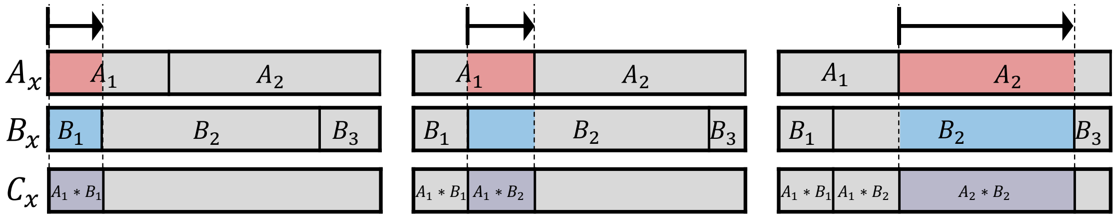
Before this thesis, continuous computation was not one community. The pieces existed, but they lived in separate worlds that did not talk to each other: computer algebra systems, spatial databases, and graphics. This section walks the same columns as before (node language, edge language, then the two lowering steps). Each paragraph first looks at what existed, then at what this thesis adds.

Node language Traditionally, two node languages existed in the continuous domain, and neither was tensor-shaped. Symbolic engines like Chebfun [113] and SymPy [114] write expressions over continuous functions, with value types richer than piecewise-constant. Spatial SQL [61] writes declarative queries with geometric predicates over tables. Sci-Vis languages such as Diderot [55] and Vivaldi [56] also compute on continuous fields, but imperatively: programmers have to evaluate fields and their derivatives by writing explicit loops [115], with no declarative graph. This thesis adds continuous Einsums [72]. It keeps the familiar Einsum-like notation, moves the indices to real values, and supports an integration. To our knowledge, it is the first Einsum-like notation over continuous domains.

Edge language On the edge side, there were plenty of data structures, but no structured language for it. The database community uses the term *spatial index* to describe data structures that organize geometries for efficient spatial queries. They have a long history: interval trees [50,63,78,116], KD-trees [117], R-trees [62,118], uniform grids [77,119], and bounding-volume hierarchies [49]. Each one helps traverse the geometries efficiently during intersection. The symbolic engines like Sympy sit at the other extreme: they have no options for data structures at all, and with no format there is no way to skip ineffectual computations, so combining two piecewise functions means comparing all pairs of pieces. This thesis extends the level format and the fibertree abstraction to $\mathbb{R}$. Interval-typed coordinates in the fibertree abstraction already cover the common indexes: a uniform grid becomes a 2-D continuous tensor with an optimized level format, and sparse grids appear the same way in our NeRF case study. Tree indexes are natural future formats: a depth-3 R-tree could be represented as a tensor with three ranks, one for each tree level. It could be an interesting research direction to study spatial indexes interpreted in the fibertree abstraction.

Graph rewrites Rewriting the computation existed too in this field, just never from a perspective of tensor programming. SymPy simplifies symbolic expressions, and spatial databases plan queries by choosing join orders and indexes. Both are real rewrite systems, but neither works on a graph of tensor operations. This thesis adds format-aware reformulation of continuous Einsums, similar to the sparse graph rewrite above: the rewrite reads the piecewise formats and emits an array-operation graph over the underlying endpoint and value arrays, and the result runs on a dense tensor framework such as PyTorch.

Codegen In spatial database, a spatial join [120] finds all intersecting pairs between two datasets; indexes prune the candidates, and the plane sweep [121,122] skips the index altogether by sorting the geometries along one axis and sweeping a line across, keeping the set of currently open intervals. These algorithms are fast and mature, but spatial databases provide them as pre-implemented operators: for each SQL query, the engine selects among available algorithms and indexes rather than generating a new, specialized spatial implementation. The original continuous-tensor work [72] adds Looplets over $\mathbb{R}$, so that the intersection or union between intervals can be generated automatically. As Figure 9.1 shows, computing $C_x = A_x \times B_x$ sweeps a real-valued rank variable x along both tensors, merging their sorted interval streams on the fly. This is precisely the plane sweep, derived rather than hand-written. We demonstrated that this generalized plane sweep on CPU often shows comparable or better performance than hand-written intersections over specialized data structures.



(a) $C_x = A_x \times B_x$, computed by sweeping a virtual line over the x-axis.

```
pA, pB, pC = 0, 0, 0
while (pA<2 and pB<3):
 lA,rA,vA = A[pA]
 lB,rB,vB = B[pB]
 lAB,rAB = max(lA,lB),min(rA,rB)
 if lAB <= rAB:
  C[pC] = (lAB, rAB, vA*vB)
  pC += 1
 pA,pB += (rA==rAB),(rB==rAB)
```

(b) Generated code.

Figure 9.1: Continuous co-iteration as a plane sweep.

Chapter 10

Conclusion and Future Directions

This thesis introduces the continuous tensor abstraction and its high-performance compilation through format-aware computation. We first define what a continuous tensor is and show that existing notation such as Einsums can be cleanly reused in the continuous domain. To make continuous tensor programs implementable while covering common use cases of continuous data, we introduce the piecewise-constant assumption and establish a format abstraction and evaluation semantics under this assumption.

To build a compiler for continuous tensor programs, we first prototyped an extension of the sparse compiler Finch to the continuous domain on CPUs and later proposed a format-aware computation both on CPUs and GPUs, which converts format-agnostic Einsums into format-aware indirect Einsums. We demonstrate that sparse computation can be compiled onto GPUs by lowering indirect Einsums into highly optimized dense tensor compilers such as PyTorch. We then extend the format-aware approach to arbitrarily many sparse operands and, finally, to continuous domains with the mask-product-merge strategy. As a result, programmers can express many applications that were difficult to express in traditional tensor programming and enjoy high performance on GPUs.

Although this thesis provides a foundation for the continuous tensor abstraction, many directions remain open for investigation.

Arbitrary Piecewise Functions Beyond Constants

Our piecewise-constant abstraction sits at one extreme of a spectrum of richer piecewise function classes—polynomials, rationals, splines, and beyond. To extend our compiler to any such piecewise-X class, we would propose reusing the same partitioning logic from piecewise-constant evaluation semantics, but each tensor piece carries a symbolic description (e.g., polynomial coefficients) rather than a single scalar. In the constant case, the reduction over a product measure admits the shortcut $c \otimes \prod_i \mu_i(I_i)$ but with non-constant integrands the compiler must instead evaluate the full nested integral: $\int_{I_1} \cdots \int_{I_n} g \; d\mu_n \cdots d\mu_1$ using closed-form anti-derivatives when available. Crucially, any chosen function class must be closed under every operations in piecewise evaluation semantics:

1. **Algebraic operations**: Adding or multiplying two pieces must stay within the class.
2. **Integration**: Integrating over any subinterval must either admit a closed-form solution or be handled by a well-defined approximation strategy.

Polynomials satisfy all properties exactly. Most other classes fail at least one step—for example, rational integrals introduce logarithms, transcendental functions often lack closed forms, and fixed-degree splines increase in degree under multiplication. In those cases, the compiler must detect the loss of closure, apply targeted approximation (e.g., numeric quadrature or projection back into the class), and propagate rigorous error bounds to guarantee overall accuracy. Supporting fully general piecewise-X functions requires careful design with incorporating a symbolic engine like SymPy [114], and is left to future work.

Arbitrary, Non-AAHR Piece Shapes

The current piecewise assumption requires pieces to be AAHRs. This covers points, intervals, and boxes, but not polygons, circles, or curves. Supporting arbitrary geometries is hard: polygon intersection requires complex algorithms such as Weiler-Atherton or Vatti clipping, and intersecting arbitrary curves becomes a root-finding problem with no closed-form solution.

One way forward, as with arbitrary function support, is to approximate geometries on a grid in the style of Adaptive Mesh Refinement, reducing the problem to AAHR-based compilation. Another is to solve the root-finding problem numerically, which would require careful design.

Need for a General Intersection Engine

Our evaluation breakdown shows that the masking stage is the bottleneck of format-aware computation with arbitrarily many sparse operands, so efficient intersection is key. Intersection over intervals is exactly an inequality join in relational algebra, which suggests that existing database engines could do this work. In our experience, however, Polars is optimized mainly for equality joins rather than inequality joins on GPU. Future work could evaluate other GPU databases and study why relational engines have not adopted the many academic techniques for inequality joins and their data structures. Alternatively, the intersection engine could be built directly with Looplets or merge lattices. The separation between the mask stage and the product-merge stage provides a clean abstraction boundary: database researchers can improve intersection while compiler researchers improve product-merge, independently.

Extending Format-Aware Computation to Arbitrary Fill Values and Operators Beyond Multiplication

Our format-aware compiler currently assumes a zero fill value and supports only multiplication. Multiplication is the easy case: zero is an annihilator, so a piece is effectual only when all operands are active, and the mask is a pure intersection. Operators such as addition and subtraction break this structure. Zero is an identity for addition, so a piece where only one operand is active still contributes to the output. The mask must therefore compute a union, which decomposes into disjoint region classes: pieces where both operands are alive, and pieces where only A or only B is alive. Each class needs its own specialized product step; the both-active class computes $M_{a,b}(AV_a+BV_b)$, while the single-operand classes reduce to masked copies. This decomposition mirrors merge lattices in discrete sparse compilation, where each lattice point yields a simplified expression for one region class. A natural generalization is to emit one indirect Einsum per lattice point and merge the results. Arbitrary fill values raise a related question: the output fill becomes $f(\mathit{fill}_A, \mathit{fill}_B)$, and whether single-operand regions can be skipped depends on whether the other operand's fill is an annihilator or identity for the operator, as Finch handles in the discrete setting.

A Kernel Language for Sparse Applications

This thesis studies declarative languages such as Einsums and their compilation for sparse and continuous tensors. However, as GPUs and accelerators grow more complex, declarative compilers such as JAX and the PyTorch compiler cannot always reach peak performance even on dense tensors. The field is therefore moving toward lower-level, imperative kernel languages such as Triton, Pallas, CuteDSL, cuTile, and Helion. These languages target dense tensors. We believe a similar kernel language is possible for sparse applications. The key property is a kernel that is imperative yet format-agnostic: the user controls the schedule, while the format remains abstract. Such a language would let the system automatically explore different formats and their corresponding implementations from a single kernel, just as format-aware computation separates algorithm from format at the declarative level.

References

[1] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. "Pytorch: An imperative style, high-performance deep learning library." *Advances in neural information processing systems*, **32**, 2019. URL: https://arxiv.org/abs/1912.01703.

[2] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, et al. "TensorFlow: a system for Large-Scale machine learning." In: *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 2016, pp. 265–283.

[3] G. Baumgartner et al. "Synthesis of High-Performance Parallel Programs for a Class of ab Initio Quantum Chemistry Models." *Proceedings of the IEEE*, **93**(2), 2005, pp. 276–292. DOI: 10.1109/JPROC.2004.840311.

[4] J. W. Backus et al. "The FORTRAN Automatic Coding System." In: *Papers Presented at the February 26–28, 1957, Western Joint Computer Conference: Techniques for Reliability*. IRE-AIEE-ACM '57 (Western). 1957, pp. 188–198. DOI: 10.1145/1455567.1455599.

[5] K. E. Iverson. "A programming language." In: *Proceedings of the May 1-3, 1962, spring joint computer conference*. 1962, pp. 345–351. DOI: 10.1145/1460833.1460872.

[6] T. M. Inc. *MATLAB version: 9.13.0 (R2022b)*. Natick, Massachusetts, United States, 2022. URL: https://www.mathworks.com.

[7] B. W. Kernighan and D. M. Ritchie. *The C Programming Language*. 2nd ed. Prentice Hall, 1988. ISBN: 978-0-13-110362-7.

[8] B. Stroustrup. *The C++ programming language*. Pearson Education, 2013.

[9] G. van Rossum. *Python Reference Manual*. Tech. rep. CS-R9525. Amsterdam, The Netherlands: Centrum voor Wiskunde en Informatica (CWI), May 1995.

[10] J. J. Gosling, B. Joy, G. L. Steele, G. Bracha, and A. Buckley. *The Java Language Specification, Java SE 8 Edition*. Addison-Wesley Professional, 2014. ISBN: 978-0-13-390069-9.

[11] C. R. Harris, K. J. Millman, S. J. Van Der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al. "Array programming with NumPy." *Nature*, **585**(7825), 2020, pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[12] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe. "Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines." *Acm Sigplan Notices*, **48**(6), 2013, pp. 519–530. DOI: 10.1145/2491956.2462176.

[13] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, et al. "TVM: An automated End-to-End optimizing compiler for deep learning." In: *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 2018, pp. 578–594. URL: https://www.usenix.org/conference/osdi18/presentation/chen.

[14] J. Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018. URL: http://github.com/jax-ml/jax.

[15] A. Shehabi, S. J. Smith, A. Hubbard, A. Newkirk, N. Lei, M. A. B. Siddik, B. Holecek, J. Koomey, E. Masanet, and D. Sartor. *2024 United States Data Center Energy Usage Report*. Tech. rep. LBNL-2001637. Berkeley, California: Lawrence Berkeley National

Laboratory, Dec. 2024. DOI: 10.71468/P1WC7Q. URL: https://doi.org/10.71468/P1WC7Q.

[16] J. J. Dongarra, P. Luszczek, and A. Petitet. "The LINPACK Benchmark: Past, Present and Future." *Concurrency and Computation: Practice and Experience*, **15**(9), 2003, pp. 803–820. DOI: 10.1002/cpe.728. URL: https://doi.org/10.1002/cpe.728.

[17] J. Dongarra, M. A. Heroux, and P. Luszczek. "High-Performance Conjugate-Gradient Benchmark: A New Metric for Ranking High-Performance Computing Systems." *The International Journal of High Performance Computing Applications*, **30**(1), 2016, pp. 3–10. DOI: 10.1177/1094342015593158. URL: https://doi.org/10.1177/1094342015593158.

[18] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe. "The tensor algebra compiler." *Proceedings of the ACM on Programming Languages*, **1**(OOPSLA), 2017, pp. 1–29. DOI: 10.1145/3133901.

[19] S. Chou, F. Kjolstad, and S. Amarasinghe. "Format abstraction for sparse tensor algebra compilers." *Proceedings of the ACM on Programming Languages*, **2**(OOPSLA), 2018, pp. 1–30. DOI: https://doi.org/10.1145/3276493.

[20] W. Ahrens, D. Donenfeld, F. Kjolstad, and S. Amarasinghe. "Looplets: A Language for Structured Coiteration." In: *Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization*. 2023, pp. 41–54. DOI: https://doi.org/10.1145/3579990.3580020.

[21] W. Ahrens, T. F. Collin, R. Patel, K. Deeds, C. Hong, and S. Amarasinghe. "Finch: Sparse and Structured Tensor Programming with Control Flow." *Proc. ACM Program. Lang.*, **9**(OOPSLA1), Apr. 2025. DOI: 10.1145/3720473. URL: https://doi.org/10.1145/3720473.

[22] A. R. Quinlan and I. M. Hall. "BEDTools: a flexible suite of utilities for comparing genomic features." *Bioinformatics*, **26**(6), 2010, pp. 841–842. DOI: 10.1093/bioinformatics/btq033.

[23] H. Tang, Z. Liu, X. Li, Y. Lin, and S. Han. "Torchsparse: Efficient point cloud inference engine." *Proceedings of Machine Learning and Systems*, **4**, 2022, pp. 302–315.

[24] J. Won, W. Ahrens, S. Amarasinghe, and J. S. Emer. "Insum: Sparse GPU Kernels Simplified and Optimized with Indirect Einsums." In: *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '26)*. 2026. DOI: 10.1145/3779212.3790176.

[25] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer. "Efficient processing of deep neural networks." *Synthesis Lectures on Computer Architecture*, **15**(2), 2020, pp. 1–341. DOI: https://doi.org/10.1007/978-3-031-01766-7.

[26] N. Nayak, T. O. Odemuyiwa, S. Ugare, C. Fletcher, M. Pellauer, and J. Emer. "TeAAL: A Declarative Framework for Modeling Sparse Tensor Accelerators." In: *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. MICRO '23. Toronto, ON, Canada: Association for Computing Machinery, 2023, pp. 1255–1270. ISBN: 9798400703294. DOI: 10.1145/3613424.3623791. URL: https://doi.org/10.1145/3613424.3623791.

[27] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, et al. "Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation." In: *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 2024, pp. 929–947.

[28] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko. "MLIR: Scaling compiler infrastructure for domain specific computation." In: *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE. 2021, pp. 2–14.

[29] A. Einstein. "The foundation of the general theory of relativity." *Annalen Phys.*, **49**(7), 1916. Ed. by J.-P. Hsu and D. Fine, pp. 769–822. DOI: 10.1002/andp.19163540702.

[30] T. O. Odemuyiwa, J. S. Emer, and J. D. Owens. *The EDGE Language: Extended General Einsums for Graph Algorithms*. en. arXiv:2404.11591 [cs]. Apr. 2024. URL: http://arxiv.org/abs/2404.11591 (visited on 04/23/2024).

[31] R. Henry, O. Hsu, R. Yadav, S. Chou, K. Olukotun, S. Amarasinghe, and F. Kjolstad. "Compilation of sparse array programming models." *Proceedings of the ACM on Programming Languages*, **5**(OOPSLA), 2021, pp. 1–29. DOI: https://doi.org/10.1145/3485505.

[32] T. A. Davis. "Algorithm 1000: SuiteSparse:GraphBLAS: Graph Algorithms in the Language of Sparse Linear Algebra." *ACM Trans. Math. Softw.*, **45**(4), 2019. ISSN: 0098-3500. DOI: 10.1145/3322125. URL: https://doi.org/10.1145/3322125.

[33] D. G. a. Smith and J. Gray. "opt_einsum - A Python package for optimizing contraction order for einsum-like expressions." *Journal of Open Source Software*, **3**(26), 2018, p. 753. DOI: 10.21105/joss.00753. URL: https://doi.org/10.21105/joss.00753.

[34] E. Mutlu, R. Tian, B. Ren, S. Krishnamoorthy, R. Gioiosa, J. Pienaar, and G. Kestor. "Comet: A domain-specific compilation of high-performance computational chemistry." In: *International Workshop on Languages and Compilers for Parallel Computing*. Springer. 2020, pp. 87–103.

[35] N. Corporation. "NVIDIA A100 Tensor Core GPU: Performance and Innovation." *IEEE Micro*, **41**(2), 2021, pp. 29–35. DOI: 10.1109/MM.2021.3061375.

[36] A. Adams, K. Ma, L. Anderson, R. Baghdadi, T.-M. Li, M. Gharbi, B. Steiner, S. Johnson, K. Fatahalian, F. Durand, et al. "Learning to optimize halide with tree search and random programs." *ACM Transactions on Graphics (TOG)*, **38**(4), 2019, pp. 1–12.

[37] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, et al. "Ansor: Generating High-Performance tensor programs for deep learning."

In: *14th USENIX symposium on operating systems design and implementation (OSDI 20)*. 2020, pp. 863–879.

[38] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. “A practical automatic polyhedral parallelizer and locality optimizer.” In: *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 2008, pp. 101–113.

[39] R. Baghdadi, J. Ray, M. B. Romdhane, E. Del Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe. “Tiramisu: A polyhedral compiler for expressing fast and portable code.” In: *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE. 2019, pp. 193–205.

[40] P. Tillet, H.-T. Kung, and D. Cox. “Triton: an intermediate language and compiler for tiled neural network computations.” In: *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 2019, pp. 10–19.

[41] A. Bik, P. Koanantakool, T. Shpeisman, N. Vasilache, B. Zheng, and F. Kjolstad. “Compiler support for sparse tensor computations in MLIR.” *ACM Transactions on Architecture and Code Optimization (TACO)*, **19**(4), 2022, pp. 1–25.

[42] K. Hegde, H. Asghari-Moghaddam, M. Pellauer, N. Crago, A. Jaleel, E. Solomonik, J. Emer, and C. W. Fletcher. “Extensor: An accelerator for sparse tensor algebra.” In: *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*. 2019, pp. 319–333.

[43] Y. N. Wu, P.-A. Tsai, A. Parashar, V. Sze, and J. S. Emer. “Sparseloop: An analytical approach to sparse tensor accelerator modeling.” In: *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE. 2022, pp. 1377–1395.

[44] R. Senanayake, C. Hong, Z. Wang, A. Wilson, S. Chou, S. Kamil, S. Amarasinghe, and F. Kjolstad. “A sparse iteration space transformation framework for sparse tensor

algebra." *Proceedings of the ACM on Programming Languages*, **4**(OOPSLA), 2020, pp. 1–30. DOI: https://doi.org/10.1145/3428226.

[45] NVIDIA Corporation. *cuSPARSE: A CUDA Library for Sparse Matrix Computations*. https://docs.nvidia.com/cuda/cusparse/. Accessed: 2025-04-10.

[46] T. Gale, M. Zaharia, C. Young, and E. Elsen. "Sparse gpu kernels for deep learning." In: *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE. 2020, pp. 1–14.

[47] R. Tian, L. Guo, J. Li, B. Ren, and G. Kestor. "A high-performance sparse tensor algebra compiler in multi-level IR." *arXiv preprint arXiv:2102.05187*, 2021.

[48] Z. Ye, R. Lai, J. Shao, T. Chen, and L. Ceze. "Sparsetir: Composable abstractions for sparse compilation in deep learning." In: *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 2023, pp. 660–678.

[49] J. T. Klosowski, M. Held, J. S. Mitchell, H. Sowizral, and K. Zikan. "Efficient collision detection using bounding volume hierarchies of k-DOPs." *IEEE transactions on Visualization and Computer Graphics*, **4**(1), 1998, pp. 21–36. DOI: https://doi.org/10.1109/2945.675649.

[50] A. V. Alekseyenko and C. J. Lee. "Nested Containment List (NCList): a new algorithm for accelerating interval query of genome alignment and interval databases." *Bioinformatics*, **23**(11), 2007, pp. 1386–1393. DOI: https://doi.org/10.1093/bioinformatics/btl647.

[51] Y. Chen, J. Liu, X. Zhang, X. Qi, and J. Jia. "Largekernel3d: Scaling up kernels in 3d sparse cnns." In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2023, pp. 13488–13498. DOI: 10.1109/CVPR52729.2023.01296.

[52] B. Graham, M. Engelcke, and L. Van Der Maaten. “3d semantic segmentation with submanifold sparse convolutional networks.” In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 9224–9232. DOI: 10.1109/CVPR.2018.00961.

[53] B.-S. Hua, M.-K. Tran, and S.-K. Yeung. “Pointwise convolutional neural networks.” In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 984–993. DOI: 10.1109/CVPR.2018.00109.

[54] H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas. “Kpconv: Flexible and deformable convolution for point clouds.” In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 6411–6420. DOI: 10.1109/ICCV.2019.00651.

[55] G. Kindlmann, C. Chiw, N. Seltzer, L. Samuels, and J. Reppy. “Diderot: a Domain-Specific Language for Portable Parallel Scientific Visualization and Image Analysis.” *IEEE Transactions on Visualization and Computer Graphics*, **22**(1), 2016, pp. 867–876. DOI: 10.1109/TVCG.2015.2467449.

[56] H. Choi, W. Choi, T. M. Quan, D. G. C. Hildebrand, H. Pfister, and W.-K. Jeong. “Vivaldi: A Domain-Specific Language for Volume Processing and Visualization on Distributed Heterogeneous Systems.” *IEEE Transactions on Visualization and Computer Graphics*, **20**(12), 2014, pp. 2407–2416. DOI: 10.1109/TVCG.2014.2346322.

[57] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa. “Plenoxels: Radiance fields without neural networks.” In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 5501–5510. DOI: 10.1109/CVPR52688.2022.00542.

[58] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. “Nerf: Representing scenes as neural radiance fields for view synthesis.” *Communications of the ACM*, **65**(1), 2021, pp. 99–106. DOI: 10.1145/3507906.

[59] GEOS contributors. *GEOS coordinate transformation software library*. Open Source Geospatial Foundation. 2021. URL: https://libgeos.org/.

[60] S. Gillies, C. van der Wel, J. Van den Bossche, M. W. Taves, J. Arnott, B. C. Ward, et al. *Shapely*. Version 2.0.2. Oct. 2023. DOI: 10.5281/zenodo.5597138. URL: https://github.com/shapely/shapely.

[61] M. J. Egenhofer. "Spatial SQL: A Query and Presentation Language." *IEEE Trans. on Knowl. and Data Eng.*, **6**(1), Feb. 1994, pp. 86–95. ISSN: 1041-4347. DOI: 10.1109/69.273029. URL: https://doi.org/10.1109/69.273029.

[62] S. T. Leutenegger, M. A. Lopez, and J. Edgington. "STR: A simple and efficient algorithm for R-tree packing." In: *Proceedings 13th international conference on data engineering*. IEEE. 1997, pp. 497–506. DOI: 10.1109/ICDE.1997.582015.

[63] J. Feng, A. Ratan, and N. C. Sheffield. "Augmented Interval List: a novel data structure for efficient genomic interval search." *Bioinformatics*, **35**(23), 2019, pp. 4907–4911. DOI: https://doi.org/10.1093/bioinformatics/btz407.

[64] A. Shaikhha, M. Huot, J. Smith, and D. Olteanu. "Functional collection programming with semi-ring dictionaries." *Proc. ACM Program. Lang.*, **6**(OOPSLA1), Apr. 2022. DOI: 10.1145/3527333. URL: https://doi.org/10.1145/3527333.

[65] T. J. Green, G. Karvounarakis, and V. Tannen. "Provenance semirings." In: *Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 2007, pp. 31–40. DOI: https://doi.org/10.1145/1265530.1265535.

[66] Y. R. Wang, S. Hutchison, J. Leang, B. Howe, and D. Suciu. "SPORES: sum-product optimization via relational equality saturation for large scale linear algebra." *Proc. VLDB Endow.*, **13**(12), July 2020, pp. 1919–1932. ISSN: 2150-8097. DOI: 10.14778/3407790.3407799. URL: https://doi.org/10.14778/3407790.3407799.

[67] J. Diestel and B. Faires. "On vector measures." *Transactions of the American Mathematical Society*, **198**, 1974, pp. 253–271. DOI: 10.1090/S0002-9947-1974-0350420-8.

[68] M. Sion. *A theory of semigroup valued measures*. Vol. 355. Springer, 2006.

[69] W. Rudin. *Real and Complex Analysis*. en. 3rd ed. McGraw-Hill series in higher mathematics. New York, NY: McGraw-Hill Professional, Sept. 1986.

[70] H. L. Royden and P. Fitzpatrick. *Real analysis*. Vol. 2. Macmillan New York, 1968.

[71] J. Won, C. Mendis, J. S. Emer, and S. Amarasinghe. "WACO: learning workload-aware co-optimization of the format and schedule of a sparse tensor program." In: *Proceedings of the 28th ACM international conference on architectural support for programming languages and operating systems, volume 2*. 2023, pp. 920–934. DOI: 10.1145/3575693.3575742.

[72] J. Won, W. Ahrens, T. F. Collin, J. S. Emer, and S. Amarasinghe. "The Continuous Tensor Abstraction: Where Indices Are Real." *Proc. ACM Program. Lang.*, **9**(OOPSLA2), Oct. 2025. DOI: 10.1145/3763146. URL: https://doi.org/10.1145/3763146.

[73] L. De Moura and N. Bjørner. "Z3: An efficient SMT solver." In: *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer. 2008, pp. 337–340. DOI: https://doi.org/10.1007/978-3-540-78800-3_24.

[74] P. A. Burrough, R. A. McDonnell, and C. D. Lloyd. *Principles of geographical information systems*. Oxford University Press, USA, 2015.

[75] S. Berchtold, C. Böhm, D. A. Keim, and H.-P. Kriegel. "A cost model for nearest neighbor search in high-dimensional data space." In: *Proceedings of the sixteenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*. 1997, pp. 78–86. DOI: https://doi.org/10.1145/263661.263671.

[76] R. H. Güting. "An introduction to spatial database systems." *the VLDB Journal*, **3**, 1994, pp. 357–399. DOI: https://doi.org/10.1007/BF01231602.

[77] J. Feng and N. C. Sheffield. "IGD: high-performance search for large-scale genomic interval datasets." *Bioinformatics*, **37**(1), 2021, pp. 118–120. DOI: 10.1093/bioinformatics/btaa1062.

[78] H. Li. *Biofast: A small benchmark for evaluating the performance of programming languages and implementations on a few common tasks in the field of Bioinformatics.* 2020. URL: https://github.com/lh3/biofast.

[79] R. K. Dale, B. S. Pedersen, and A. R. Quinlan. "Pybedtools: a flexible Python library for manipulating genomic datasets and annotations." *Bioinformatics*, **27**(24), 2011, pp. 3423–3424. DOI: https://doi.org/10.1093/bioinformatics/btr539.

[80] Open2C, N. Abdennur, G. Fudenberg, I. Flyamer, A. A. Galitsyna, A. Goloborodko, M. Imakaev, and S. V. Venev. "Bioframe: operations on genomic intervals in pandas dataframes." *bioRxiv*, 2022, pp. 2022–02. DOI: 10.1101/2022.08.26.505502.

[81] E. B. Stovner and P. Sætrom. "PyRanges: efficient comparison of genomic intervals in Python." *Bioinformatics*, **36**(3), 2020, pp. 918–919. DOI: 10.1093/bioinformatics/btz615.

[82] W. McKinney et al. "pandas: a foundational Python library for data analysis and statistics." *Python for high performance and scientific computing*, **14**(9), 2011, pp. 1–9. DOI: https://doi.org/10.5281/zenodo.3509134.

[83] L. Liu, J. Gu, K. Zaw Lin, T.-S. Chua, and C. Theobalt. "Neural sparse voxel fields." *Advances in Neural Information Processing Systems*, **33**, 2020, pp. 15651–15663. DOI: 10.48550/arXiv.2007.11571.

[84] T. Takikawa, J. Litalien, K. Yin, K. Kreis, C. Loop, D. Nowrouzezahrai, A. Jacobson, M. McGuire, and S. Fidler. "Neural geometric level of detail: Real-time rendering with implicit 3d shapes." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, pp. 11358–11367. DOI: 10.1109/CVPR46437.2021.01130.

[85] J. Won, C. Hong, C. Mendis, J. Emer, and S. Amarasinghe. "Unified Convolution Framework: A compiler-based approach to support sparse convolutions." *Proceedings of Machine Learning and Systems*, **5**, 2023, pp. 666–679.

[86] C. Choy, J. Gwak, and S. Savarese. "4d spatio-temporal convnets: Minkowski convolutional neural networks." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019, pp. 3075–3084. DOI: 10.1109/CVPR.2019.00319.

[87] J. L. Blanco and P. K. Rai. *nanoflann: a C++ header-only fork of FLANN, a library for Nearest Neighbor (NN) with KD-trees*. https://github.com/jlblancoc/nanoflann. 2014.

[88] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao. "3d shapenets: A deep representation for volumetric shapes." In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 1912–1920. DOI: 10.1109/CVPR.2015.7298801.

[89] A. James. *PyTorch 2.1: Quansight's Improvements to BSR Sparse Matrix Multiplication*. https://quansight.com/post/pytorch-2-1-quansights-improvements-to-bsr-sparse-matrix-multiplication/. Accessed: 2025-04-10. 2023.

[90] M. Geiger and T. Smidt. "e3nn: Euclidean neural networks." *arXiv preprint arXiv:2207.09453*, 2022.

[91] J. R. Rice and R. F. Boisvert. *Solving elliptic problems using ELLPACK*. Vol. 2. Springer Science & Business Media, 2012.

[92] J. Reed, Z. DeVito, H. He, A. Ussery, and J. Ansel. "torch. fx: Practical program capture and transformation for deep learning in python." *Proceedings of Machine Learning and Systems*, **4**, 2022, pp. 638–651.

[93] A. Buluc and J. R. Gilbert. "On the representation and multiplication of hypersparse matrices." In: *2008 IEEE International Symposium on Parallel and Distributed Processing*. 2008, pp. 1–11. DOI: 10.1109/IPDPS.2008.4536313.

[94] Y. Wang, B. Feng, Z. Wang, G. Huang, and Y. Ding. "TC-GNN: Bridging sparse GNN computation and dense tensor cores on GPUs." In: *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 2023, pp. 149–164.

[95] H. Tang, S. Yang, Z. Liu, K. Hong, Z. Yu, X. Li, G. Dai, Y. Wang, and S. Han. "Torchsparse++: Efficient training and inference framework for sparse convolution on gpus." In: *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 2023, pp. 225–239.

[96] K. Hong, Z. Yu, G. Dai, X. Yang, Y. Lian, N. Xu, and Y. Wang. "Exploiting hardware utilization and adaptive dataflow for efficient sparse convolution in 3D point clouds." *Proceedings of Machine Learning and Systems*, **5**, 2023, pp. 428–441.

[97] NVIDIA. *cuEquivariance: High-Performance Equivariant Neural Network Library*. https://github.com/NVIDIA/cuEquivariance. Accessed: 2025-04-10. 2025.

[98] D. Ivchenko, D. Van Der Staay, C. Taylor, X. Liu, W. Feng, R. Kindi, A. Sudarshan, and S. Sefati. "Torchrec: a pytorch domain library for recommendation systems." In: *Proceedings of the 16th ACM Conference on Recommender Systems*. 2022, pp. 482–483.

[99] R. Xu, J. Yang, Y. Xu, H. Li, X. Liu, D. Shankar, H. Zhang, M. Liu, B. Li, Y. Hu, et al. "Enhancing Performance and Scalability of Large-Scale Recommendation Systems with Jagged Flash Attention." In: *Proceedings of the 18th ACM Conference on Recommender Systems*. 2024, pp. 778–780.

[100] T. W. Pratt and M. V. Zelkowitz. "Programming Languages: Design and Implementation." In: 1975. URL: https://api.semanticscholar.org/CorpusID:60514508.

[101] C. Cecka. *CuTe Layout Representation and Algebra*. 2026. arXiv: 2603.02298 [cs.MS]. URL: https://arxiv.org/abs/2603.02298.

[102] K. Zhou et al. "Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using F_2." In: *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ASPLOS '26. Pittsburgh, PA, USA: Association for Computing Machinery, 2025, pp. 132–146. ISBN: 9798400721656. DOI: 10.1145/3760250.3762221. URL: https://doi.org/10.1145/3760250.3762221.

[103] M. Puschel et al. “SPIRAL: Code Generation for DSP Transforms.” *Proceedings of the IEEE*, **93**(2), 2005, pp. 232–275. DOI: 10.1109/JPROC.2004.840306.

[104] E. Solomonik, D. Matthews, J. Hammond, and J. Demmel. “Cyclops tensor framework: Reducing communication and eliminating load imbalance in massively parallel contractions.” In: *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*. IEEE. 2013, pp. 813–824.

[105] Z. Jia, O. Padon, J. Thomas, T. Warszawski, M. Zaharia, and A. Aiken. “TASO: optimizing deep learning computation with automatic generation of graph substitutions.” In: *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. SOSP ’19. Huntsville, Ontario, Canada: Association for Computing Machinery, 2019, pp. 47–62. ISBN: 9781450368735. DOI: 10.1145/3341301.3359630. URL: https://doi.org/10.1145/3341301.3359630.

[106] A. Sabne. “Xla: Compiling machine learning for peak performance.” *Google Res*, 2020.

[107] T. Andrulis, M. Gilbert, V. Sze, and J. S. Emer. *Fast and Fusiest: An Optimal Fusion-Aware Mapper for Accelerator Design*. 2026. arXiv: 2602.15166 [cs.AR]. URL: https://arxiv.org/abs/2602.15166.

[108] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. “FLASHATTENTION: fast and memory-efficient exact attention with IO-awareness.” In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. NIPS ’22. New Orleans, LA, USA: Curran Associates Inc., 2022. ISBN: 9781713871088.

[109] Y. Zhang, M. Yang, R. Baghdadi, S. Kamil, J. Shun, and S. Amarasinghe. “GraphIt: a high-performance graph DSL.” *Proc. ACM Program. Lang.*, **2**(OOPSLA), Oct. 2018. DOI: 10.1145/3276491. URL: https://doi.org/10.1145/3276491.

[110] M. Y. Wang. “Deep graph library: Towards efficient and scalable deep learning on graphs.” In: *ICLR workshop on representation learning on graphs and manifolds*. 2019.

[111] Y. Hu, T.-M. Li, L. Anderson, J. Ragan-Kelley, and F. Durand. "Taichi: a language for high-performance computation on spatially sparse data structures." *ACM Transactions on Graphics (TOG)*, **38**(6), 2019, pp. 1–16. DOI: 10.1145/3355089.3356506.

[112] K. Deeds, W. Ahrens, M. Balazinska, and D. Suciu. "Galley: Modern query optimization for sparse tensor programs." *Proceedings of the ACM on Management of Data*, **3**(3), 2025, pp. 1–24.

[113] Z. Battles and L. N. Trefethen. "An Extension of MATLAB to Continuous Functions and Operators." *SIAM Journal on Scientific Computing*, **25**(5), 2004, pp. 1743–1770. DOI: 10.1137/S1064827503430126.

[114] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, et al. "SymPy: symbolic computing in Python." *PeerJ Computer Science*, **3**, 2017, e103. DOI: 10.7717/peerj-cs.103.

[115] M. Pharr, W. Jakob, and G. Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann, 2016. DOI: https://doi.org/10.1016/C2013-0-15557-2.

[116] J. Enderle, M. Hampel, and T. Seidl. "Joining interval data in relational databases." In: *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*. SIGMOD '04. Paris, France: Association for Computing Machinery, 2004, pp. 683–694. ISBN: 1581138598. DOI: 10.1145/1007568.1007645. URL: https://doi.org/10.1145/1007568.1007645.

[117] J. L. Bentley. "Multidimensional binary search trees used for associative searching." *Commun. ACM*, **18**(9), Sept. 1975, pp. 509–517. ISSN: 0001-0782. DOI: 10.1145/361002.361007. URL: https://doi.org/10.1145/361002.361007.

[118] A. Guttman. "R-trees: a dynamic index structure for spatial searching." *SIGMOD Rec.*, **14**(2), June 1984, pp. 47–57. ISSN: 0163-5808. DOI: 10.1145/971697.602266. URL: https://doi.org/10.1145/971697.602266.

[119] L. Becker, K. Hinrichs, and U. Finke. “A New Algorithm for Computing Joins with Grid Files.” In: *Proceedings of the Ninth International Conference on Data Engineering*. USA: IEEE Computer Society, 1993, pp. 190–197. ISBN: 0818635703. DOI: 10.1109/ICDE.1993.344063.

[120] E. Jacox and H. Samet. “Spatial Join Techniques.” *ACM Trans. Database Syst.*, **32**, Mar. 2007, p. 7. DOI: 10.1145/1206049.1206056.

[121] J. Nievergelt and F. P. Preparata. “Plane-sweep algorithms for intersecting geometric figures.” *Commun. ACM*, **25**(10), Oct. 1982, pp. 739–747. ISSN: 0001-0782. DOI: 10.1145/358656.358681. URL: https://doi.org/10.1145/358656.358681.

[122] E. H. Jacox and H. Samet. “Iterative spatial join.” *ACM Trans. Database Syst.*, **28**(3), Sept. 2003, pp. 230–256. ISSN: 0362-5915. DOI: 10.1145/937598.937600. URL: https://doi.org/10.1145/937598.937600.